

Capstone Project-1

“Customer Profiling for UK Online Retail Industry using K-Means
Clustering and RFM Model”

In partial fulfillment of requirements for the degree of
Master of Science (MS) In Artificial Intelligence
Probability and Statistics for Artificial Intelligence (AAI-500-IN2)

Submitted by Group 9:

Ms. Nehpreet Kaur

Mr. Nikhil Bembi

Mr. Tamayi Mlanda



Under the Guidance of

Professor Azka Azka

University of San Diego

June 2025

Table of Contents

Project Overview	1
1. PROJECT OVERVIEW	1
1.1 PLATFORMS USED:	2
1.2 MOTIVATION:	3
1.3 OBJECTIVE:	3
1.4 ORGANIZATION OF THESIS:	3
Chapter 1: Introduction to the Dataset	5
Chapter 2: Data Cleaning and Preparation	6
A. Exploring the Dataset:	6
B. Checking for Missing Data in the Dataset	7
C. Key Observations:	9
Chapter 3: Exploratory Data Analysis (EDA)	12
Research Questions:	12
Descriptive Statistics and Exploratory Data Analysis	13
Chapter 4: Model Selection	21
Model 01: RFM: Recency, Frequency, and Monetary Value Segmentation	21
Model 02: K-Means Clustering	21
Model 03: Hierarchical Clustering	21
Chapter 5: Model Analysis	22
Model 01: RFM: Recency, Frequency, and Monetary Value Segmentation	22
Model 02: K-Means Clustering	34
Model 03: Hierarchical Clustering	44
Chapter 6: Conclusion and Recommendations	46
Appendix 1: Jupyter Notebook	47
Appendix 2: PowerPoint Presentation	48

Project Overview

1. PROJECT OVERVIEW

This project centers around a descriptive analysis of a UK-based Online Retail Dataset that captures real-world transactions from a giftware e-commerce business. With over 541,000 records, the dataset offers a detailed snapshot of customer purchases made through an invoice-based system.

By exploring this data, we hope to uncover meaningful insights into customer purchasing behavior, product popularity, and sales trends over time. Our goal is to identify patterns that can inform smarter business decisions—whether it's segmenting customers, spotting seasonal demand, or optimizing inventory. This analysis serves as the groundwork for more advanced exploration, potentially leading to predictive models and actionable strategies that add real value to the retail space.



Figure 1.1: Online Retail Illustration

1.1 PLATFORMS USED:

- Google Colab-



Figure 1.1.1: Google Colab Logo

Google Colab is a free cloud-based platform provided by Google that enables users to develop Python code within an interactive notebook environment.

It's widely used by data professionals, researchers, and students for various tasks including data analysis, machine learning, and creating visualizations. Because everything runs on Google's cloud servers, there's no need to set up anything on your computer. You also get access to powerful computing resources, including GPUs, which help speed up processing. Plus, sharing your work is easy — just like sharing a regular Google Doc — making teamwork and collaboration simple and seamless.

- Turnitin-



Figure 1.1.2: Turnitin Logo

Turnitin is a popular online tool that helps schools and universities make sure students' work is original. It scans submitted papers by comparing them to a huge collection of academic articles,

websites, and other materials to spot any copied content. By doing this, Turnitin supports honest work and encourages students to give proper credit to the sources they use.

1.2 MOTIVATION:

This dataset comprises over 540,000 transactions from a UK-based online gift retailer. By analyzing this data, we can uncover meaningful patterns in customer purchasing behavior and identify the most popular products. It also enables us to observe how sales fluctuate over time and vary across different regions. These insights can inform more strategic decisions regarding inventory management and marketing planning. Furthermore, gaining a deeper understanding of distinct customer segments allows for the creation of more personalized promotional campaigns. Overall, this analysis provides a solid foundation for future advanced studies, such as predictive modeling.

1.3 OBJECTIVE:

The objective of this project is:

1. Explore and summarize the dataset to understand its structure, key features, and quality.
2. Identify patterns and trends in customer purchasing behavior over time.

1.4 ORGANIZATION OF THESIS:

This report is divided into several sections to walk you through the entire analysis of the UK online gift store dataset:

1. Introduction to the Dataset

We start by explaining the background and reasons behind this project, outlining what we aim to achieve, and giving a brief overview of the dataset.

2. Data Cleaning and Preparation

Here, we describe how we cleaned the data—fixing missing information, removing duplicates, and making sure everything is ready for analysis.

3. Exploratory Data Analysis (EDA)

In this part, we dive into the data using charts and summaries to spot interesting patterns and trends.

4. Model Selection

We explain how we chose the right models to analyze the data and make predictions.

5. Model Analysis

This section shows the results from our models and what they tell us about the data.

6. Conclusion and Recommendations

Finally, we wrap up by summarizing what we found, how it can help the business, and what could be done next.

At the end, you'll find an Appendix with all the code and output from the notebook we used, so you can see exactly how the analysis was done.

Chapter 1: Introduction to the Dataset

The Online Retail dataset, available through the UCI Machine Learning Repository, offers a detailed view into actual e-commerce activity. It's frequently used in data science, machine learning, and business analytics due to its rich and practical content.

Here are the key highlights:

- ✓ **Period:** Contains transaction data collected between December 2010 and December 2011.
- ✓ **Source:** Sourced from a UK-based online retailer that specializes in selling a variety of gift items.
- ✓ **Data Size:** Includes information from over half a million transactions.
- ✓ **Key Data Fields:** The dataset features several important attributes, such as:

Invoice Number	Product Description	Quantity Purchased	Date of Purchase
Price per Item	Customer ID	Country of Purchase	

- ✓ **Potential Use Cases:**

- Understanding customer purchase behavior
- Identifying customer frequency patterns over time
- Analyzing which products perform best
- Segmenting customers into meaningful groups
- Predicting future demand and improving inventory planning

Chapter 2: Data Cleaning and Preparation

Before diving into any in-depth analysis of the Online Retail dataset, it was important to clean and organize the data carefully. This ensures that the insights drawn are accurate and meaningful. The process involved several key steps:

A. Exploring the Dataset:

We started by inspecting the initial rows of the dataset to get a clear sense of its structure and contents. Following this, we reviewed the data types for each column and assessed the dataset for any missing or incomplete entries.

What we found:

- The dataset contains more than 540,000 records, spanning 8 different fields.
- Around one-quarter of the CustomerID values were missing, which is significant.
- Few data in Description column is also incomplete or missing.

Because CustomerID is essential for monitoring individual customer transactions and behavior, we chose not to impute missing CustomerID values. Filling in these gaps could lead to inaccuracies, so it was preferable to either exclude these records or treat them with caution during the analysis.

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	12/1/2010 8:26	2.55	17850.0	United Kingdom
1	536365	71053	WHITE METAL LANTERN	6	12/1/2010 8:26	3.39	17850.0	United Kingdom
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	12/1/2010 8:26	2.75	17850.0	United Kingdom
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	12/1/2010 8:26	3.39	17850.0	United Kingdom
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	12/1/2010 8:26	3.39	17850.0	United Kingdom

Figure 2.1.1: Preview of the Dataset

B. Checking for Missing Data in the Dataset

Before starting any detailed analysis, it's important to identify any missing information in the dataset, as this can impact the accuracy of the results.

1. Counting Missing Entries:

- a) Using the command `online_retail_df.isnull().sum()`, we check missingness in the dataset.
- b) The results show that most columns are complete, with missing values in:
 - i) 1,454 missing values in the Description column.
 - ii) 135,080 missing values in the CustomerID column (25% of the data).

2. Visualizing Missing Data Patterns:

- The `msno.matrix()` function generates a nullity matrix.
- This plot visualises the missing data and shows how it is distributed in the dataset and helps to improve our understanding of the missingness.

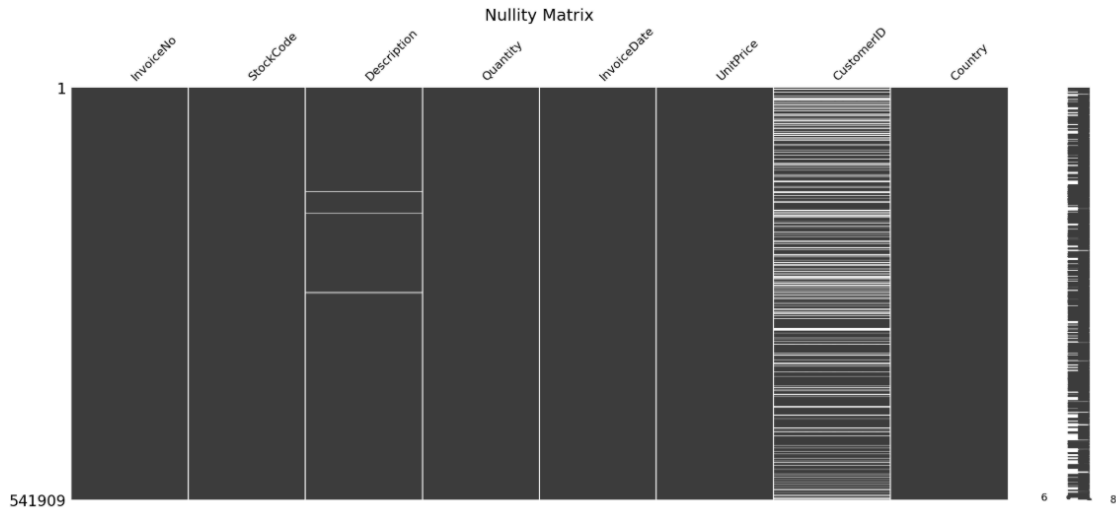


Figure 2.1.2: Nullity Matrix

3. Calculating Missing Data Percentage:

- The code works out the percentage of missing values for each column by taking the average of missing values and multiplying by 100.
- A horizontal bar chart displays which columns have missing data and how much, making it easy to compare.

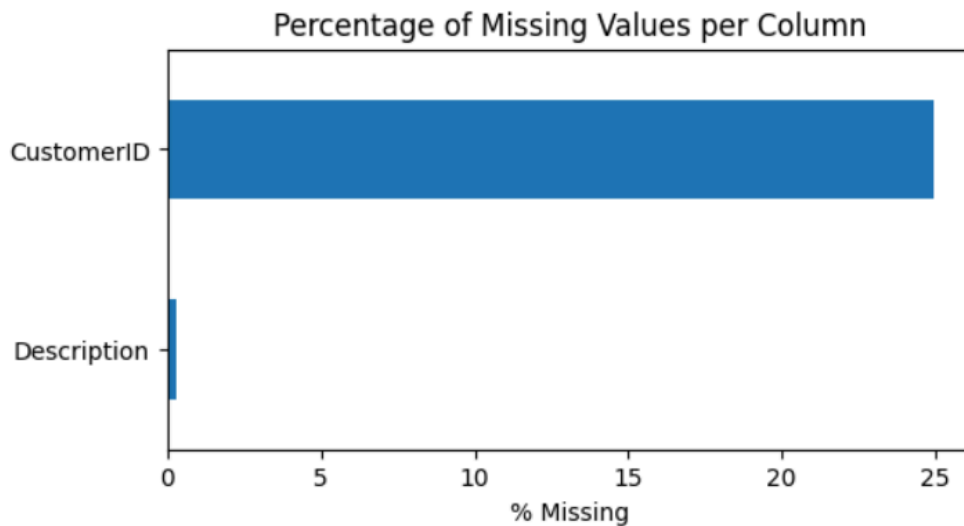


Figure 2.1.3: Missing Data Percentage

C. Key Observations:

After reviewing the initial rows, data types, and missing data, we observed that although the dataset is largely complete, several important adjustments are necessary. The InvoiceDate column requires conversion to a proper datetime format to enable time-based analyses. Additionally, the CustomerID column, currently stored as a float, should be handled as a categorical variable.

A significant concern is that approximately 25% of the CustomerID values are missing. Given that CustomerID is critical for customer-level analysis, imputing these missing values could compromise the accuracy of the results. As a result, we chose to exclude these entries from customer-focused analyses to maintain the validity and reliability of our findings.

Negative Values

- Negative Quantities:

Over 10,600 records had negative quantities, which likely represent returned products or cancelled orders. While valid from a transactional perspective, these were excluded from the main analysis focused on successful sales.

- Negative Prices:

Only two entries had negative unit prices, which could suggest input errors or refund adjustments. These were removed to maintain data integrity.

The dataset was then segmented into:

- Valid transactions, which include only positive quantities and non-cancelled entries.

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country	DateID	IsCancelled	TotalPrice
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	2010-12-01 08:26:00	2.55	17850	United Kingdom	2010- 12-01	False	15.30
1	536365	71053	WHITE METAL LANTERN	6	2010-12-01 08:26:00	3.39	17850	United Kingdom	2010- 12-01	False	20.34
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	2010-12-01 08:26:00	2.75	17850	United Kingdom	2010- 12-01	False	22.00
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	2010-12-01 08:26:00	3.39	17850	United Kingdom	2010- 12-01	False	20.34
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	2010-12-01 08:26:00	3.39	17850	United Kingdom	2010- 12-01	False	20.34

Figure 2.1.3: Valid transactions

- Cancellations/returns, retained for separate exploration of return behavior.

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country	DateID	IsCancelled	TotalPrice
141	C536379	D	Discount	-1	2010-12-01 09:41:00	27.50	14527	United Kingdom	2010-12- 01	True	-27.50
154	C536383	35004C	SET OF 3 COLOURED FLYING DUCKS	-1	2010-12-01 09:49:00	4.65	15311	United Kingdom	2010-12- 01	True	-4.65
235	C536391	22556	PLASTERS IN TIN CIRCUS PARADE	-12	2010-12-01 10:24:00	1.65	17548	United Kingdom	2010-12- 01	True	-19.80
236	C536391	21984	PACK OF 12 PINK PAISLEY TISSUES	-24	2010-12-01 10:24:00	0.29	17548	United Kingdom	2010-12- 01	True	-6.96
237	C536391	21983	PACK OF 12 BLUE PAISLEY TISSUES	-24	2010-12-01 10:24:00	0.29	17548	United Kingdom	2010-12- 01	True	-6.96
***	...	...	...	...	...	...	...	...	...	...	...

Figure 2.1.4: Cancellations Identified in the data

Outlier Detection and Handling

During the data preparation phase, special attention was given to anomalies within the key numerical fields, namely Quantity and UnitPrice. These anomalies included outliers and negative values, which can potentially distort statistical analysis and customer segmentation results.

However, since the primary objective is to conduct customer segmentation using RFM-based clustering (K-Means and Hierarchical), we chose to retain outliers in the dataset. This decision ensures that the segmentation reflects the full spectrum of customer behavior, including high-volume or high-spending transactions, which are critical for identifying top-tier customers.

While outlier detection was conducted for diagnostic purposes and is documented in a subsequent section, we deliberately avoided removing these data points to maintain an unbiased representation of Recency, Frequency, and Monetary metrics. This approach enhances the integrity of the clustering process and supports more robust and inclusive customer profiling.

Outlier Mitigation Strategies

Retail data, especially sales volume (Quantity) and revenue (TotalPrice), often contains extreme values. These can result from high-volume purchases or data entry issues.

Several methods were considered to address such outliers:

1. Filtering Cancellations and Returns:

All transactions marked as cancelled or involving negative quantities were excluded to focus the analysis on confirmed purchases.

2. Winsorization:

This method involves capping values at predefined percentile limits (e.g., 1st and 99th), reducing the influence of outliers without removing data outright.

3. IQR Filtering:

Using the Interquartile Range (IQR), values outside 1.5 times the IQR from the lower and upper quartiles were flagged as extreme and filtered accordingly.

4. Quantile-Based Outlier Detection:

Ultimately, we implemented a quantile-based method that grouped records by CustomerID and StockCode and flagged any TotalPrice values falling outside the 5th–95th percentile range. This strategy enabled us to retain useful data while removing approximately 80,000 anomalous records from the core dataset.

Chapter 3: Exploratory Data Analysis (EDA)

By summarizing, visualizing, and identifying early trends, anomalies, or correlations, exploratory data analysis (EDA) aims to provide a thorough understanding of the dataset. Strategic decision-making, statistical modeling, and hypothesis formation are all based on EDA.

To guide our analysis effectively, we propose a set of research questions rooted in the variables provided in the dataset, such as

InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID, Country, along with derived features like TotalPrice and IsCancelled.

Research Questions:

These questions are designed to facilitate inferential analysis, which aims to derive insights about the broader customer base through statistical testing and predictive modeling.

- Understand Customer Behavior and Segmentation:

Do customers from different countries have significantly different purchasing patterns (e.g. average order value or frequency)?

Cross-national purchasing behavior can inform geo-targeted marketing strategies. A one-way ANOVA can be employed to assess whether the mean TotalPrice differs significantly across countries.

- Identify Trends in Sales Over Time:

Is there a significant seasonal effect on sales volume or transaction frequency?

Recognizing temporal patterns such as seasonality or periodic spikes in sales can support inventory and marketing planning. Time series analysis methods (e.g., decomposition or moving averages) can help detect these effects.

- Insights Into Customer Retention:

Do first-time customers have a significantly lower likelihood of repeat purchases compared to returning customers?

Analyzing retention patterns can support customer loyalty program design. Customers can be segmented into cohorts (first-time vs. repeat) based on CustomerID and purchase history. Predictive models such as logistic regression or survival analysis can estimate the probability of repeat purchases.

Descriptive Statistics and Exploratory Data Analysis

This EDA provides a detailed overview of the transaction data collected from the UK-based online retail business. The dataset covers the period 01-Dec-2010, to 09-Dec-2011, and includes more than 530,000 individual transactions.

Dataset Overview and Temporal Coverage

The dataset includes sales and return transactions, with timestamps ranging from:

From: 2010-12-01 08:26:00

To: 2011-12-09 12:50:00

This confirms that the dataset captures approximately one year of business activity.

1. Country Distribution

The majority of transactions originate from the United Kingdom, which aligns with the retailer's location. The top five countries by number of transaction are:

Country	Transactions
United Kingdom	487,620
Germany	9,042
France	8,408
Ireland (EIRE)	7,894
Spain	2,485

These figures show the UK as the retailer's primary market and its reach across other European countries.

2. Summary Statistics for Numerical Features

The dataset contains two key numerical attributes: Quantity and UnitPrice. Below is a summary of each of these columns.

Metric	Quantity	UnitPrice
Count	532,619	532,619
Mean	10.24	£3.89
Standard Deviation	159.59	£35.83
Minimum	-9,600	£0.00
25th Percentile	1.0	£1.25
Median	3.0	£2.08
75th Percentile	10.0	£4.13
Maximum	80,995	£13,541.33
Interquartile Range (IQR)	9.0	£2.88

Negative quantities represent returns or cancelled transactions. Unit prices of zero may indicate promotional offers or potential data entry issues.

3. Distribution Analysis

To better understand the distributions and detect skewness and outliers, both histograms and boxplots were plotted for Quantity and UnitPrice.

Histogram Insights:

- Quantity: Displays a strong right skew, with a small subset of transactions involving large quantities.

- UnitPrice: Also exhibits a right-skewed distribution, with most prices clustered at the lower end and a few high-value items stretching the upper tail.

To reduce the impact of outliers, histograms were plotted within the 1st to 99th percentile ranges.

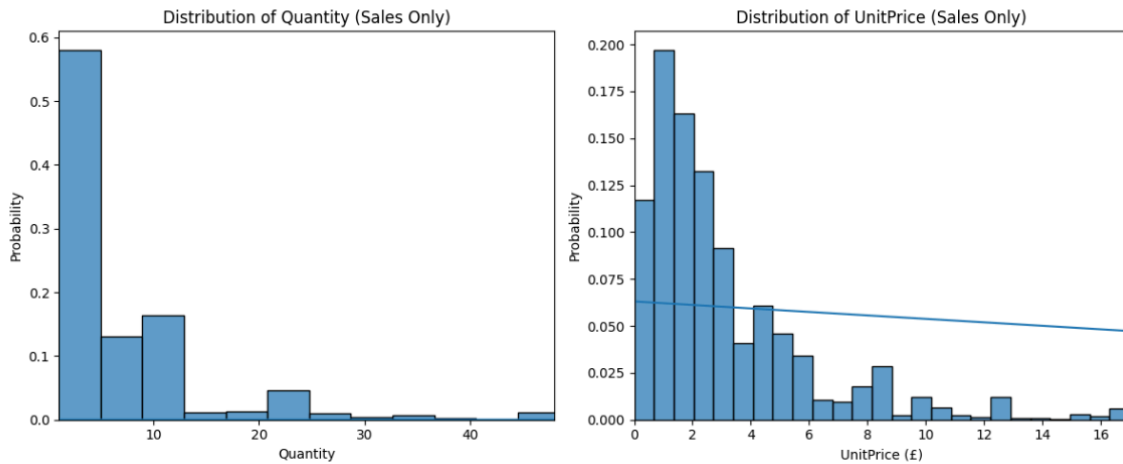


Figure 3.1: Distribution of Quantity and Unit Price

Boxplot Insights:

- Both variables include numerous outliers.
- Extreme values suggest a need for robust preprocessing in subsequent modeling (e.g., log transformation or outlier handling).

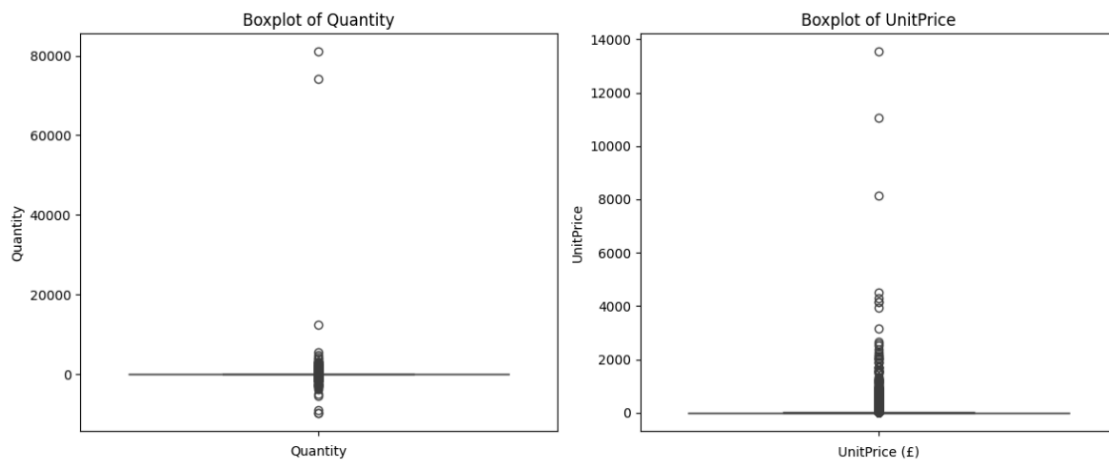


Figure 3.2: Boxplot of Quantity and Unit Price

4. Temporal Trends

To examine how transaction volume evolved, the number of transactions was aggregated monthly:

- A consistent pattern of sales is observed across months, with several spikes suggesting seasonal effects.
- Notable increases occur toward the end of the year, likely reflecting holiday season activity.

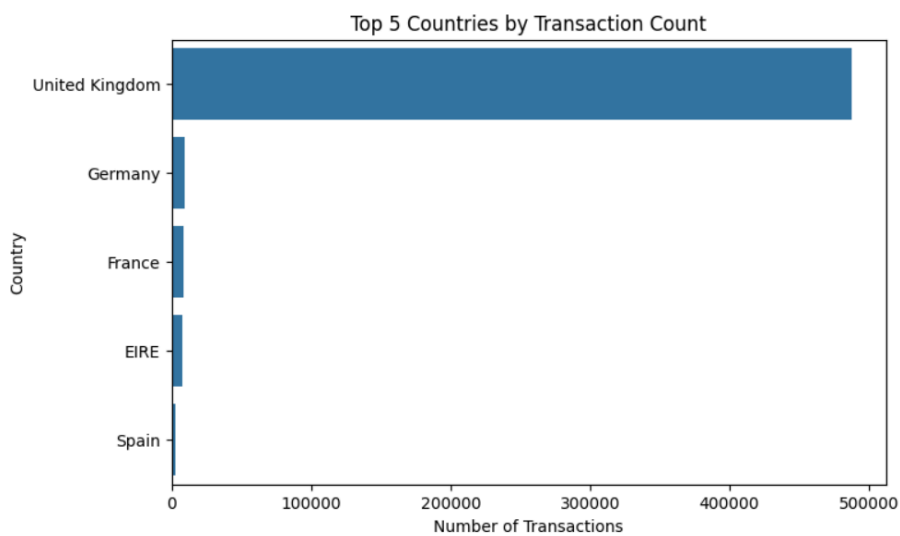


Figure 3.2: Top 5 Countries by Transaction Count

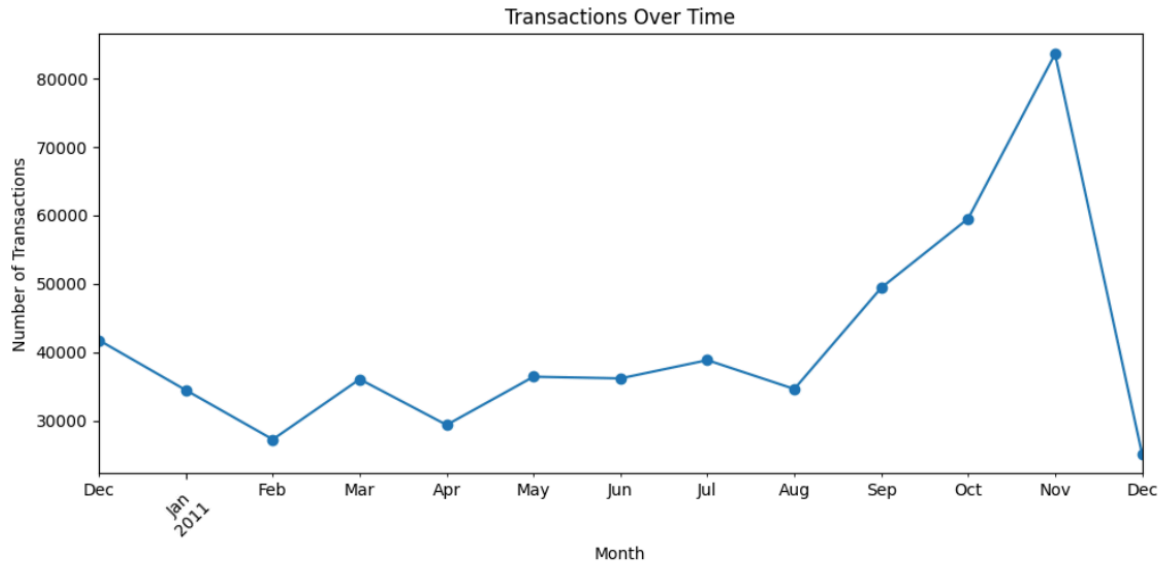


Figure 3.3: Top 5 Countries by Transaction Count

5. Daily Revenue Trend

This chart shows how much money the store made each day over time. It adds up the total revenue from all purchases made on each day. By looking at this chart, we can:

- See how earnings change from day to day
- Spot patterns, such as higher revenue during sales or holidays
- Identify slow periods where revenue drops

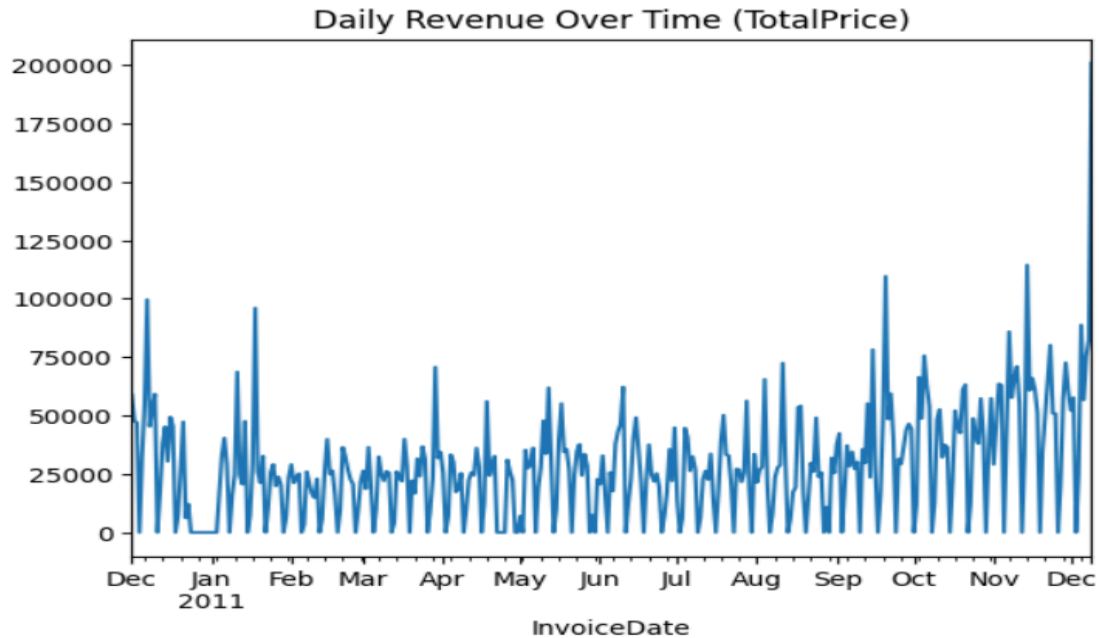


Figure 3.4: Daily Revenue Trend

6. Daily Sales Volume Trend

This chart displays the total number of items sold per day. It adds up how many units were purchased across all orders for each day. This helps us:

- Understand how daily sales change over time
- Find busy shopping days or times when fewer products were sold
- Detect regular patterns, such as weekly or seasonal trends

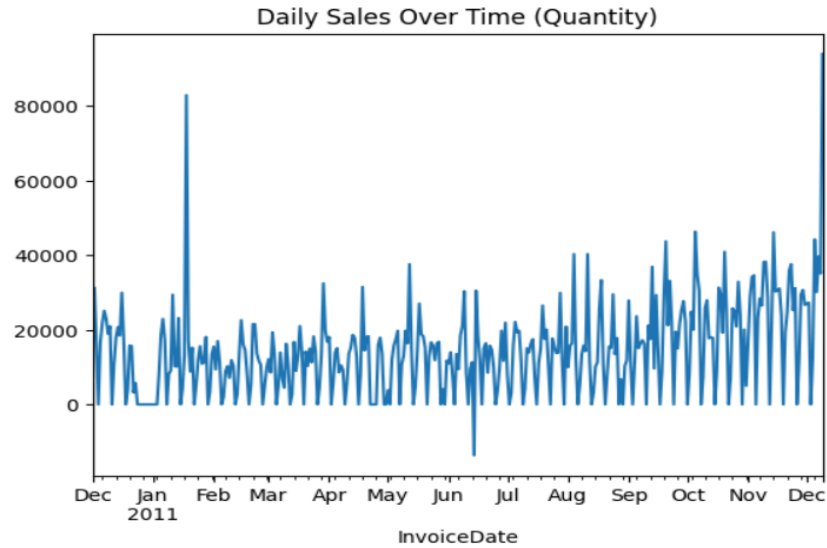


Figure 3.4: Daily Sales Volume Trend

Descriptive Statistics for Categorical Features

In addition to numerical attributes, the dataset includes several categorical variables that help describe the structure and diversity of the data. These include:

- Country: The geographic location of the customer
- StockCode: A unique identifier for each product
- CustomerID: An identifier for individual customers
- IsCancelled: A binary indicator for cancelled or returned transactions

1. Unique Values Analysis

An initial exploration of the categorical columns revealed the following:

- Country contains 38 unique values, indicating that customers placed orders from 38 different countries.
- StockCode includes 4,059 distinct products, showing a wide variety in the inventory.

- CustomerID has 4,340 unique entries, reflecting the number of individual customers recorded.
- Cancellations (via IsCancelled) sum to 0, suggesting that the filtered dataset includes only completed (non-cancelled) transactions.

This information helps assess data granularity, diversity, and coverage of customer and product activity.

2. Frequency Distribution Visualization

To gain further insight, bar charts were plotted to show the top 10 most frequent values in each categorical column:

- Top Countries: Identifies the nations with the highest number of transactions, emphasizing dominant markets such as the UK.
- Top StockCodes: Highlights the most frequently purchased products.
- Top CustomerIDs: Displays the customers with the most transactions, offering a view into repeat purchasing behavior.

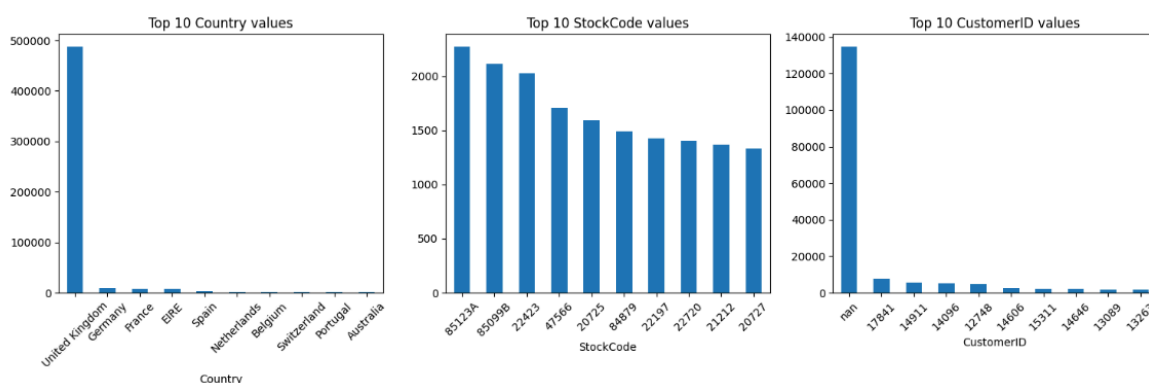


Figure 3.5: Descriptive histogram

These visual summaries are useful for detecting concentration patterns, such as heavy reliance on certain markets or products, and may help in targeted marketing or inventory optimization.

Chapter 4: Model Selection

Model 01: RFM: Recency, Frequency, and Monetary Value Segmentation

The online retail dataset was subjected to an RFM (Recency, Frequency, Monetary) analysis to enhance strategic decision-making in customer relationship management and gain a deeper understanding of customer behavior. Based on past transaction data, this method is a tried-and-true unsupervised technique for customer segmentation. It is especially helpful for identifying high-value consumers and adjusting marketing campaigns appropriately.

Model 02: K-Means Clustering

We used K-Means clustering to the RFM (Recency, Frequency, Monetary) features in order to segment the customer base by their purchase patterns. K-Means is an unsupervised machine learning technique that creates predetermined clusters from related data points. In order to reduce the distance within clusters, Kmeans iteratively updates the cluster centroids after initializing them and assigning each transaction to its closest cluster centroid.

This approach is especially effective for customer segmentation as it enables companies to distinguish unique customer groups based on their behavior, helping to tailor marketing efforts, improve retention, and optimize resource distribution.

Model 03: Hierarchical Clustering

Hierarchical clustering aims to create a hierarchy of clusters. It is frequently used in bioinformatics, pattern recognition, and data mining to cluster related items according to their characteristics.

Chapter 5: Model Analysis

Model 01: RFM: Recency, Frequency, and Monetary Value Segmentation

To enhance the understanding of customer behavior and inform strategic decisions in customer relationship management, an RFM (Recency, Frequency, Monetary) analysis was applied to an online retail dataset. This well-known unsupervised technique categorizes customers based on their purchasing history and is particularly effective in identifying high-value customers and developing targeted marketing campaigns

1. Rationale for RFM Segmentation

The RFM model evaluates customer involvement based on three key factors:

- Recency (R): Refers to the time elapsed since a customer's most recent purchase. Customers with more recent purchases are generally seen as more actively engaged.
- Frequency (F): Reflects how often a customer buys within a specific period. Those who purchase more frequently are usually viewed as loyal and repeat buyers.
- Monetary (M): Represents the total spend by a customer. Customers with higher spending levels are considered more valuable due to their greater financial contribution.

This model is particularly suitable for the dataset at hand as it provides meaningful customer insights without requiring predictive modeling techniques or labeled training data. It enables segmentation based on observable transactional behavior alone.

2. Data Preparation for RFM Analysis

To guarantee the accuracy and dependability of the RFM analysis, several data preprocessing steps were carried out beforehand.

- Exclusion of cancelled transactions: Invoices that begin with the letter 'C' were removed, as these represent cancellations or returns.
- CustomerID filtering: Only transactions with a valid (non-null) CustomerID were retained. Since customer-level segmentation is central to RFM, missing values in this field cannot be imputed.
- Removal of invalid records: Records with negative values in either Quantity or UnitPrice—aside from recognized discounts—were removed to avoid skewing the spending analysis.
- Feature Engineering (Total Price): A new variable was generated to determine the total spend on each transaction as the product of Quantity and UnitPrice. This is used as the RFM model's Monetary value feature.

3. RFM Metric Calculation

The features are defined as follows:

- Recency measured the number of days since the customer's last purchase, relative to the last transaction in the dataset.
- Frequency represented each customer's total number of purchase events.
- Monetary value was calculated as the total spend by each customer across all valid transactions.
- Valid transactions are understood to be successful purchases without returns.

1. Scoring Methodology

To enable standardized comparisons across customers, each RFM component was assigned a quartile-based score ranging from 1 to 4, where:

- A score of 4 represents the most favorable behavior (e.g., most recent purchase, highest frequency, or highest spending).
- A score of 1 represents the least favorable segment for that metric.

This quantile-based binning allows for a simple yet interpretable ranking system, facilitating downstream segmentation and cluster analysis. The final RFM scores can also be combined to form composite customer profiles (e.g., “444” indicating top-tier customers in all three categories).

5. Use Case and Strategic Implications

RFM segmentation enables businesses to:

- Identify loyal and high-value customers for retention strategies
- Spot inactive or low-engagement customers for reactivation campaigns
- Design personalized promotions based on customer profiles
- Allocate marketing budgets more efficiently by focusing on high-return segments

Overall, the RFM analysis serves as a robust, data-driven foundation for customer-centric strategies and supports deeper behavioral modeling in future work.

PLOT 1: DISTRIBUTION OF RFM SCORES

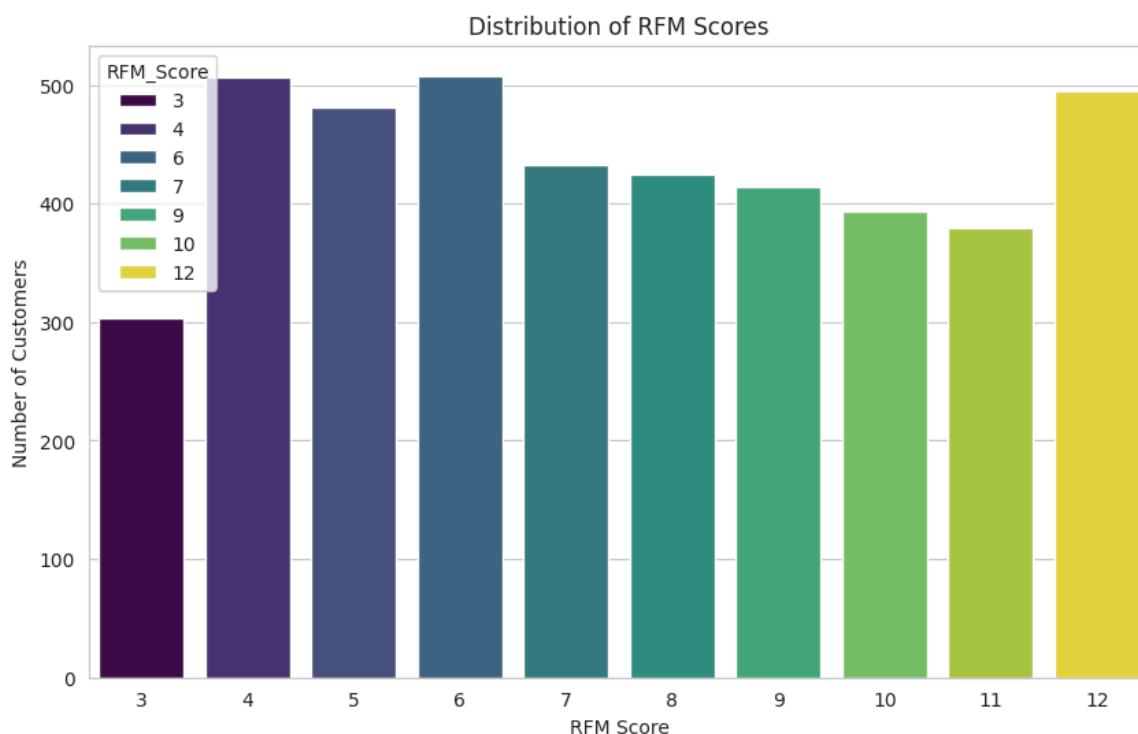


Figure 3.6: Distribution of RFM Scores

RFM Score Distribution Analysis

- Distribution Summary

The RFM score distribution reveals a non-uniform and bimodal pattern, with customer counts ranging from approximately 100 to 500 per score. Peaks are observed at scores 5 and 12, suggesting two dominant behavioral clusters.

- Analytical Trends

- ✓ Score 12 (~500 customers): Indicates customers with optimal recency, frequency, and monetary values—representing the highest engagement tier.
- ✓ Score 5 (~500 customers): Reflects a sizable segment with moderate engagement, likely strong in one or two RFM dimensions.
- ✓ Scores 3–4 (100–450 customers): Correspond to low-engagement segments, with infrequent, low-value, or older purchases.

- ✓ Scores 6–11: Display a progressive decline, implying a drop in sustained high performance, followed by a sharp recovery at score 12.
- Strategic Implications
- ✓ Top-tier (RFM 12): The high-value customer segment is ideal for retention-focused strategies, including loyalty programs, personalized promotions, and predictive models for customer lifetime value..
- ✓ Mid-tier (RFM 5): Ideal for conversion-focused campaigns, including cross-sell and upsell efforts to elevate them to premium tiers.
- ✓ Low-tier (RFM 3–4): Require reactivation tactics such as targeted promotions or re-engagement workflows to mitigate churn.
- ✓ Declining Middle (RFM 6–11): Represents a leakage zone, where customers may be slipping from high to mid engagement—warranting early intervention.

PLOT 2: RECENCY VS. MONETARY SCATTER

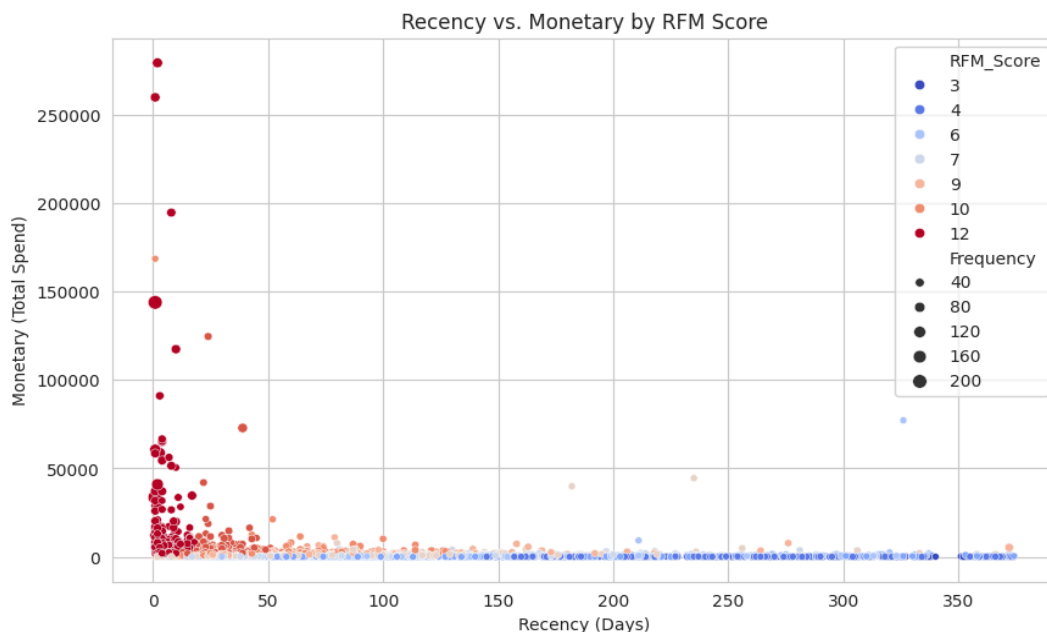


Figure 3.7: Recency vs. Monetary Scatter by RFM Score

The scatter plot visualizes customer behavior using two primary RFM dimensions: Recency (days since last purchase, x-axis) and Monetary value (total spend in GBP, y-axis). Color encodes RFM scores, while point size represents Frequency (number of transactions). This multidimensional view enables a nuanced understanding of customer segments.

Key Observations

1. Recency Distribution

- A significant cluster of customers made purchases within the last 100 days, suggesting strong recent engagement.
- A smaller subset shows recency values between 300–350 days, indicating a segment nearing inactivity or churn risk.

2. Monetary Value Distribution

- Total customer spend varies widely, with the y-axis extending up to £250,000.
- Most customers spend under £50,000, concentrated in the lower y-axis range.

- A few high-value outliers exceed £150,000, representing key contributors to revenue.

3. RFM Score Segmentation

- Low RFM Scores (3–7, blue tones): These customers tend to have high recency (i.e., older purchases) and low monetary values, indicating disengagement.
- Moderate RFM Scores (9–10, orange): Positioned in the mid-range of both spend and recency, this group is active but not yet high-value.
- High RFM Score (12, red): These customers are both recent and high-spending, with monetary values often exceeding £50,000. There are few but critically important.

4. Frequency Correlation

- Point size reflects transaction count; larger points (high frequency) appear more often in high RFM segments, especially at high spend levels.
- Smaller points dominate lower RFM scores, reinforcing the link between low frequency and lower customer value.

Strategic Business Insights

- High-Value Segment (RFM = 12): Small but highly profitable group; ideal for VIP loyalty programs, exclusive offers, and retention-focused strategies.
- At-Risk Customers (RFM 3–7): Characterized by long inactivity and low spend; require re-engagement campaigns, such as personalized discounts or reminders.
- Growth Segment (RFM 9–10): Mid-tier customers with upward potential; targeted upselling or purchase incentives could push them into the top tier.
- Frequency as a Lever: A clear correlation exists between frequency and monetary value; driving repeat purchases could significantly increase overall customer value.

PLOT 3: TOP 10 CUSTOMERS BY MONETARY VALUE

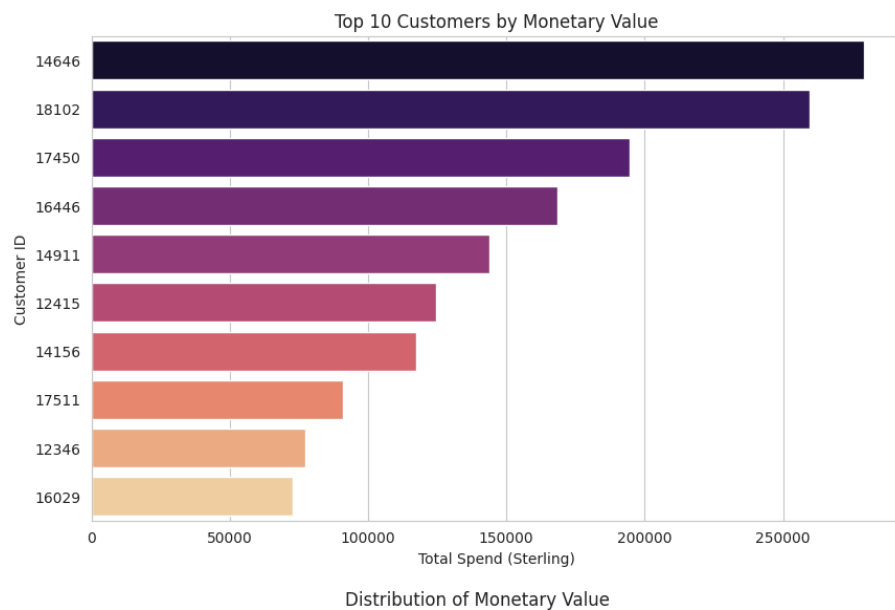


Figure 3.7: Top 10 Customers by Monetary Value

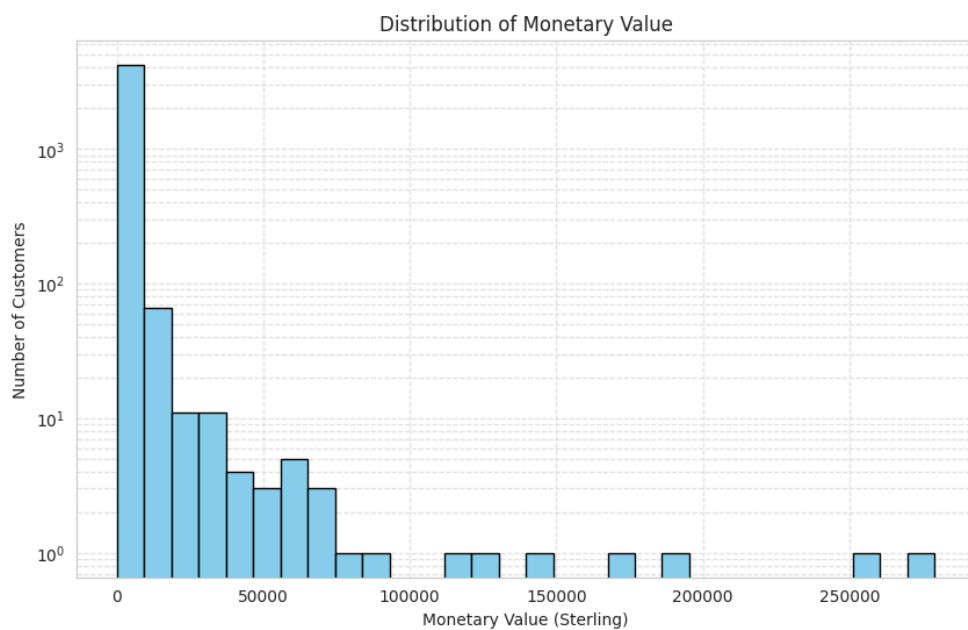


Figure 3.8: Distribution of Monetary Value

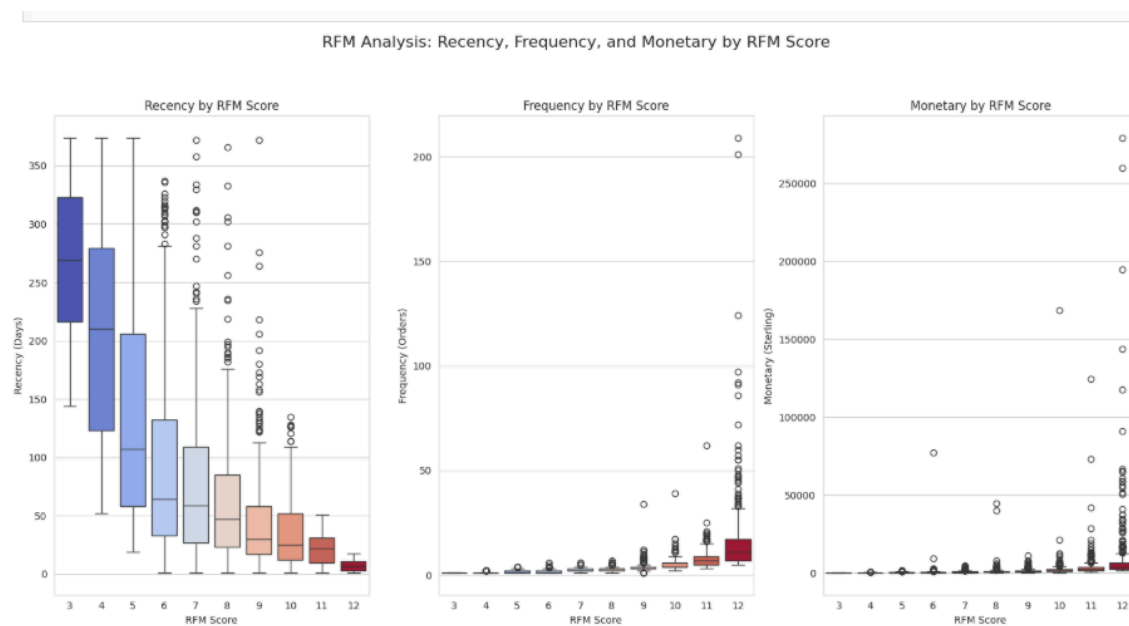


Figure 3.9: Recency, Frequency, and Monetary by RFM Score

This visualization displays the geographic distribution of customers within top RFM segments (e.g., ‘444’, ‘433’) by country. It helps identify whether high-value customers are concentrated in specific regions—such as the UK, which is the primary market, or in international markets.

By mapping customer value across geographies, businesses can:

- Detect regional concentration of premium customers
- Assess international market potential
- Tailor regional marketing efforts toward high-performing customer segments

Given that exploratory analysis shows a majority of transactions originate in the UK, this visualization is key to uncovering untapped value in other countries.

Distribution Overview

The histogram of customer monetary values shows a strong right-skew, where most customers spend less than £50,000. A long tail stretches toward £250,000, highlighting a few high-spending outliers. Using a logarithmic scale helps to better visualize the dense cluster of lower spenders while still displaying rare, high-value customers.

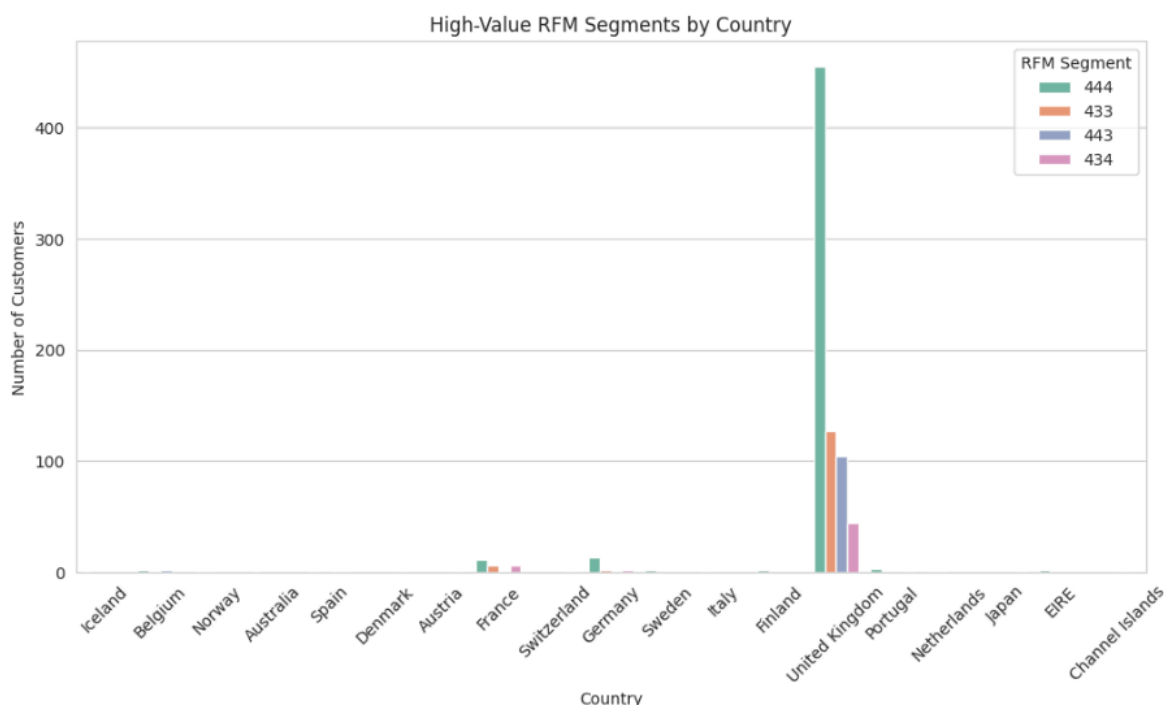


Figure 3.10: High-Value RFM Segments by Country

A time series analysis of transaction counts for top RFM customers (e.g., score '444') reveals temporal purchasing patterns. This helps identify seasonal spikes, such as increased activity during holiday periods or end-of-year sales, which are critical for marketing calendar planning. Given the dataset covers the period from December 2010 to December 2011, this analysis provides insights into key revenue-generating time frames for high-value customer segments.

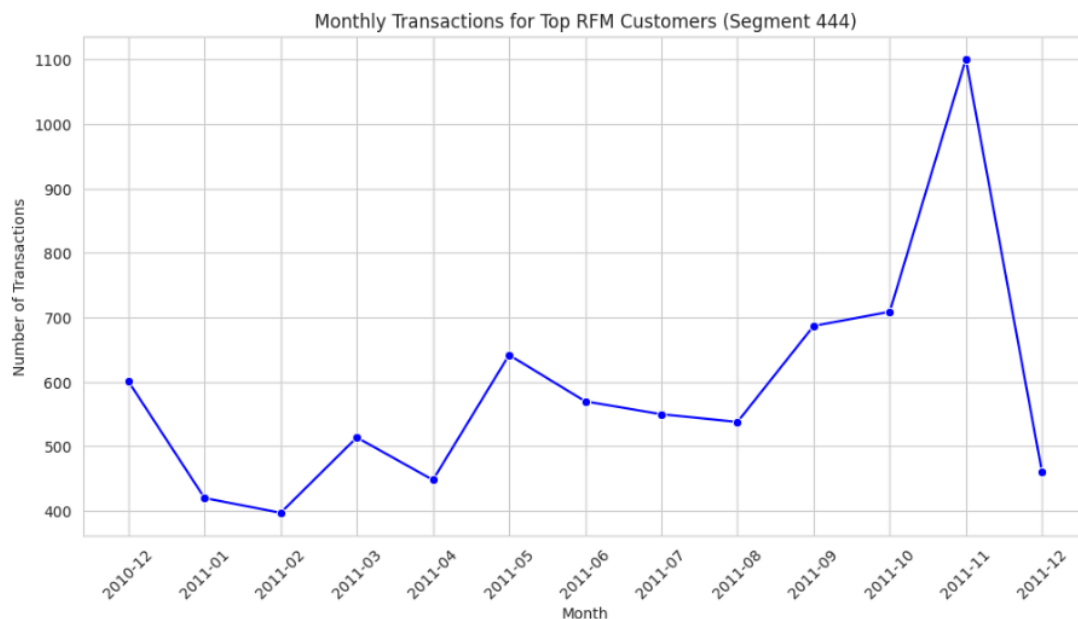


Figure 3.11: Monthly Transactions for Top RFM Customers

The time series shows that top RFM customers shop more often in November and December, likely due to holiday shopping around Christmas. There is also increased activity around Easter.

These patterns suggest that the best time to run big sales or targeted promotions is during these key holiday periods, especially late in the year.

Heatmap of RFM Score Combinations

Below is a heatmap of customer counts across combinations of R, F, and M scores to show which combinations are most common. This provides a compact view of how Recency, Frequency, and Monetary scores interact, highlighting dominant customer profiles.

The heatmap is essentially a pivot table where the x-axis represents Frequency (F) scores, the y-axis represents Recency (R) scores, and the color intensity (with annotations) indicates the number of customers in each R-F combination. This visualization helps identify the most common customer profiles based on their RFM scores.

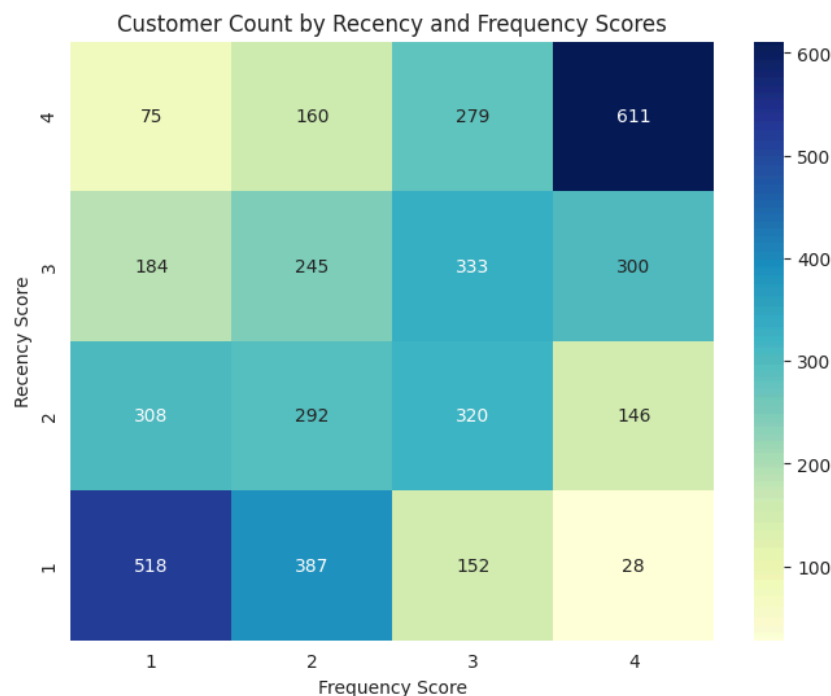


Figure 3.11: Customer Count by Recency and Frequency Scores

- **Dominant Profile:** The heatmap shows 611 customers with R=4 and F=4, indicating strong recent and frequent activity, aligning with top spenders like Customer 14646.
- **At-Risk Segment:** A notable 1-1 or 1-2 area suggests churn risk. With 518 customers in the 1-1 segment, re-engagement strategies are needed for those inactive since late 2010.
- **Growth Opportunity:** A 4-1 (75 customers) or 4-2 (160 customers) clusters indicate new customers. Offering discounts could boost their frequency.

Business action: Focus retention on 4-4/4-3, re-engagement on 1-1, and growth on 4-1

Key Business Insights

1. Top Spenders

- Customers 14646 and 18102 alone contribute over £400,000, making them the most valuable.
- These customers should be targeted with personalized retention strategies, such as exclusive offers or loyalty rewards.

2. High-Potential Clients

- Customers 17450, 16446, and 14911 (spending £100,000–£175,000) form a strong mid-tier group.
- Encouraging repeat purchases or increasing order value could move them to the top segment.

3. Growth Segment

- Customers like 14156, 17511, 12346, and 16029 (spending £25,000–£75,000) are in the top 10 but have room to grow.
- These are ideal for upselling or cross-selling strategies.

4. Revenue Concentration

- A large share of total revenue comes from a small group of top customers.
- Focusing on this group can improve ROI, but over-dependence should be managed with diversified engagement.

5. Segmentation Opportunity

- The sharp drop after £50,000 highlights a segment of mid-range spenders.
- Targeting this group with loyalty programs could increase their value and fill the gap between low and high spenders.

Model 02: K-Means Clustering

This method is particularly suitable for customer segmentation because it allows businesses to identify distinct customer groups based on behavior, which can guide targeted marketing, retention strategies, and resource allocation.

Scaling RFM Features

Prior to performing clustering, the RFM variables were standardized using z-score normalization through the StandardScaler method. As Recency, Frequency, and Monetary values vary in scale

and units, normalization was essential to prevent any one feature from disproportionately influencing the clustering outcome. This process adjusted each feature to have a mean of 0 and a standard deviation of 1.

Clustering Approach

Two clustering techniques were explored:

- K-Means Clustering: Used the scaled RFM features to assign customers into k groups based on behavioral similarity.
- Hierarchical Clustering (Ward's Linkage): Built a dendrogram to visually explore the cluster hierarchy. This helps in understanding natural divisions in the data without predefining the number of clusters.

Determining Optimal Number of Clusters

To identify the most appropriate number of clusters (k) for K-Means, we used two evaluation metrics:

- Silhouette Score: Indicates how well a point fits within its cluster. Values range from -1 to +1, with higher values meaning better cohesion and separation.
- Calinski-Harabasz Index: Measures the ratio of between-cluster dispersion to within-cluster dispersion. A higher score indicates well-separated and compact clusters.

These metrics are calculated for k values ranging from 2 to 9, and the results are plotted to visually assess the best cluster number.

For k-Means, to determine the most suitable number of clusters, we tested several values of k within the range of 2 to 9. For each value, it applies K-Means clustering to the scaled RFM data and calculates two key metrics: the Silhouette Score, which reflects how well each customer matches with its assigned cluster (higher values suggest better cohesion), and the Calinski-Harabasz Index, which measures how distinct and well-separated the clusters are.

These scores are then visualized through plots, allowing easy comparison across different k values. The function identifies the optimal number of clusters as the one with the highest Silhouette Score, ensuring the clustering structure is both meaningful and reliable.

Clustering and Optimal k Selection

When using clustering methods like K-Means or Hierarchical Clustering, selecting the optimal number of clusters (k) requires assessing how effectively the data is grouped at different k values. In this study, we evaluate cluster counts from 2 to 9 by applying two metrics to gauge cluster quality:

- Silhouette Score: Indicates how closely each data point matches its own cluster compared to others, with values ranging from -1 to 1; higher scores signify better clustering.
- Calinski-Harabasz Score: Represents the ratio of separation between clusters to the compactness within clusters; there is no upper limit, and higher values indicate superior clustering.

The process involves:

- Running K-Means for each k from 2 to 9.
- Calculating Silhouette and Calinski-Harabasz scores for each k .
- Selecting the k that optimizes one or both metrics—usually by identifying the highest score or a “knee” point where further increases in k offer limited improvement.

In this study, we selected the Silhouette Score to determine the optimal k due to its interpretability and reliability. Since it is bounded between -1 and 1, it is easier to interpret and compare across datasets (e.g., scores above 0.5 indicate good clustering, while scores below 0.2 suggest poor clustering). The Silhouette Score is also robust, as it considers both intra-cluster cohesion (how compact each cluster is) and inter-cluster separation (how distinct clusters are), making it a balanced metric. Additionally, it is sensitive to the clustering quality of individual points.

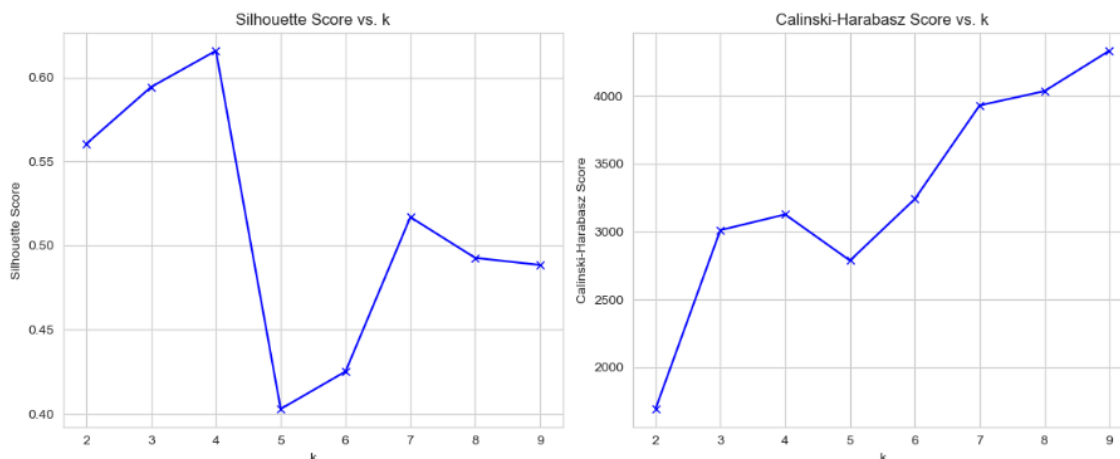


Figure 3.12: Elbow charts for Silhouette Score and Calinski-Harabasz Index

Business Context

In the context of RFM (Recency, Frequency, Monetary) analysis, selecting the right number of clusters is essential for producing actionable customer segments:

- Too few clusters (e.g., $k = 2$) may group high-value and loyal customers together, missing important distinctions for targeted marketing.
- Too many clusters (e.g., $k = 8$) might split similar customers unnecessarily, complicating strategic decisions.
- An optimal number of clusters (e.g., $k = 4$) typically yields meaningful segments such as High-Value, Loyal, At-Risk, and Emerging customers. These can support specific business actions like loyalty programs or re-engagement campaigns.

Applying K-Means

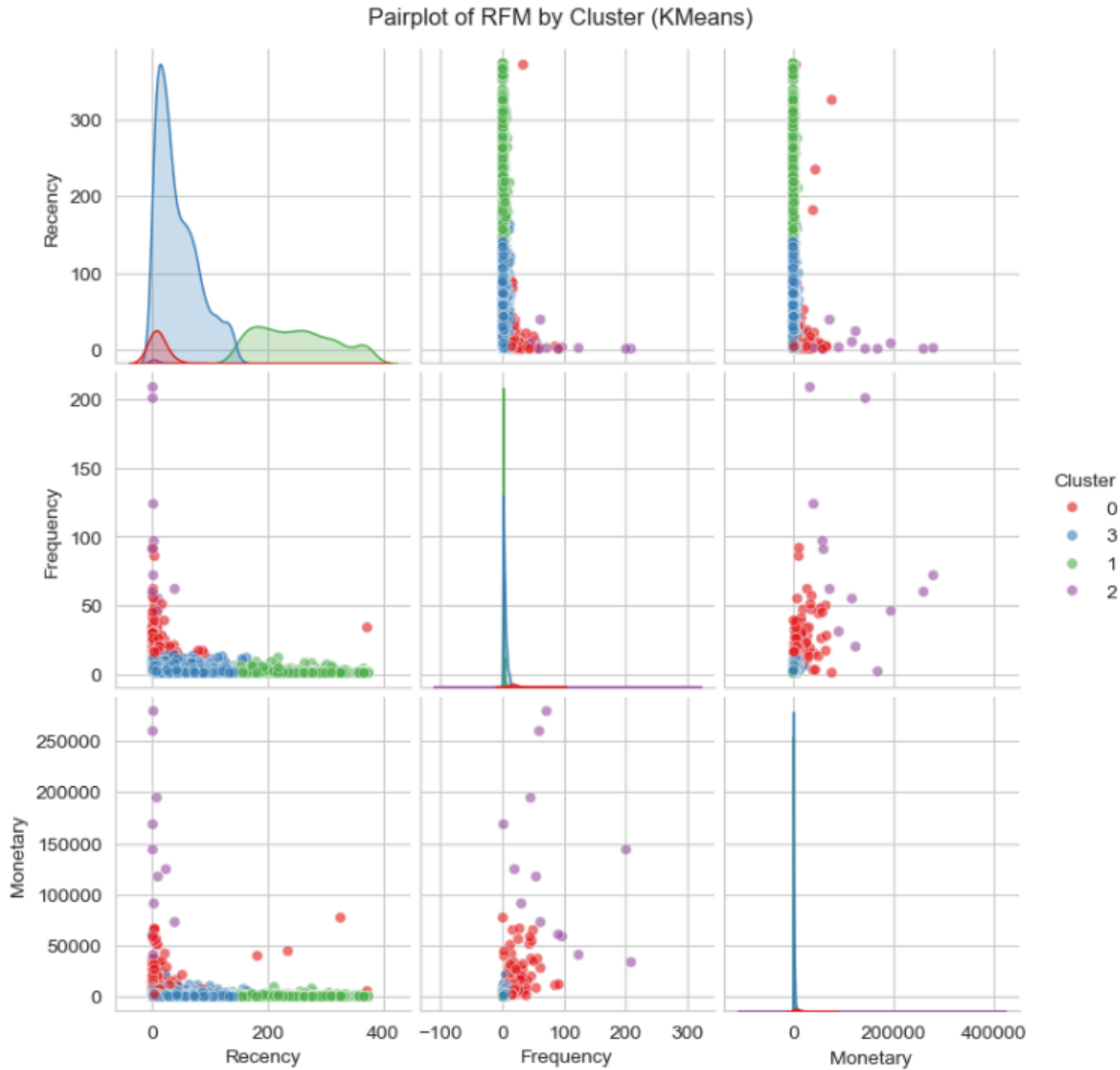


Figure 3.13: Pairplot of RFM by K-Means

Insight:

The Recency vs. Monetary scatter shows Cluster 0 tightly clustered at low Recency and high Monetary, which confirms a distinct High-Value segment.

Cluster 1 (At-Risk) has high Recency and low Frequency, target these customers with re-engagement campaign.

Cluster Profile Bar Plot (Plotly)

Description: Bar plot comparing mean Recency, Frequency, and Monetary across clusters, with bars grouped by cluster. Significance: Highlights average RFM values per cluster, making it easy to compare segment characteristics. It shows which metrics define each cluster (e.g., high Monetary for Cluster 0).

Clustering Choice: Distinct bar heights (e.g., Cluster 0: low Recency, high Monetary vs. Cluster 2: high Recency, low Monetary) indicate good clustering.



Figure 3.14: Cluster Profiles

Insight: High-Value customers ie Cluster 0 and Cluster 2 need loyalty programs (e.g., VIP discounts in France), while At-Risk customers (Cluster 1) need re-engagement (e.g., €5 vouchers in Germany).

Box Plots

Description: Three box plots (one for each RFM metric) shows median, quartiles, and outliers per cluster.

It displays the spread and variability of RFM metrics within each cluster. It helps us identify outliers (e.g., a customer in Cluster 2 with unusually high Monetary). Various median differences across clusters are also highlighted.

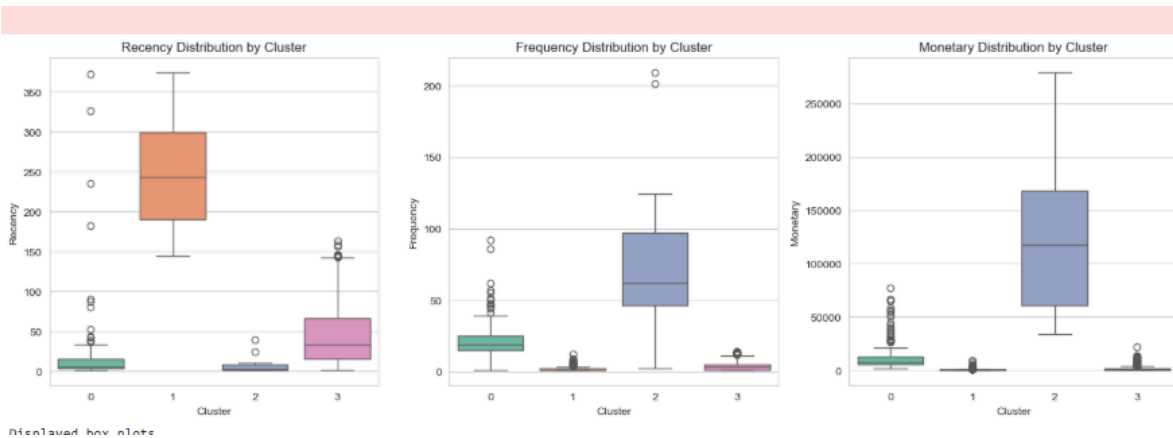


Figure 3.15: Box Plots for RFM by K-Means

Insight:

Clusters with wide Monetary spread (e.g., Cluster 2) may include diverse Loyal customers, suggesting targeted upsell campaigns (e.g., bundle offers in Germany)

Heatmap of Standardized Centroids

Description: Heatmap of standardized (z-scored) RFM means per cluster, with color intensity (red for positive, blue for negative) and annotations. It shows relative differences in RFM metrics across clusters on a standardized scale. It also highlights defining traits (e.g., negative Recency, positive Monetary for High-Value).

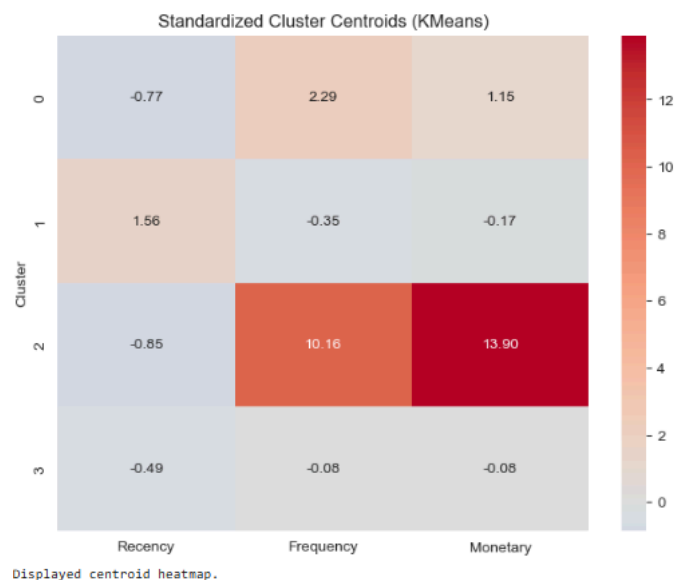


Figure 3.16: Heatmap for RFM by K-Means

Insight:

Cluster 0's strong positive Monetary z-score suggests high-spending customers, ideal for premium offers (e.g., exclusive products in France).

2D Scatter Plots:

Description: Three interactive scatter plots showing pairs of RFM metrics, colored by cluster, with hover data (CustomerID).



Figure 3.17: Scatterplot for RFM by K-Means

Insight:

Clear separation in Recency vs. Monetary (e.g., Cluster 1 at high Recency, low Monetary) supports re-engagement campaigns for At-Risk customers (e.g., Cluster 1).

Violin Plots

Description: Three violin plots showing the density and distribution of Recency, Frequency, and Monetary per cluster.

Combines box plot (median, quartiles) and KDE to show distribution shape and density.

Highlights multimodal distributions or skewness (e.g., Monetary in Cluster 3 may be skewed).

Reveals variability and density differences across clusters.

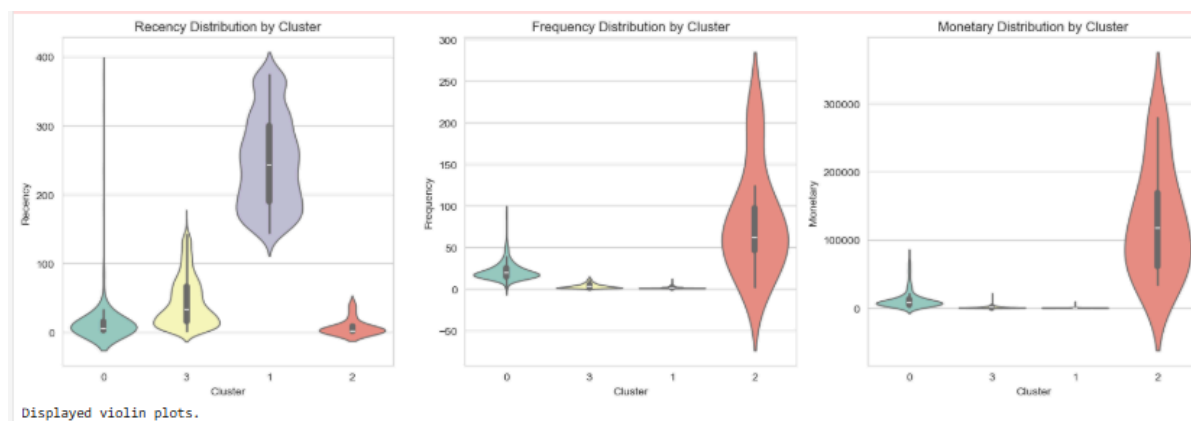


Figure 3.18: Violin Plots for RFM by K-Means

Insight:

Wide violins in Cluster 2 (Loyal) indicate varied spending, making them ideal for personalized upsell offers (e.g., Germany).

K-Means Cluster Summary

RFM Summary for Chosen Method:

	Recency_Mean	Recency_Std	Frequency_Mean	Frequency_Std	Monetary_Mean	Monetary_Std	Count
Cluster							
0	15.57	40.11	21.89	11.56	12345.37	12964.55	213
1	248.39	66.17	1.55	1.07	474.72	626.59	1064
2	7.38	11.49	82.31	63.39	126602.56	79979.48	13
3	43.92	36.15	3.64	2.81	1329.76	1504.99	3048

Model 03: Hierarchical Clustering

In cluster analysis, the hierarchical clustering technique that aims to create a nested structure of clusters. It is commonly applied in data mining, pattern recognition, and bioinformatics.

```
# Applying hierarchical clustering
hier_labels, hier_sil_score = apply_hierarchical(rfm_scaled, optimal_k)
```



Figure 3.19: Hierarchical clustering Dendrogram

Hierarchical Clustering Summary:

```
43]: hier_summary = summarize_rfm_per_cluster(rfm.copy(), hier_labels)
print("\nHierarchical Clustering RFM Summary per Cluster:")
print(hier_summary)
```

Hierarchical Clustering RFM Summary per Cluster:

Cluster	Recency_Mean	Recency_Std	Frequency_Mean	Frequency_Std
0	7.00	8.29	40.86	24.51
1	5.81	9.73	85.38	52.22
2	237.20	71.51	1.59	1.03
3	38.89	33.46	4.77	5.25

Cluster	Monetary_Mean	Monetary_Std	Count
0	176403.59	72263.41	7
1	50058.26	30746.14	16
2	505.13	653.09	1178
3	1977.31	3979.83	3137

Comparing the performance of K-Means and Hierarchical Clustering

We compare the clustering performance of K-Means and Hierarchical Clustering using their Silhouette Scores, a metric that measures cluster cohesion and separation. It prints out which algorithm performed better based on the scores and provides a recommendation:

- K-Means is preferred if it yields a higher score due to its simplicity and scalability.

Compare SIL Score of both clustering methods

```
159]: # Compare and decide
print("\nComparison and Decision:")
if kmeans_sil_score > hier_sil_score:
    print(f"K-Means is better (Silhouette Score: {kmeans_sil_score:.4f} > {hier_sil_score:.4f})")
    print("Recommendation: Use K-Means for RFM segmentation due to higher cluster quality.")
elif hier_sil_score > kmeans_sil_score:
    print(f"Hierarchical is better (Silhouette Score: {hier_sil_score:.4f} > {kmeans_sil_score:.4f})")
    print("Recommendation: Use Hierarchical clustering if nested structures are valuable.")
else:
    print(f"Equal Silhouette Scores ({kmeans_sil_score:.4f})")
    print("Recommendation: We would use K-Means for simplicity and scalability.")
```

```
Comparison and Decision:
K-Means is better (Silhouette Score: 0.6157 > 0.5806)
Recommendation: Use K-Means for RFM segmentation due to higher cluster quality.
```

Insight: K-Means is better (Silhouette Score: 0.6157 > 0.5806)

Recommendation: Use K-Means for RFM segmentation due to higher cluster quality

Chapter 6: Conclusion and Recommendations

Final Insights:

- High-Value (e.g., Cluster 0): Bar plot/heatmap show low Recency, high Monetary → Offer VIP discounts (e.g., France).
- At-Risk (e.g., Cluster 1): High Recency in scatters/box plots → Re-engagement emails (e.g., €5 vouchers in Norway).
- Loyal (e.g., Cluster 2): Moderate RFM in bar plot, wide violin plots → Upsell bundles (e.g., Germany).
- Emerging (e.g., Cluster 3): Moderate bars | Box plots in → Onboarding offers (e.g., free shipping).

Appendix 1: Jupyter Notebook

Final-Project-Presentation-Team-9-Notebook

June 23, 2025

1 G9: Probability and Statistics for Artificial Intelligence (AAI-500-IN2)

1.1 Introduction

Our project looks at a UK-based online retail for gift sales dataset.

Conduct a descriptive analysis of the Online Retail dataset as part of your data analysis project. We have a transactional dataset with 541,909 instances and 6 key features: InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID, and Country.

```
[1]: %pip install ucimlrepo missingno plotly --quiet
```

Note: you may need to restart the kernel to use updated packages.

```
[4]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import missingno as msno
from datetime import datetime
from ucimlrepo import fetch_ucirepo
import os
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, calinski_harabasz_score
from scipy.cluster.hierarchy import dendrogram, linkage, fcluster
import plotly.express as px
import plotly.graph_objects as go
```

```
[ ]: # Load the dataset
try:
    # Define the file path in Colab's temporary storage
    file_path = '/content/online_retail.csv'
    #file_path = "C:/Users/tamay/Downloads/online_retail.csv"

    # Check if the file already exists
    if os.path.exists(file_path):
        # Load the dataset from the local file
```

```

online_retail_df = pd.read_csv(file_path)
print('Loaded dataset from local file.')
else:
    # Fetch the dataset from UCI repository
    online_retail = fetch_ucirepo(id=352)
    # Get the original DataFrame
    online_retail_df = online_retail.data.original
    # Save the DataFrame to a CSV file
    online_retail_df.to_csv(file_path, index=False)
    print('Downloaded dataset and saved to local file.')
except:
    print('Error loading or saving the dataset using fetch_ucirepo.')
    from google.colab import drive
    try:
        drive.mount("/content/drive")
    finally:
        online_retail_df = pd.read_csv('/content/drive/My Drive/Colab Notebooks/
↳ Group 9 Project/Data/Online Retail.csv', encoding='ISO-8859-1')

```

Loaded dataset from local file.

```

[ ]: #file_path = './Online Retail.csv'
    #if os.path.exists(file_path):
    # online_retail_df = pd.read_csv('./Online Retail.csv', encoding='ISO-8859-1')

```

1.2 Data Cleaning and Preparation

1.2.1 Preview the data

First 5 rows of the dataset:

```

[7]: # Display first few rows
    online_retail_df.head()

```

```

[7]: InvoiceNo StockCode Description Quantity \
0  536365 85123A WHITE HANGING HEART T-LIGHT HOLDER 6
1  536365 71053 WHITE METAL LANTERN 6
2  536365 84406B CREAM CUPID HEARTS COAT HANGER 8
3  536365 84029G KNITTED UNION FLAG HOT WATER BOTTLE 6
4  536365 84029E RED WOOLLY HOTTIE WHITE HEART. 6

InvoiceDate UnitPrice CustomerID Country
0 2010-12-01 08:26:00 2.55 17850.0 United Kingdom
1 2010-12-01 08:26:00 3.39 17850.0 United Kingdom
2 2010-12-01 08:26:00 2.75 17850.0 United Kingdom
3 2010-12-01 08:26:00 3.39 17850.0 United Kingdom
4 2010-12-01 08:26:00 3.39 17850.0 United Kingdom

```

1.3 Display the data types:

```
[8]: online_retail_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 541909 entries, 0 to 541908
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype
---  -
0   InvoiceNo        541909 non-null object
1   StockCode        541909 non-null object
2   Description      540455 non-null object
3   Quantity         541909 non-null int64
4   InvoiceDate       541909 non-null object
5   UnitPrice        541909 non-null float64
6   CustomerID       406829 non-null float64
7   Country          541909 non-null object
dtypes: float64(2), int64(1), object(5)
memory usage: 33.1+ MB
```

1.3.1 Check for missing values

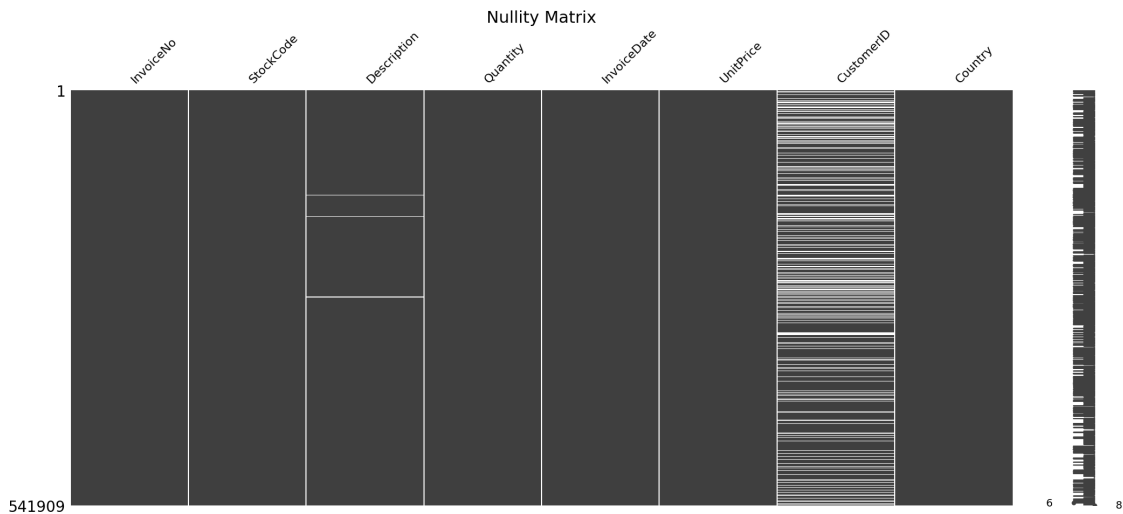
```
[9]: # Check for missing values
print("\nMissing Values:")
print(online_retail_df.isnull().sum())

# Show nullity across all columns
plt.figure(figsize=(8, 6))
msno.matrix(online_retail_df)
plt.title('Nullity Matrix', fontsize=22)
plt.show()
```

Missing Values:

```
InvoiceNo      0
StockCode      0
Description    1454
Quantity       0
InvoiceDate    0
UnitPrice      0
CustomerID    135080
Country        0
dtype: int64
```

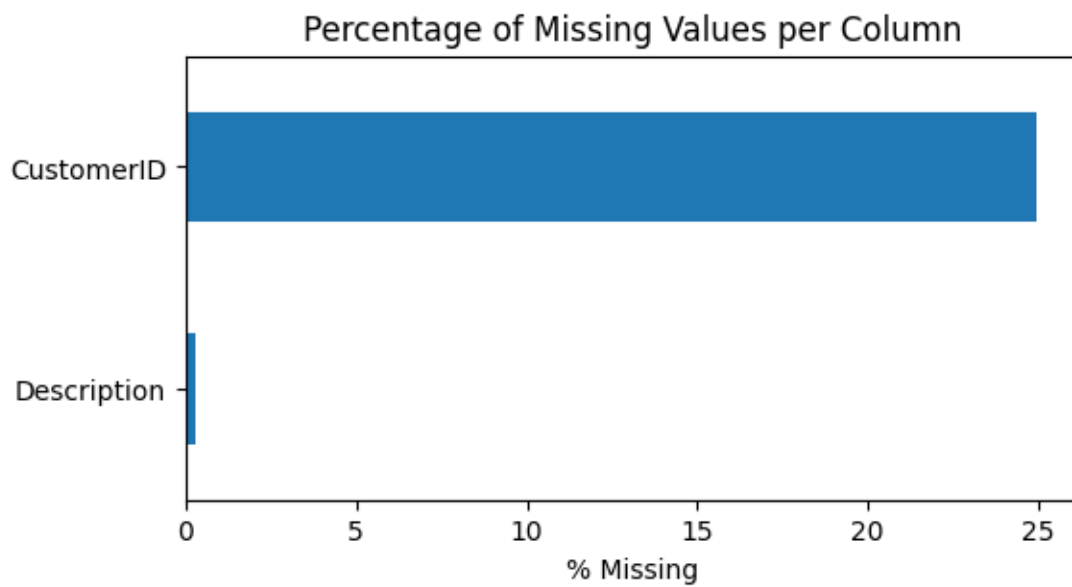
<Figure size 800x600 with 0 Axes>



1.4 Missing Percentage

```
[11]: missing_percent = online_retail_df.isnull().mean() * 100

plt.figure(figsize=(6, 3))
missing_percent[missing_percent > 0].sort_values().plot.barh()
plt.title('Percentage of Missing Values per Column')
plt.xlabel('% Missing')
plt.show()
```



We have seen the first 5 rows, data types, and missing value counts. The dataset has no missing values as per the documentation, but we verified this anyways. The `InvoiceDate` may need parsing as a datetime object, and `CustomerID` might be float but should be treated as categorical.

We miss 25% of customer IDs, since customer IDs are required field in the data and is the base of our analysis, we can not impute the information of missing customer IDs

1.5 Data Cleaning and Preparation

Even though the dataset has no missing values, we need to:

- Convert `InvoiceDate` to datetime.
- Check for cancellations (transactions where `InvoiceNo` starts with 'C').
- Ensure `CustomerID` is treated as a string (categorical).
- Look for outliers or invalid data (e.g., negative `Quantity` or `UnitPrice`).
- Add a `TotalPrice` to allow revenue analysis

Update Data Types

```
[12]: # Convert InvoiceDate to datetime
online_retail_df['InvoiceDate'] = pd.
    ↳to_datetime(online_retail_df['InvoiceDate'])
online_retail_df['DateID'] = online_retail_df['InvoiceDate'].dt.date
online_retail_df['DateID'] = pd.to_datetime(online_retail_df['DateID'])

online_retail_df['DateID'] = online_retail_df['InvoiceDate'].dt.date
online_retail_df['DateID'] = pd.to_datetime(online_retail_df['DateID'])

# Convert CustomerID to string (categorical) and remove '.0'
online_retail_df['CustomerID'] = online_retail_df['CustomerID'].astype(str).
    ↳replace(r'\.0$', '', regex=True)
```

```
[13]: online_retail_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 541909 entries, 0 to 541908
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  -
0   InvoiceNo        541909 non-null object
1   StockCode       541909 non-null object
2   Description     540455 non-null object
3   Quantity        541909 non-null int64
4   InvoiceDate     541909 non-null datetime64[ns]
5   UnitPrice       541909 non-null float64
6   CustomerID      541909 non-null object
7   Country         541909 non-null object
8   DateID          541909 non-null datetime64[ns]
dtypes: datetime64[ns](2), float64(1), int64(1), object(5)
memory usage: 37.2+ MB
```

1.6 Create new data fields

```
[14]: # Identify cancellations
online_retail_df['IsCancelled'] = online_retail_df['InvoiceNo'].str.
    ↳startswith('C')

# Calculate TotalPrice
online_retail_df['TotalPrice'] = online_retail_df['Quantity'] *
    ↳online_retail_df['UnitPrice']

online_retail_df.head()
```

```
[14]: InvoiceNo StockCode Description Quantity \
0 536365 85123A WHITE HANGING HEART T-LIGHT HOLDER 6
1 536365 71053 WHITE METAL LANTERN 6
2 536365 84406B CREAM CUPID HEARTS COAT HANGER 8
3 536365 84029G KNITTED UNION FLAG HOT WATER BOTTLE 6
4 536365 84029E RED WOOLLY HOTTIE WHITE HEART. 6

InvoiceDate UnitPrice CustomerID Country DateID \
0 2010-12-01 08:26:00 2.55 17850 United Kingdom 2010-12-01
1 2010-12-01 08:26:00 3.39 17850 United Kingdom 2010-12-01
2 2010-12-01 08:26:00 2.75 17850 United Kingdom 2010-12-01
3 2010-12-01 08:26:00 3.39 17850 United Kingdom 2010-12-01
4 2010-12-01 08:26:00 3.39 17850 United Kingdom 2010-12-01

IsCancelled TotalPrice
0 False 15.30
1 False 20.34
2 False 22.00
3 False 20.34
4 False 20.34
```

Treatment of Outliers The `Quantity` field includes both positive and negative values (negative likely due to cancellations, as `InvoiceNo` starting with ‘C’ indicates)

- Negative quantities (cancellations) may affect the shape of the distribution.
- The data may have extreme positive outliers (e.g., bulk orders).
- The underlying distribution might be log-normal rather than normal, which is common for quantities in retail data.

To address outliers, we can follow several approaches:

• Option 1: Filter Out Negative Quantities (Cancellations)

Since negative quantities likely represent cancellations, you can exclude them to focus on actual sales.

```
# Filter out cancellations and negative quantities
online_retail_df = online_retail_df[(online_retail_df['IsCancelled'] == False) & (online_r
```

- **Option 2: Winsorize or Cap Outliers**

Winsorizing replaces extreme values with a specified percentile threshold (e.g., 1st and 99th percentiles), reducing the impact of outliers without removing data.

```
# Winsorize Quantity at 1st and 99th percentiles
```

```
online_retail_df['QuantityWinsorized'] = mstats.winsorize(online_retail_df['Quantity'], li
```

- **Option 3: Remove Extreme Outliers Using IQR**

Use the Interquartile Range (IQR) method to identify and remove outliers, keeping data within 1.5 times the IQR from the quartiles.

```
# Calculate IQR
```

```
Q1 = online_retail_df['Quantity'].quantile(0.25)
```

```
Q3 = online_retail_df['Quantity'].quantile(0.75)
```

```
IQR = Q3 - Q1
```

```
lower_bound = Q1 - 1.5 * IQR
```

```
upper_bound = Q3 + 1.5 * IQR
```

```
# Filter out outliers
```

```
online_retail_df = online_retail_df[(online_retail_df['Quantity'] >= lower_bound) & (online
```

Identifying Outliers – Using IQR

```
[15]: # Calculate IQR
# def identify_outliers_iqr(group):
#     Q1 = group['TotalPrice'].quantile(0.25)
#     Q3 = group['TotalPrice'].quantile(0.75)
#     IQR = Q3 - Q1
#     lower_bound = Q1 - 1.5 * IQR
#     upper_bound = Q3 + 1.5 * IQR

#     # Mark outliers
#     group['lower_bound'] = lower_bound
#     group['upper_bound'] = upper_bound
#     group['IsOutlier'] = (group['Quantity'] < lower_bound) |
    ↪ (group['Quantity'] > upper_bound)
#     return group

def identify_quantile_outliers(group, column='TotalPrice', lower_q=0.05,
    ↪ upper_q=0.95):
    lower_bound = group[column].quantile(lower_q)
    upper_bound = group[column].quantile(upper_q)

    group['lower_bound'] = lower_bound
    group['upper_bound'] = upper_bound
    group['IsOutlier'] = (group[column] < lower_bound) | (group[column] >
    ↪ upper_bound)
    return group
```

```
# Apply the function to groups, excluding grouping columns
outlier_df = online_retail_df.groupby(['CustomerID', 'StockCode'],
    group_keys=False).apply(
    identify_quantile_outliers, include_groups=True
)
outlier_df.head()
```

C:\Users\tamay\AppData\Local\Temp\ipykernel_7964\1733339231.py:25:

DeprecationWarning: DataFrameGroupBy.apply operated on the grouping columns. This behavior is deprecated, and in a future version of pandas the grouping columns will be excluded from the operation. Either pass `include\_groups=False` to exclude the groupings or explicitly select the grouping columns after groupby to silence this warning.

```
outlier_df = online_retail_df.groupby(['CustomerID', 'StockCode'],
group_keys=False).apply(
```

```
[15]: InvoiceNo StockCode Description Quantity \
0 536365 85123A WHITE HANGING HEART T-LIGHT HOLDER 6
1 536365 71053 WHITE METAL LANTERN 6
2 536365 84406B CREAM CUPID HEARTS COAT HANGER 8
3 536365 84029G KNITTED UNION FLAG HOT WATER BOTTLE 6
4 536365 84029E RED WOOLLY HOTTIE WHITE HEART. 6

InvoiceDate UnitPrice CustomerID Country DateID \
0 2010-12-01 08:26:00 2.55 17850 United Kingdom 2010-12-01
1 2010-12-01 08:26:00 3.39 17850 United Kingdom 2010-12-01
2 2010-12-01 08:26:00 2.75 17850 United Kingdom 2010-12-01
3 2010-12-01 08:26:00 3.39 17850 United Kingdom 2010-12-01
4 2010-12-01 08:26:00 3.39 17850 United Kingdom 2010-12-01

IsCancelled TotalPrice lower_bound upper_bound IsOutlier
0 False 15.30 15.30 31.560 False
1 False 20.34 20.34 40.680 False
2 False 22.00 16.50 25.300 False
3 False 20.34 20.34 29.832 False
4 False 20.34 20.34 21.696 False
```

1.6.1 Negative Values

Negative quantities might indicate returns or cancellations, which we'll analyze separately.

If negative UnitPrice exists, it could be an error unless it represents adjustments.

We'll keep cancellations for now to analyze their impact.

```
[16]: # Check for negative quantities or prices
print("\nNegative Quantities:", (online_retail_df['Quantity'] < 0).sum())
print("Negative Unit Prices:", (online_retail_df['UnitPrice'] < 0).sum())
```

```

# Basic data cleaning: Remove invalid entries if needed
#online_retail_df = online_retail_df[online_retail_df['Quantity'] >= 0] #Drop
↳negative Quantity if they exist
valid_transactions = online_retail_df[online_retail_df['UnitPrice'] >= 0] #Drop
↳negative UnitPrice if they exist

# Filter out cancellations
df_cancellations = valid_transactions[(valid_transactions['IsCancelled'] ==
↳True)]
valid_transactions = valid_transactions[(valid_transactions['IsCancelled'] ==
↳False)]

valid_transactions.describe()

```

Negative Quantities: 10624

Negative Unit Prices: 2

```

[16]:
      Quantity      InvoiceDate      UnitPrice \
count  532619.000000      532619  532619.000000
mean    10.240006  2011-07-04 17:05:58.181701632    3.889174
min   -9600.000000  2010-12-01 08:26:00    0.000000
25%     1.000000  2011-03-28 12:13:00    1.250000
50%     3.000000  2011-07-20 11:54:00    2.080000
75%    10.000000  2011-10-19 12:21:00    4.130000
max    80995.000000  2011-12-09 12:50:00  13541.330000
std     159.593850      NaN    35.831786

      DateID      TotalPrice
count      532619  532619.000000
mean  2011-07-04 03:31:18.739586560    20.026857
min    2010-12-01 00:00:00    -0.000000
25%    2011-03-28 00:00:00    3.750000
50%    2011-07-20 00:00:00    9.900000
75%    2011-10-19 00:00:00   17.700000
max    2011-12-09 00:00:00  168469.600000
std      NaN    269.721209

```

1.7 Exploratory Data Analysis (EDA)

For our exploratory data analysis, we will initially just try to understand the data we have. It might be important to formulate a number of research questions that will help structure our EDA i.e. give us something to look for in the data.

1.7.1 Research Questions

When we look at the variables: InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID, Country, and the derived TotalPrice and IsCancelled, we can start thinking of

inferential research questions that we can ask on this dataset.

Inferential analysis aims to draw conclusions about the population beyond descriptive summaries, often using statistical tests or models to test hypotheses or predict outcomes.

Goals for our inferential analysis could include testing hypotheses, identifying relationships, or predicting outcomes to inform business decisions (e.g. customer behavior, sales trends).

1. Understand Customer Behavior and Segmentation: *Do customers from different countries have significantly different purchasing patterns (e.g. average order value or frequency)?*
Understanding country-based differences can inform targeted marketing strategies. We could use ANOVA to test if mean TotalPrice differs across countries.
2. Understanding Cancellation Patterns: *Are cancellations (IsCancelled) more likely for specific product types or customer segments?*
Identifying cancellation drivers can reduce revenue loss. We can use a regression model to predict IsCancelled using features like StockCode, Quantity, UnitPrice, and Country. Chi-square tests can assess associations between cancellations and categorical variables.
3. Identify Trends in Sales Over Time: *Is there a significant seasonal effect on sales volume or transaction frequency?*
Detecting seasonality can optimize inventory and marketing campaigns. This could be done through time-series analysis to identify seasonality.
4. Determine Product Popularity and Revenue Contribution: *Do certain products (StockCode) contribute disproportionately to total revenue, and are these differences statistically significant?*
Identifying top-performing products aids inventory management. Perform analysis to identify top revenue-generating products, followed by t-tests or ANOVA to compare TotalPrice across product groups.
5. Insights Into Customer Retention: *Do first-time customers have a significantly lower likelihood of repeat purchases compared to returning customers?*
Retention insights can guide customer loyalty programs. Define first-time vs. returning customers based on CustomerID and InvoiceDate. Develop models (e.g. regression, survival analysis, etc) to predict repeat purchase likelihood.

1.7.2 Descriptive Statistics

Given that this is a dataset for an online retailer store, we may expect to find:

- Quantity and UnitPrice may have skewed distributions (e.g., large orders or high-priced items).
- Country will likely show the UK as dominant (since it's UK-based retail).
- Cancellations (IsCancelled) will help quantify returns.
- The date range confirms transactions from Dec 2010 to Dec 2011.

```
[17]: # Top 5 countries by transaction count
print("\nTop 5 Countries by Transaction Count:")
valid_transactions['Country'].value_counts().head()
```

Top 5 Countries by Transaction Count:

```
[17]: Country
      United Kingdom    487620
      Germany           9042
      France            8408
      EIRE              7894
      Spain             2485
      Name: count, dtype: int64
```

```
[18]: # Transaction date range
      print("\nDate Range:")
      print(f"From: {valid_transactions['InvoiceDate'].min()} to_
      ↪{valid_transactions['InvoiceDate'].max()}")
```

Date Range:

From: 2010-12-01 08:26:00 to 2011-12-09 12:50:00

Descriptive Statistics for Numerical Features:

```
[19]: numeric_cols = ['Quantity', 'UnitPrice']

      # Numerical features summary
      stats = valid_transactions[numeric_cols].describe().T
      stats['IQR'] = stats['75%'] - stats['25%']
      stats[['count', 'mean', 'std', 'min', '25%', '50%', '75%', 'max', 'IQR']].T
```

```
[19]:
```

	Quantity	UnitPrice
count	532619.000000	532619.000000
mean	10.240006	3.889174
std	159.593850	35.831786
min	-9600.000000	0.000000
25%	1.000000	1.250000
50%	3.000000	2.080000
75%	10.000000	4.130000
max	80995.000000	13541.330000
IQR	9.000000	2.880000

```
[20]: # Set plot style
      #plt.style.use('seaborn')

      # Histogram for Quantity and UnitPrice
      plt.figure(figsize=(12, 5))

      plt.subplot(1, 2, 1)
      bin_edges = np.linspace(
          # for this data, after trial and error,
```

```

    # we rather define our own explicit bins
    valid_transactions['Quantity'].quantile(0.01),
    valid_transactions['Quantity'].quantile(0.99),
    26 # 25 bins needs 26 edges
)
sns.histplot(valid_transactions['Quantity'], bins=bin_edges, kde=True,
    stat='probability', edgecolor='black', linewidth=1, alpha=0.7)
plt.title('Distribution of Quantity (Sales Only)')
plt.xlabel('Quantity')
plt.xlim(valid_transactions['Quantity'].quantile(0.025),
    valid_transactions['Quantity'].quantile(0.975)) # Limit outliers

plt.subplot(1, 2, 2)
bin_edges = np.linspace(
    0,
    valid_transactions['UnitPrice'].quantile(0.99),
    26 # 25 bins needs 26 edges
)
sns.histplot(valid_transactions['UnitPrice'], bins=bin_edges, kde=True,
    stat='probability', edgecolor='black', linewidth=1, alpha=0.7)
plt.title('Distribution of UnitPrice (Sales Only)')
plt.xlabel('UnitPrice (£)')
plt.xlim(0, valid_transactions['UnitPrice'].quantile(0.99)) # Limit outliers

plt.tight_layout()
plt.show()

# Boxplot for Quantity and UnitPrice
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
sns.boxplot(valid_transactions['Quantity'])
plt.title('Boxplot of Quantity')
plt.xlabel('Quantity')

plt.subplot(1, 2, 2)
sns.boxplot(valid_transactions['UnitPrice'])
plt.title('Boxplot of UnitPrice')
plt.xlabel('UnitPrice (£)')
plt.tight_layout()
plt.show()

# Bar chart for top 5 countries
top_countries = valid_transactions['Country'].value_counts().head(5)
plt.figure(figsize=(8, 5))
sns.barplot(x=top_countries.values, y=top_countries.index)
plt.title('Top 5 Countries by Transaction Count')
plt.xlabel('Number of Transactions')

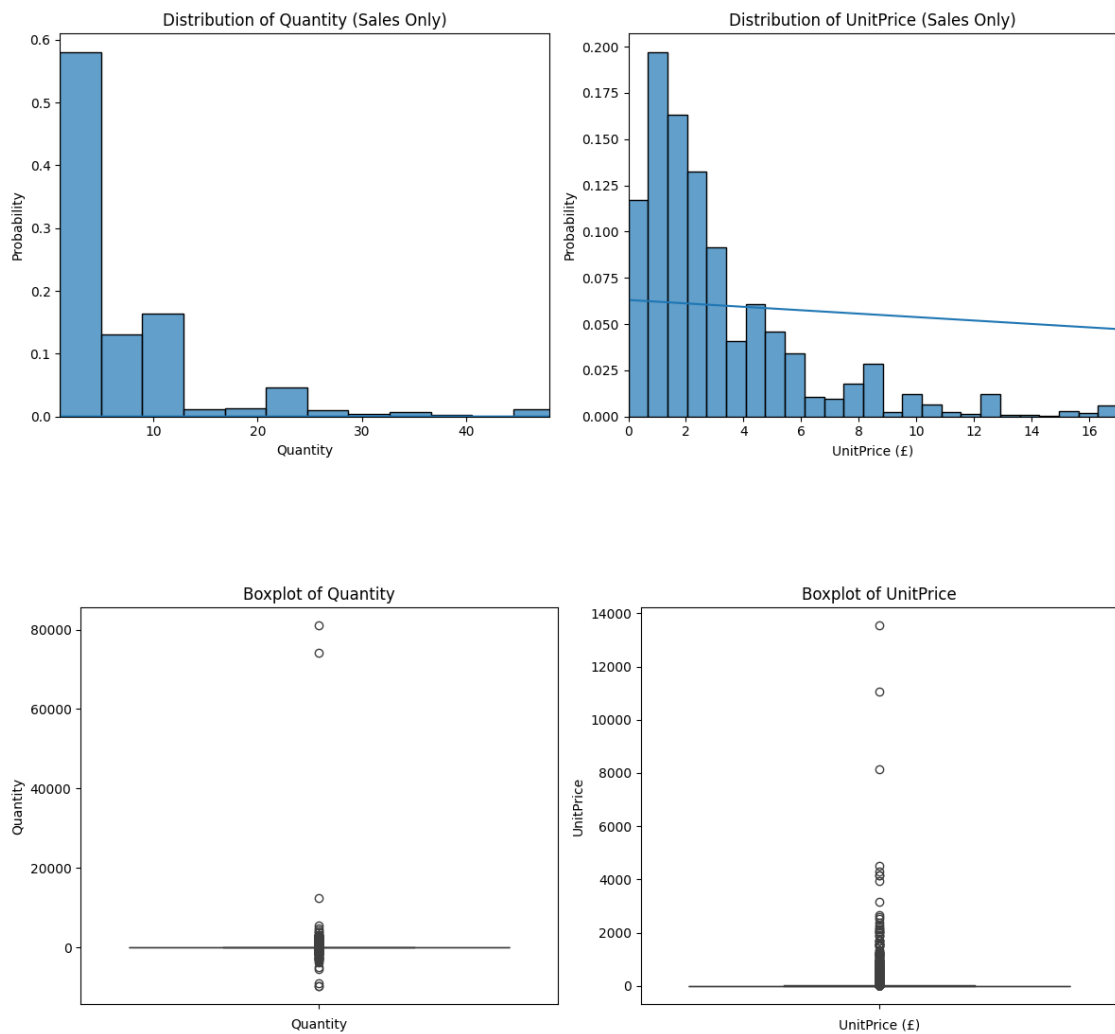
```

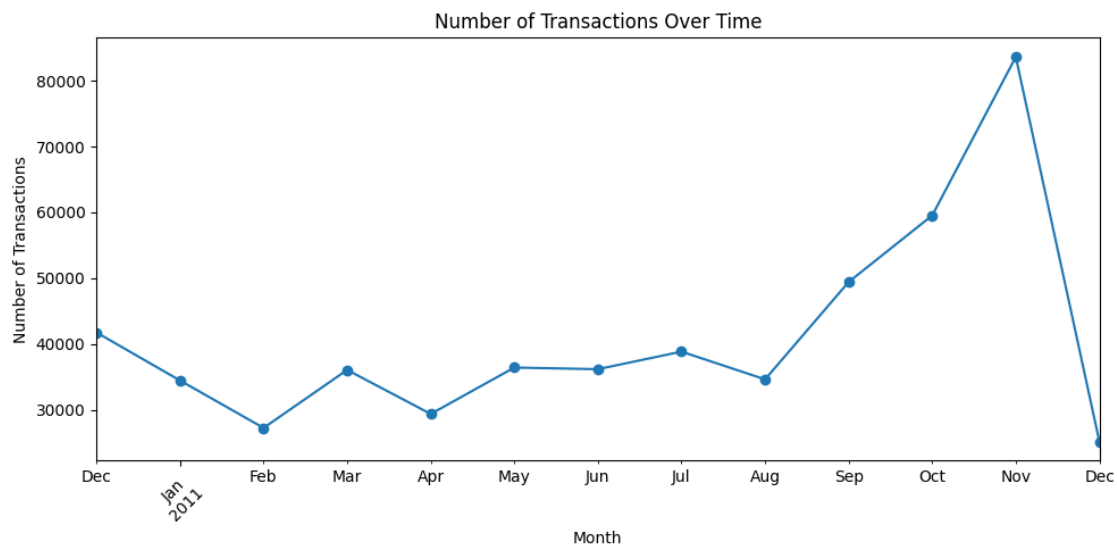
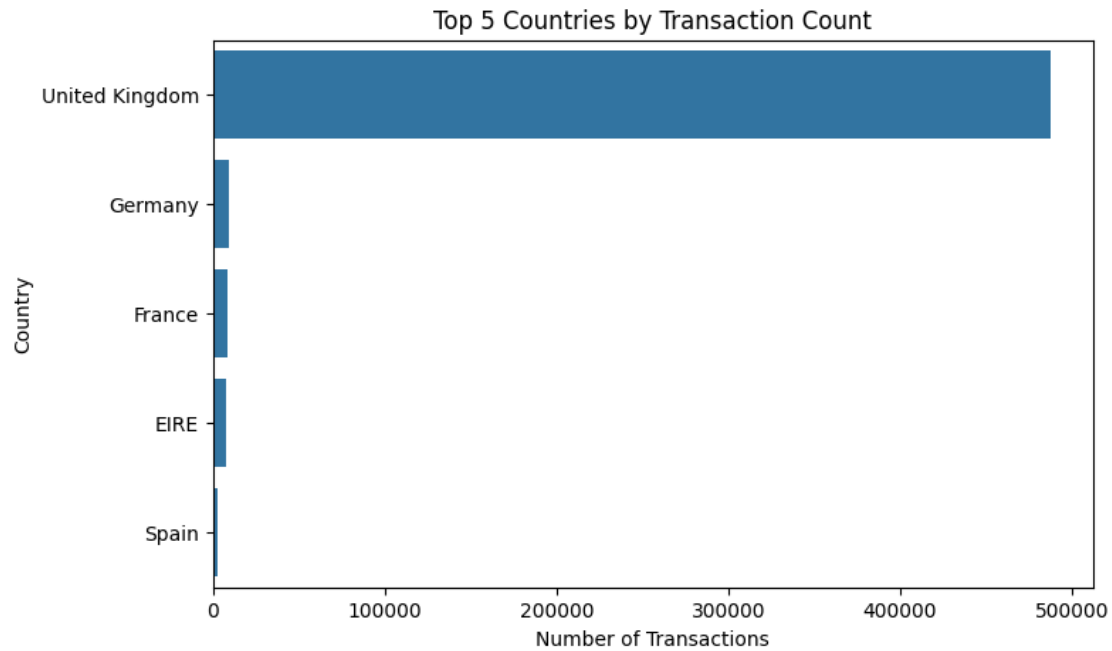
```

plt.ylabel('Country')
plt.show()

# Time series of transactions
valid_transactions['InvoiceMonth'] = valid_transactions['InvoiceDate'].dt.
    ↳to_period('M')
monthly_transactions = valid_transactions.groupby('InvoiceMonth').size()
plt.figure(figsize=(10, 5))
monthly_transactions.plot(kind='line', marker='o')
plt.title('Number of Transactions Over Time')
plt.xlabel('Month')
plt.ylabel('Number of Transactions')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

```





Descriptive Statistics for Categorical Features:

```
[21]: categorical_cols = ['Country', 'StockCode', 'CustomerID'] # CustomerID might
      ↪ have too many unique values

      # Categorical features summary
      print("\nUnique Values in Categorical Features:")
```

```

for col in categorical_cols:
    print(f"{col}: {valid_transactions[col].nunique()} unique values")
print(f"Cancellations: {valid_transactions['IsCancelled'].sum()}")

plt.figure(figsize=(15, 5))
for i, col in enumerate(categorical_cols, 1):
    plt.subplot(1, len(categorical_cols), i)
    if col == 'CustomerID':
        valid_transactions[valid_transactions['CustomerID'] != 'nan'][col].
        ↪value_counts().head(10).plot.bar() # Top 10 only
    else:
        valid_transactions[col].value_counts().head(10).plot.bar() # Top 10 only
    plt.title(f'Top 10 {col} values')
    plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

```

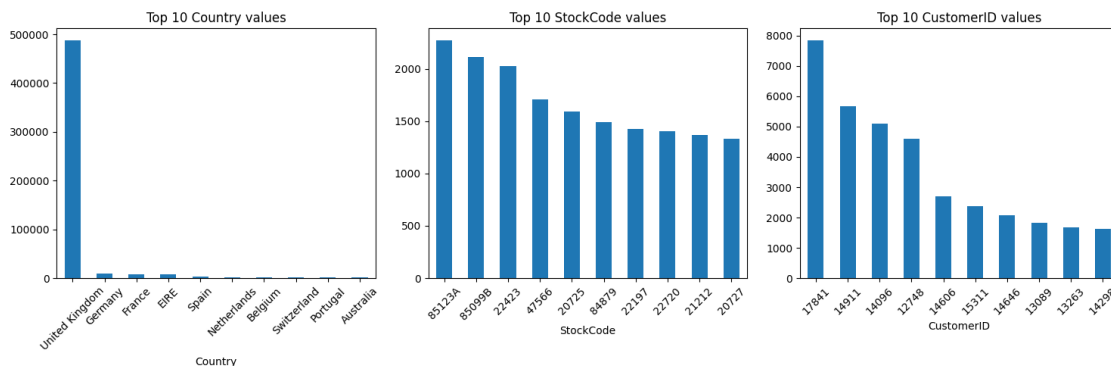
Unique Values in Categorical Features:

Country: 38 unique values

StockCode: 4059 unique values

CustomerID: 4340 unique values

Cancellations: 0



1.7.3 RFM Analysis (Recency, Frequency and Monetary)

An RFM analysis can be conducted to segment customers based on their purchasing behavior using the online retail dataset. This approach helps identify high-value customers and inform targeted marketing strategies. RFM analysis was chosen as a clustering approach to segment customers without requiring predictive modeling.

To perform an RFM (Recency, Frequency, Monetary) analysis on the provided online retail dataset, we'll analyze customer purchasing behavior based on three metrics: - Recency: how recently a customer made a purchase, - Frequency: how often they purchase, and - Monetary: how much they spend

CustomerID is critical for RFM analysis and cannot be imputed, we'll focus on records with non-null CustomerID values. We will also perform some additional cleanup of the data, including:

- Filter out cancellations (InvoiceNo starting with 'C').
- Remove rows with null CustomerID.
- Remove invalid records (e.g. negative quantities or prices, except for discounts).
- Calculate total transaction value (Quantity * UnitPrice).

Once the dataset is clean, we can calculate the RFM score. Quartile-based scoring (1 - 4) will be used for simplicity and interpretability.

```
[22]: # Setting up the environment
sns.set_style("whitegrid")
plt.rcParams["figure.figsize"] = (10, 6)

# Cleaning the dataset
def clean_data(df):
    """Clean the dataset by removing cancellations, nulls, and invalid records.
    ↪ """
    # Remove cancellations
    df_clean = df[~df['IsCancelled']]

    # Remove rows with null CustomerID or where CustomerID = 'nan'
    df_clean = df_clean[df_clean['CustomerID'].notnull() &
    ↪ df_clean['CustomerID'].isin(['nan'])]

    # Remove non-product entries (e.g., 'POSTAGE', 'Discount')
    df_clean = df_clean[~df_clean['StockCode'].isin(['POST', 'D'])]

    # Remove invalid quantities and prices
    df_clean = df_clean[(df_clean['Quantity'] > 0) & (df_clean['UnitPrice'] >
    ↪ 0)]

    # Verify no NaN CustomerIDs remain
    if df_clean['CustomerID'].isna().sum() > 0:
        print(f"Warning: {df_clean['CustomerID'].isna().sum()} NaN CustomerIDs
    ↪ remain after filtering.")

    return df_clean

# Calculating RFM metrics
def calculate_rfm(data):
    """Calculate Recency, Frequency, and Monetary metrics for each customer."""

    # Set reference date as one day after the last transaction
    reference_date = data['InvoiceDate'].max() + pd.Timedelta(days=1)

    # Group by CustomerID to determine RFM metrics
    rfm = data.groupby('CustomerID').agg({
```

```

        # Days since the customer's last purchase, relative to the latest date
        ↪ in the dataset
        'InvoiceDate': lambda x: (reference_date - x.max()).days,
        # Number of transactions per customer
        'InvoiceNo': 'nunique',
        # Total spend per customer
        'TotalPrice': 'sum' # Monetary
    }).reset_index()

    rfm.columns = ['CustomerID', 'Recency', 'Frequency', 'Monetary']

    return rfm

# Assigning RFM scores
def assign_rfm_scores(rfm):
    """Assign quartile-based scores (1-4) for Recency, Frequency, and Monetary.
    ↪ """
    # Recency: Lower is better (more recent)
    rfm['R_Score'] = pd.qcut(rfm['Recency'], 4, labels=[4, 3, 2, 1],
    ↪ duplicates='drop')

    # Frequency: Higher is better
    rfm['F_Score'] = pd.qcut(rfm['Frequency'].rank(method='first'), 4,
    ↪ labels=[1, 2, 3, 4])

    # Monetary: Higher is better
    rfm['M_Score'] = pd.qcut(rfm['Monetary'].rank(method='first'), 4,
    ↪ labels=[1, 2, 3, 4])

    # Combine RFM scores
    rfm['RFM_Segment'] = rfm['R_Score'].astype(str) + rfm['F_Score'].
    ↪ astype(str) + rfm['M_Score'].astype(str)
    rfm['RFM_Score'] = rfm['R_Score'].astype(int) + rfm['F_Score'].astype(int)
    ↪ + rfm['M_Score'].astype(int)

    return rfm

# Clean data
df_clean = clean_data(valid_transactions)
print(f"Cleaned dataset shape: {df_clean.shape}")

# Calculate RFM
rfm = calculate_rfm(df_clean)
print(f"RFM dataset shape: {rfm.shape}")

# Assign RFM scores

```

```
rfm = assign_rfm_scores(rfm)

# Save RFM results
# rfm.to_csv('rfm_results.csv', index=False)
```

Cleaned dataset shape: (396785, 12)

RFM dataset shape: (4338, 4)

```
[23]: rfm.head()
```

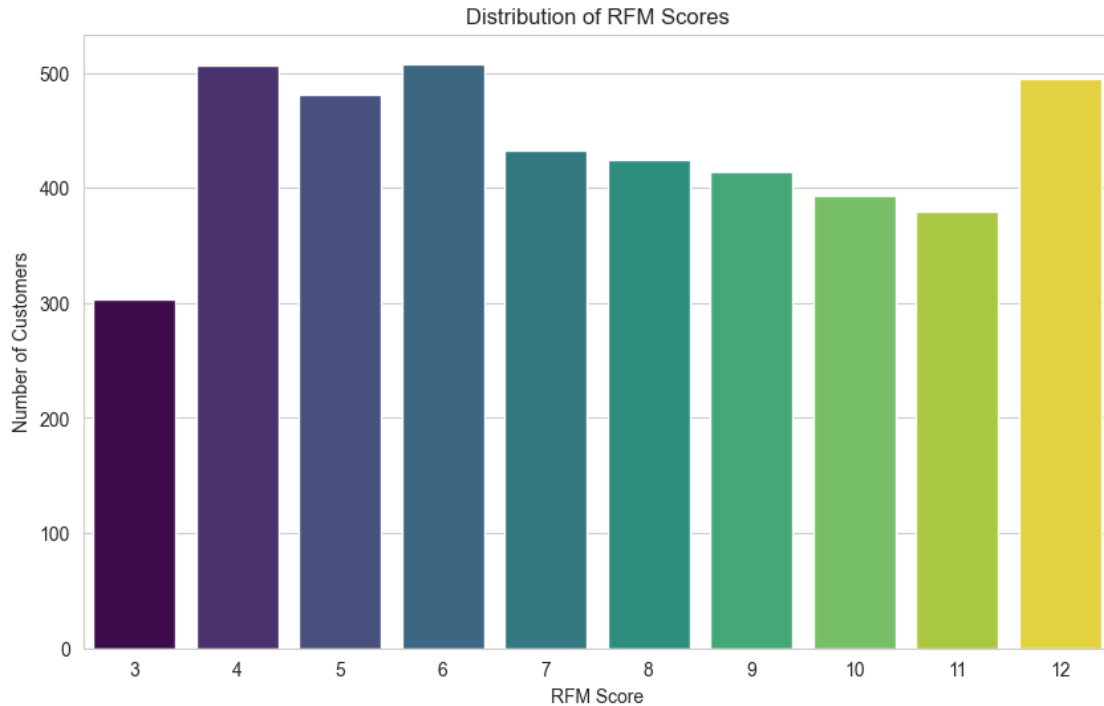
```
[23]:
```

	CustomerID	Recency	Frequency	Monetary	R_Score	F_Score	M_Score	\
0	12346	326	1	77183.60	1	1	4	
1	12347	2	7	4310.00	4	4	4	
2	12348	75	4	1437.24	2	3	3	
3	12349	19	1	1457.55	3	1	3	
4	12350	310	1	294.40	1	1	1	

	RFM_Segment	RFM_Score
0	114	6
1	444	12
2	233	8
3	313	7
4	111	3

Section 1: Distribution of RFM Scores

```
[24]: plt.figure()
sns.countplot(x='RFM_Score', data=rfm, palette='viridis', hue='RFM_Score',
             legend=False)
plt.title('Distribution of RFM Scores')
plt.xlabel('RFM Score')
plt.ylabel('Number of Customers')
plt.show()
plt.close()
```

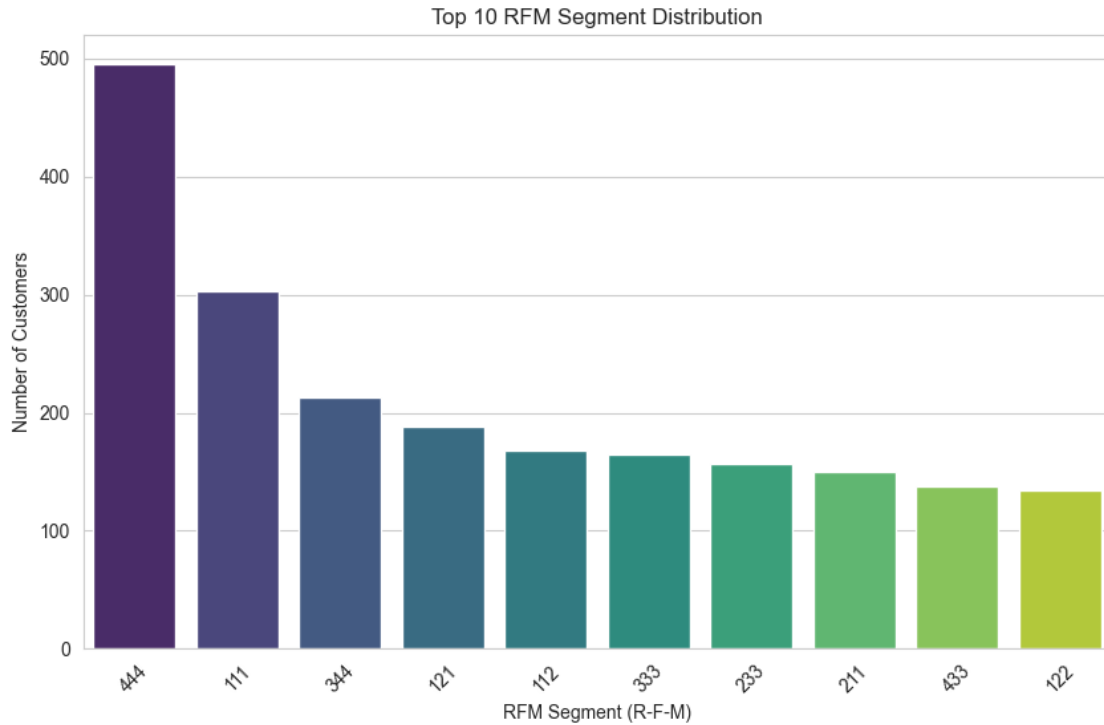


RFM Segment Distribution (Bar Plot)

Shows the count of customers in each RFM segment (e.g., '444' for best customers, '111' for lost customers) to highlight the size of key customer groups. This visualization makes it easier to understand the proportion of high-value vs. low-value customers, which is critical for marketing strategies.

For segments with many customers, we emphasis retention strategies, and for segments with few customers, we emphasise re-engagement strategies.

```
[25]: plt.figure()
      # Count top 10 RFM segments for clarity
      top_segments = rfm['RFM_Segment'].value_counts().head(10)
      sns.barplot(x=top_segments.index, hue=top_segments.index, y=top_segments.
        ↪values, legend=False, palette='viridis')
      plt.title('Top 10 RFM Segment Distribution')
      plt.xlabel('RFM Segment (R-F-M)')
      plt.ylabel('Number of Customers')
      plt.xticks(rotation=45)
      plt.show()
      plt.close()
```



This bar shows our 'Best Customers' (444) who shop frequently, recently, and spend a lot.

Analysis Overall Distribution: - The total number of customers ranges from approximately 100 to 500, with a noticeable variation across RFM scores. - The distribution is not uniform, with a peak around the middle scores and a decline toward the extremes.

Trends: - The distribution shows a bimodal pattern, with peaks at RFM scores 5 and 12, suggesting two dominant customer segments: moderately engaged (score 5) and highly engaged (score 12). - Scores 3 and 4 have fewer customers, indicating that very low engagement is less common. - There's a general decline from score 5 to 11, with a sharp rise at 12, highlighting that the best customers (RFM 12) are a significant portion of the customer base.

Insights for Business

- **Highly Engaged Customers (RFM 12):** The large number of customers with an RFM score of 12 (around 450 - 500) indicates a strong segment of recent, frequent, and high-spending customers. These are the most valuable clients and should be prioritized for retention through loyalty programs or exclusive offers.
- **Moderately Engaged Customers (RFM 5):** The peak at score 5 (450 - 500 customers) suggests a large middle ground of customers with moderate recency, frequency, or monetary value. Upselling or increasing purchase frequency could move them toward higher scores.
- **Low Engagement (RFM 3 - 4):** The smaller counts at scores 3 and 4 (100 - 450 customers) indicate a minority of less engaged customers. Re-engagement campaigns targeting these customers could prevent churn.

- Declining Middle Segment (RFM 6 - 11): The gradual decrease from 6 to 11 suggests a natural tapering of engagement levels, with fewer customers achieving higher combined scores until the spike at 12.

Additional RFM Scores Analysis Box Plot of RFM Metrics by RFM Score

Display the distribution (median, quartiles, outliers) of Recency, Frequency, and Monetary values across RFM scores to show how these metrics vary by customer value. This highlights the range and variability within each RF Liberation score, helping identify outliers (e.g. customers with unusually high spending).

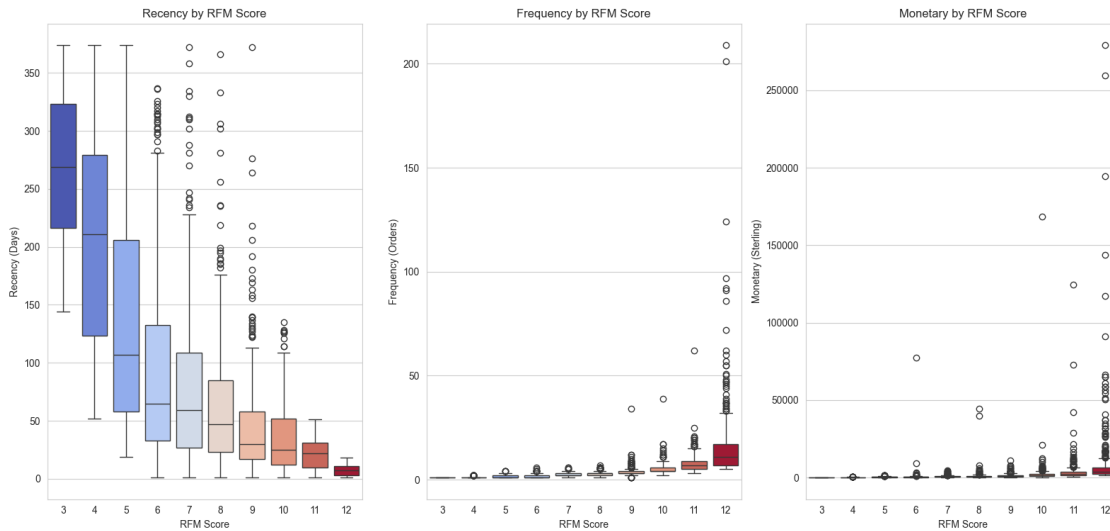
```
[26]: plt.figure(figsize=(16, 8))
plt.suptitle('RFM Analysis: Recency, Frequency, and Monetary by RFM Score',
            ↪fontsize=16, y=1.05)

plt.subplot(1, 3, 1)
sns.boxplot(x='RFM_Score', y='Recency', hue='RFM_Score', data=rfm,
            ↪legend=False, palette='coolwarm')
plt.title('Recency by RFM Score')
plt.xlabel('RFM Score')
plt.ylabel('Recency (Days)')

plt.subplot(1, 3, 2)
sns.boxplot(x='RFM_Score', y='Frequency', hue='RFM_Score', data=rfm,
            ↪legend=False, palette='coolwarm')
plt.title('Frequency by RFM Score')
plt.xlabel('RFM Score')
plt.ylabel('Frequency (Orders)')

plt.subplot(1, 3, 3)
sns.boxplot(x='RFM_Score', y='Monetary', hue='RFM_Score', data=rfm,
            ↪legend=False, palette='coolwarm')
plt.title('Monetary by RFM Score')
plt.xlabel('RFM Score')
plt.ylabel('Monetary (Sterling)')

plt.tight_layout()
plt.show()
plt.close()
```

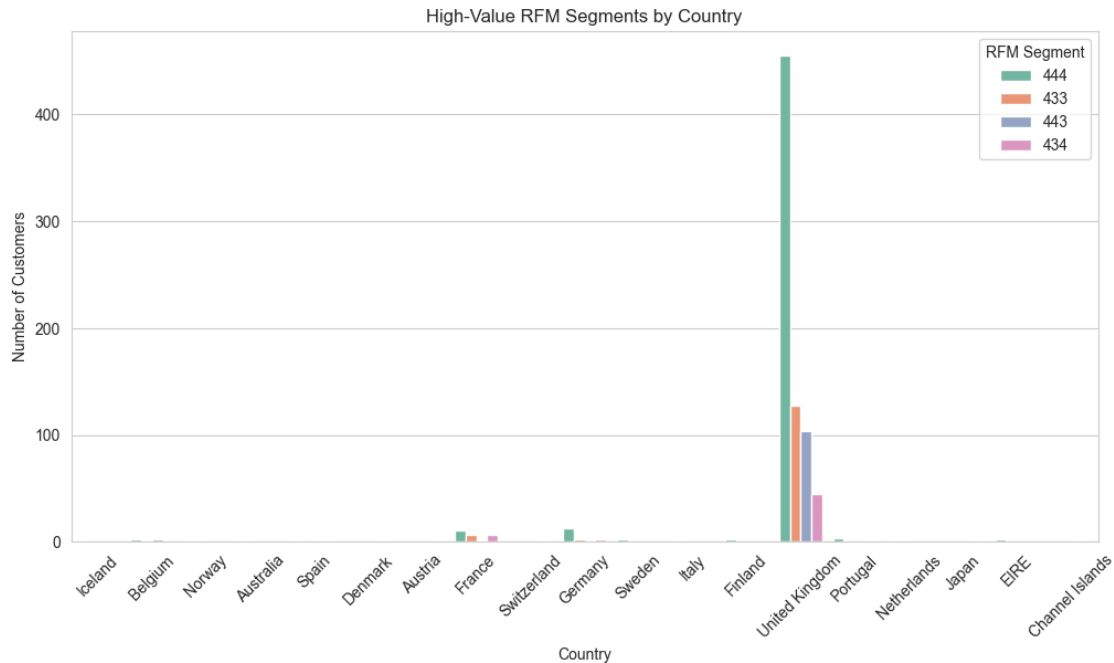


These boxplots show that our best customers (high RFM scores) shop often and spend more, but some spend way above average, like superstars!

Customer Count by Country for Top RFM Segments

This visualizes the number of customers in top RFM segments (e.g., '444', '433') by country to identify geographic patterns in high-value customer distribution. This can reveal if high-value customers are concentrated in specific regions (e.g., UK vs. international), informing targeted marketing. The EDA notes most transactions are UK-based, so this checks for international potential.

```
[27]: rfm_with_country = rfm.merge(valid_transactions[['CustomerID', 'Country']],
    ↪drop_duplicates(), on='CustomerID')
top_segments = rfm_with_country[rfm_with_country['RFM_Segment'].isin(['444', '433', '443', '434'])]
plt.figure(figsize=(12, 6))
sns.countplot(data=top_segments, x='Country', hue='RFM_Segment', palette='Set2')
plt.title('High-Value RFM Segments by Country')
plt.xlabel('Country')
plt.ylabel('Number of Customers')
plt.xticks(rotation=45)
plt.legend(title='RFM Segment')
plt.show()
plt.close()
```



Most high-value customers (RFM ‘444’) are in the UK, but Germany and France show potential for targeted campaigns.

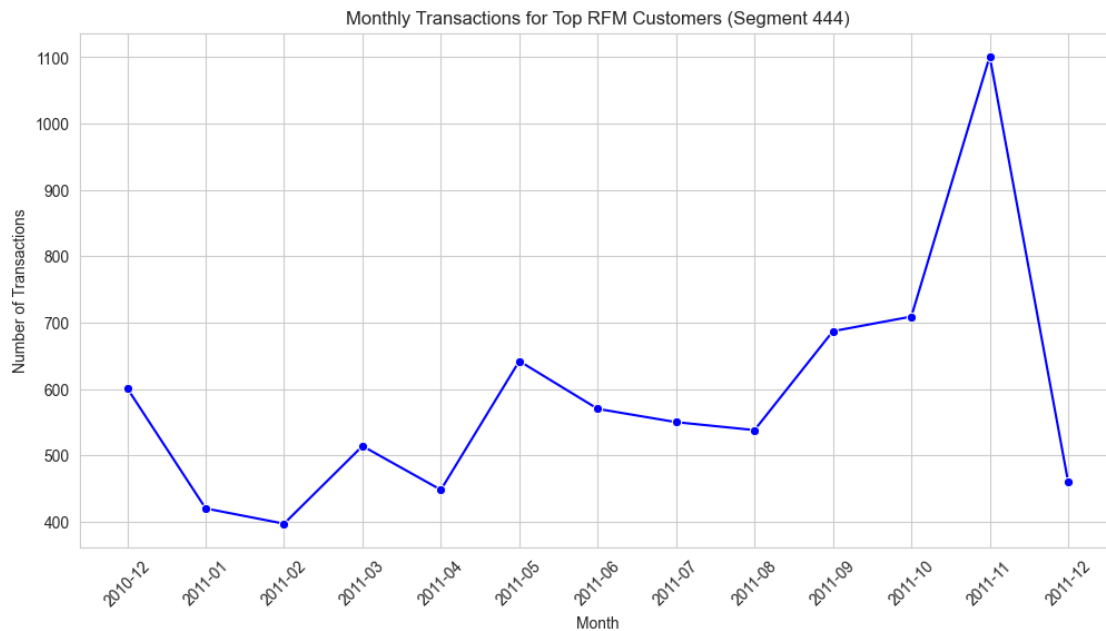
Time Series of Transactions for Top RFM Customers

Plot the number of transactions over time for customers in the top RFM segment (e.g., ‘444’) to show purchasing patterns. This reveals seasonal or temporal trends (e.g., holiday spikes) for high-value customers, which is valuable for planning marketing campaigns. The dataset spans December 2010 to December 2011, so this can highlight key periods.

```
[28]: top_customers = rfm[rfm['RFM_Segment'] == '444']['CustomerID']
df_top = valid_transactions[valid_transactions['CustomerID'].
    ↪isin(top_customers)][['InvoiceDate', 'InvoiceNo']].copy()
df_top['InvoiceDate'] = pd.to_datetime(df_top['InvoiceDate'])
df_top['Month'] = df_top['InvoiceDate'].dt.to_period('M')
monthly_counts = df_top.groupby('Month')['InvoiceNo'].nunique().reset_index()
monthly_counts['Month'] = monthly_counts['Month'].astype(str)

plt.figure(figsize=(12, 6))
sns.lineplot(x='Month', y='InvoiceNo', data=monthly_counts, marker='o',
    ↪color='b')
plt.title('Monthly Transactions for Top RFM Customers (Segment 444)')
plt.xlabel('Month')
plt.ylabel('Number of Transactions')
plt.xticks(rotation=45)
plt.show()
```

```
plt.close()
```



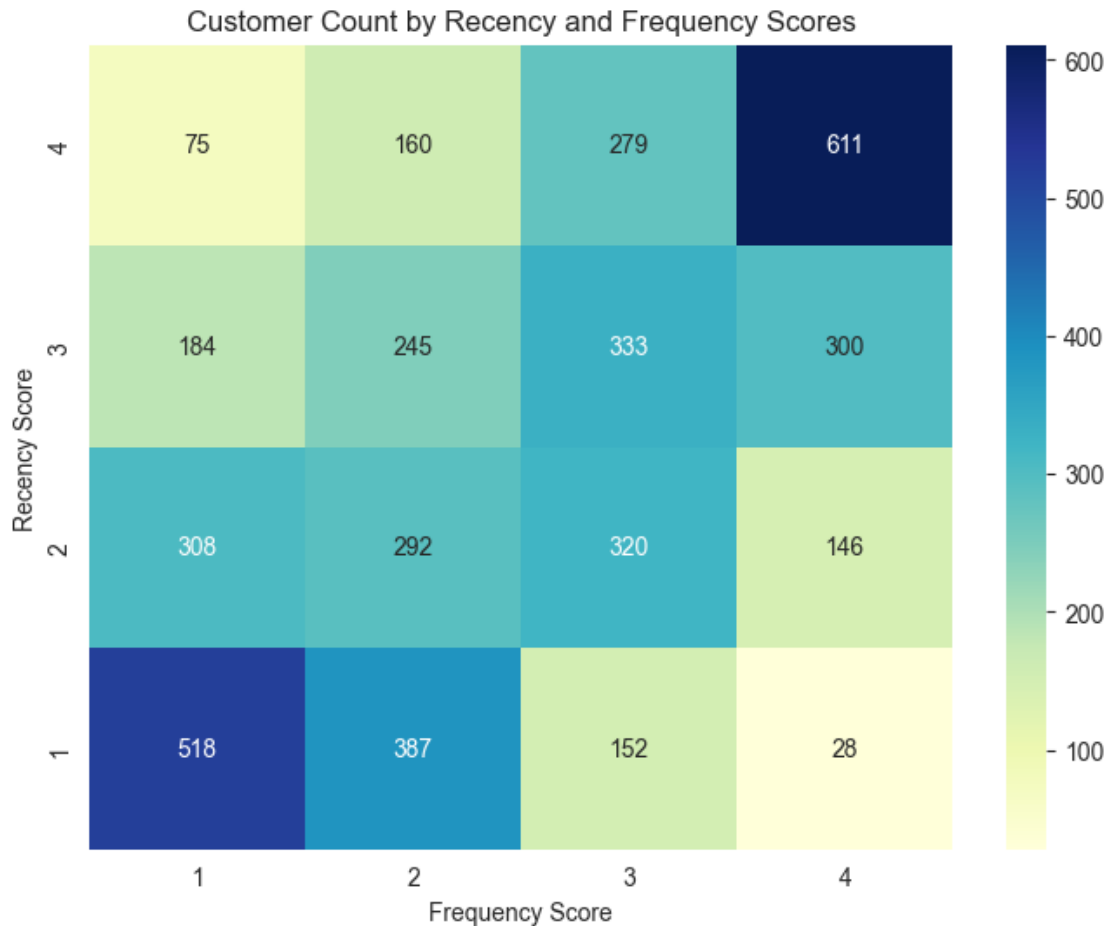
Top RFM customers show increased activity in November - December, likely due to holiday shopping. This graph shows our best customers shop more around the holidays (Christmas and Easter), so we can plan big sales for November and December.

Heatmap of RFM Score Combinations

Below is a heatmap of customer counts across combinations of R, F, and M scores to show which combinations are most common. This provides a compact view of how Recency, Frequency, and Monetary scores interact, highlighting dominant customer profiles.

The heatmap is essentially a pivot table where the x-axis represents Frequency (F) scores, the y-axis represents Recency (R) scores, and the color intensity (with annotations) indicates the number of customers in each R-F combination. This visualization helps identify the most common customer profiles based on their RFM scores.

```
[29]: rf_pivot = rfm.pivot_table(index='R_Score', columns='F_Score',  
    ↪ values='CustomerID', aggfunc='count', fill_value=0, observed=False)  
plt.figure(figsize=(8, 6))  
sns.heatmap(rf_pivot, annot=True, fmt='d', cmap='YlGnBu')  
plt.title('Customer Count by Recency and Frequency Scores')  
plt.xlabel('Frequency Score')  
plt.ylabel('Recency Score')  
plt.show()  
plt.close()
```



Dominant Profile: The heatmap shows 611 customers with R=4 and F=4, indicating strong recent and frequent activity, aligning with top spenders like Customer 14646.

At-Risk Segment: A notable 1-1 or 1-2 area suggests churn risk. With 518 customers in the 1-1 segment, re-engagement strategies are needed for those inactive since late 2010.

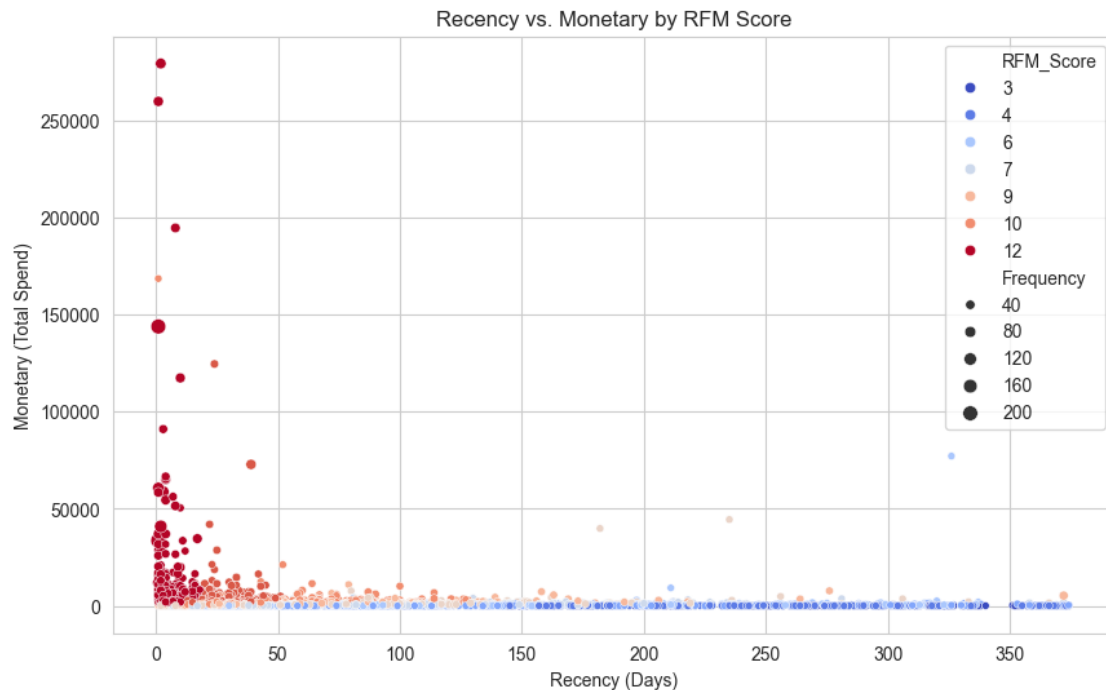
Growth Opportunity: A 4-1 (75 customers) or 4-2 (160 customers) clusters indicate new customers. Offering discounts could boost their frequency.

Business action: Focus retention on 4-4/4-3, re-engagement on 1-1, and growth on 4-1

Section 2: Recency vs. Monetary Scatter

```
[30]: plt.figure()
sns.scatterplot(x='Recency', y='Monetary', hue='RFM_Score', size='Frequency',
               data=rfm, palette='coolwarm')
plt.title('Recency vs. Monetary by RFM Score')
plt.xlabel('Recency (Days)')
plt.ylabel('Monetary (Total Spend)')
plt.show()
```

```
plt.close()
```



The “Recency vs. Monetary by RFM Score” scatter plot provides insights into customer behavior based on two key RFM (Recency, Frequency, Monetary) metrics: Recency (days since the last purchase) on the x-axis and Monetary (total spend in pounds) on the y-axis. The plot uses color and size to represent RFM scores and frequency, respectively.

Analysis Recency Distribution:

- Most customers are clustered between 0 and 100 days, suggesting a significant portion made purchases relatively recently.
- A smaller number of customers extend toward 300–350 days, indicating some have not purchased in nearly a year and may be at risk of churn.

Monetary Distribution:

- The y-axis ranges from 0 to 250,000 pounds, showing a wide range of total spending. There are a few outliers that have spend exceeding 150,000 pounds.
- The majority of customers have monetary values below 50,000 pounds, with a dense cluster near the lower end.

RFM Score Distribution:

- **Low RFM Scores (3 - 7, blue to light blue):** These customers, with lower combined RFM scores, are concentrated at lower monetary values (below 50,000 pounds) and higher recency values (100 - 350 days). This suggests they are less recent, less frequent, or lower spenders, potentially indicating disengagement.
- **Moderate RFM Scores (9 - 10, orange):** These customers appear in the middle range, with monetary values between 10,000 and 100,000 pounds and recency mostly below 150 days. They represent a moderately active segment.
- **High RFM Scores (12, red):** Customers with an RFM score of 12 (the highest) are sparse but notable, with monetary values ranging from 50,000 to over 200,000 pounds and recency typically below 100 days. These are the most valuable customers—recent, frequent, and high-spending.

Frequency Influence:

- The size of the points represents frequency (number of transactions), with larger points indicating higher frequency. - Larger points (e.g., 40 - 200 transactions) are more prominent among high RFM scores (red and orange), especially at higher monetary values, confirming that frequent purchases correlate with higher spending. - Smaller points (lower frequency) dominate the lower RFM scores (blue), particularly at higher recency values.

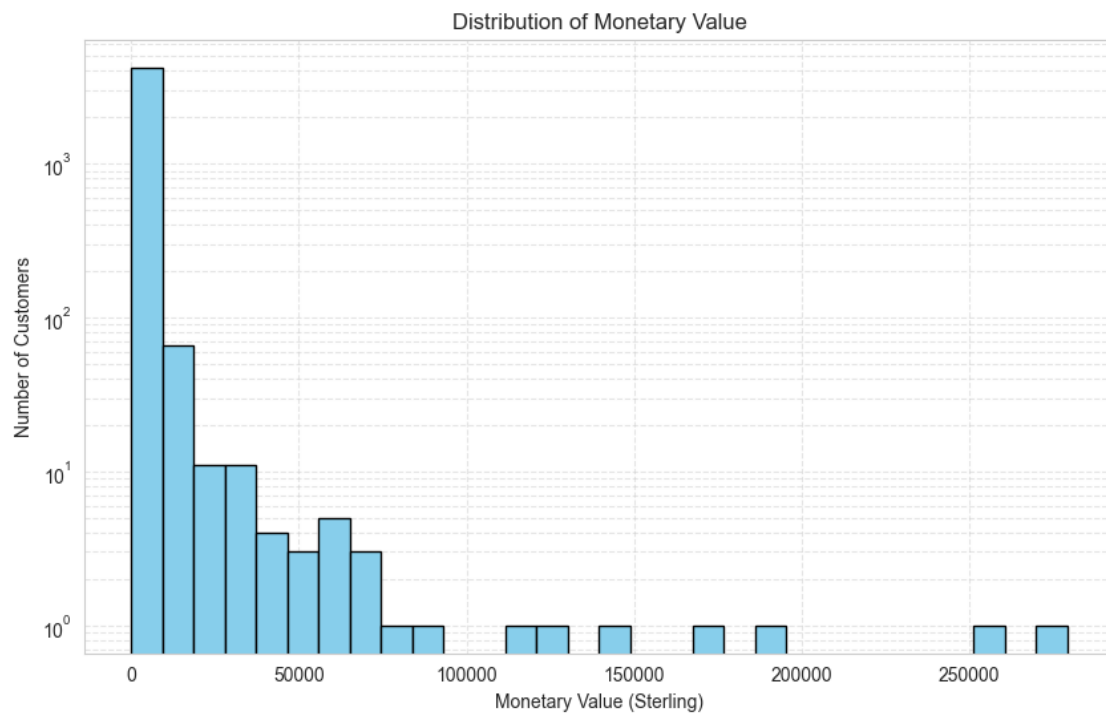
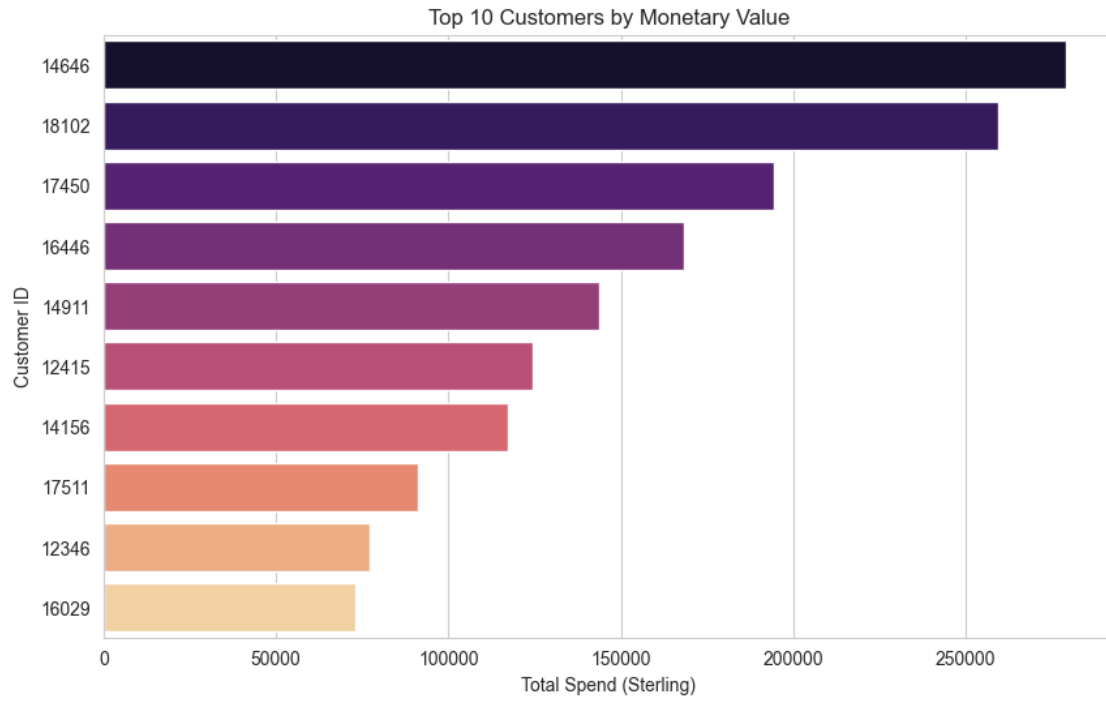
Insights for Business

- High-Value Customers: The red points (RFM score 12) indicate a small but critical segment of recent, frequent, and high-spending customers. Targeted loyalty programs or personalized offers could retain these valuable clients.
- At-Risk Customers: Blue points (RFM scores 3-7) at high recency values suggest customers who haven't purchased in months. Marketing campaigns to re-engage them could prevent churn.
- Growth Opportunities: Orange points (RFM scores 9-10) represent a middle tier with potential. Encouraging more frequent purchases or upselling could elevate them to the high-value segment.
- Frequency Correlation: The larger points among high monetary values highlight that frequency drives spending. Incentives for repeat purchases could boost overall revenue.

Section 3: Top 10 Customers by Monetary Value

```
[31]: top_customers = rfm.nlargest(10, 'Monetary')
plt.figure()
sns.barplot(x='Monetary', y='CustomerID', hue='CustomerID', data=top_customers,
            palette='magma')
plt.title('Top 10 Customers by Monetary Value')
plt.xlabel('Total Spend (Sterling)')
plt.ylabel('Customer ID')
plt.show()
plt.close()

# Plot histogram of Monetary value
plt.hist(rfm['Monetary'], bins=30, color='skyblue', edgecolor='black')
plt.title('Distribution of Monetary Value')
plt.xlabel('Monetary Value (Sterling)')
plt.ylabel('Number of Customers')
plt.yscale('log') # Using log scale for better visibility of distribution
plt.grid(True, which="both", ls="--", alpha=0.5)
plt.show()
plt.close()
```



Analysis: The histogram of monetary value shows a strongly right-skewed distribution, with most customers spending less than 50,000 Sterling, a long tail extending to 250,000 Sterling, and a few peaks corresponding to high-value customers.

The log scale helps to reveal the density of low-to-moderate spenders while still showing the rare high spenders.

Insights for Business:

- **High-Value Concentration:** Customers 14646 and 18102 account for the bulk of the top spending (over 200,000 and nearly 200,000 Sterling, respectively), representing a critical segment for retention (and profitability). Personalized loyalty programs or exclusive offers could maintain their engagement.
- **Mid-Tier Value:** Customers like 17450, 16446, and 14911 (100,000 - 175,000 Sterling) form a second tier of valuable clients. Strategies to increase their frequency or average order value could elevate them closer to the top.
- **Lower Tier in Top 10:** Customers 14156, 17511, 12346, and 16029 (25,000 - 75,000 Sterling) indicate potential for growth. Upselling or cross-selling could boost their spending to match higher tiers.
- **Revenue Impact:** The top 10 customers collectively contribute a significant portion of revenue, with the top two alone accounting for over 400,000 Sterling. Focusing on these customers could maximize return on investment.
- **Segmentation Opportunity:** On the histogram, the drop-off after 50,000 Sterling suggests potential to target mid-range spenders (50,000 - 100,000 Sterling) for upselling or loyalty programs to move them toward higher spending tiers.

1.8 Clustering

Scaling the RFM to make cluster

```
[32]: # Preprocessing for clustering
def preprocess_rfm(rfm):
    rfm_features = rfm[['Recency', 'Frequency', 'Monetary']]
    scaler = StandardScaler()
    rfm_scaled = scaler.fit_transform(rfm_features)
    return rfm_scaled, rfm_features
```

```
[33]: rfm_scaled, rfm_features = preprocess_rfm(rfm)
```

```
[34]: rfm_scaled
```

```
[34]: array([[ 2.33421005e+00, -4.24307820e-01,  8.38791567e+00],
        [-9.05614995e-01,  3.56620192e-01,  2.53786252e-01],
        [-1.75654415e-01, -3.38438143e-02, -6.68703598e-02],
        ...,
        [-8.45618235e-01, -2.94153152e-01, -2.07420771e-01],
        [-8.85616075e-01,  1.52801221e+00,  6.53522016e-03],
        [-4.95637135e-01, -1.63998483e-01, -2.22180147e-02]],
        shape=(4338, 3))
```

Clustering:

- K-Means: Uses scaled RFM features to group customers into clusters based on similarity.
- Hierarchical: Uses Ward's method to build a hierarchy of clusters, visualized via a dendrogram.

```
[35]: # Determining optimal number of clusters
def find_optimal_clusters(rfm_scaled):
    silhouette_scores = []
    ch_scores = []
    K = range(2, 10)

    for k in K:
        kmeans = KMeans(n_clusters=k, random_state=42)
        labels = kmeans.fit_predict(rfm_scaled)
        silhouette_scores.append(silhouette_score(rfm_scaled, labels))
        ch_scores.append(calinski_harabasz_score(rfm_scaled, labels))

    # Plotting silhouette and Calinski-Harabasz scores
    plt.figure(figsize=(12, 5))
    plt.subplot(1, 2, 1)
    plt.plot(K, silhouette_scores, 'bx-')
    plt.xlabel('k')
    plt.ylabel('Silhouette Score')
    plt.title('Silhouette Score vs. k')
    plt.grid(True)

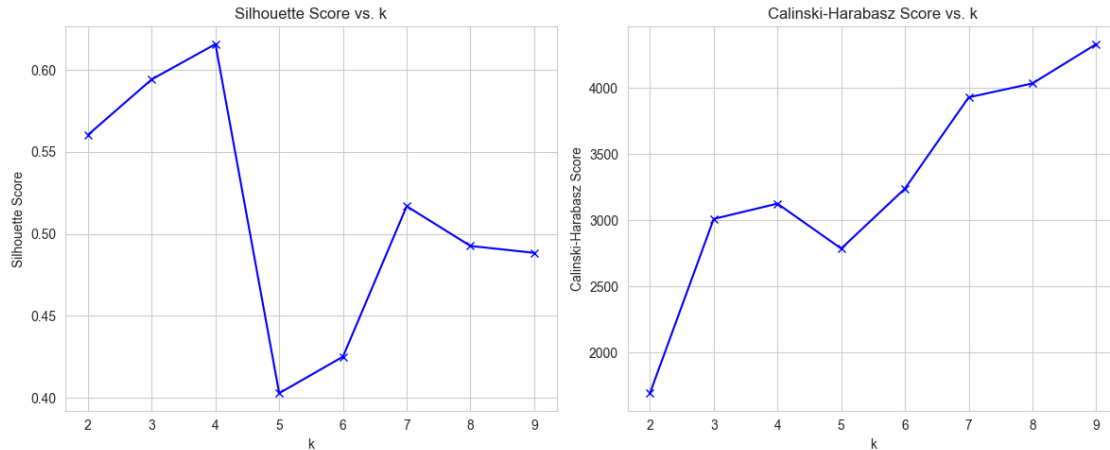
    plt.subplot(1, 2, 2)
    plt.plot(K, ch_scores, 'bx-')
    plt.xlabel('k')
    plt.ylabel('Calinski-Harabasz Score')
    plt.title('Calinski-Harabasz Score vs. k')
    plt.grid(True)
    plt.tight_layout()
    # plt.savefig('cluster_evaluation.png')
    plt.show()

    optimal_k = K[np.argmax(silhouette_scores)]

    return optimal_k, silhouette_scores, ch_scores
```

Optimal Clusters: Determined by maximizing Silhouette Score (measures cluster cohesion) and Calinski-Harabasz Score (measures cluster separation).

```
[36]: # Finding optimal clusters
optimal_k, sil_scores, ch_scores = find_optimal_clusters(rfm_scaled)
```



```
[37]: print(f"Optimal number of clusters (K-Means): {optimal_k}")
      print(f"silhouette_scores (K-Means): {sil_scores}")
      print(f"Calinski-Harabasz Score (K-Means): {ch_scores}")
```

```
Optimal number of clusters (K-Means): 4
silhouette_scores (K-Means): [np.float64(0.5603967985381435),
np.float64(0.5942151572646268), np.float64(0.6157254282479783),
np.float64(0.40286021074708533), np.float64(0.42511885526971477),
np.float64(0.516867399929898), np.float64(0.4926542652405674),
np.float64(0.4884390732104456)]
'Calinski-Harabasz Score (K-Means): [np.float64(1694.013202467827),
np.float64(3011.374843159982), np.float64(3126.888344040915),
np.float64(2787.20095264403), np.float64(3240.580987943109),
np.float64(3930.29510861536), np.float64(4035.305504850242),
np.float64(4332.52019808885)]
```

In clustering (e.g., K-Means or hierarchical clustering), the optimal number of clusters (k) is determined by evaluating how well the data is grouped for different values of k . Here we see clusters ranging from 2 to 9 and uses two metrics to assess cluster quality: - Silhouette Score: Measures how similar each point is to its own cluster compared to other clusters (range: -1 to 1, higher is better). - Calinski-Harabasz Score: Measures the ratio of between-cluster dispersion to within-cluster dispersion (no upper bound, higher is better).

The above process illustrates:

- Running K-Means for each k in the range (2 to 9).
- Computing Silhouette and Calinski-Harabasz scores for each k .
- Selecting the k that optimizes one or both metrics, typically by looking for the highest score or a “knee” point where adding more clusters yields diminishing returns.

We took Silhouette Score to calculate for an Optimal k It helps us with the interpretability of the results: Since, the Silhouette Score is bounded (-1 to 1), it is easier to compare across datasets and interpret (e.g., >0.5 is good, <0.2 is poor). Solhouette scores is also more robust, it evaluates both

intra-cluster cohesion (how tight clusters are) and inter-cluster separation (how distinct clusters are), providing a balanced metric. It is also very sensitive to the quality of individual points

Business Context In our RFM analysis, selecting the optimal k is critical for meaningful customer segmentation:

Too few clusters (e.g., k=2): Might merge High-Value and Loyal customers, missing nuanced marketing opportunities. Too many clusters (e.g., k=8): Might split similar customers unnecessarily, complicating strategies. Optimal k (e.g., k=4): Likely yields segments like High-Value, Loyal, At-Risk, and Emerging, aligning with actionable marketing strategies (e.g., loyalty programs, re-engagement campaigns).

```
[38]: # Applying K-Means clustering
def apply_kmeans(rfm_scaled, n_clusters):
    kmeans = KMeans(n_clusters=n_clusters, random_state=42)
    labels = kmeans.fit_predict(rfm_scaled)
    sil_score = silhouette_score(rfm_scaled, labels)
    return labels, kmeans, sil_score

[39]: # Visualizing clusters
# def visualize_clusters(rfm, labels, method='KMeans'):
#     rfm['Cluster'] = labels
#     fig = px.scatter_3d(rfm, x='Recency', y='Frequency', z='Monetary',
# ↪color='Cluster',
#                               title=f'Customer Segments ({method})', size_max=15)
#     # fig.write_to_file('cluster_3d.html')

#     # Pairplot
#     sns.pairplot(rfm, vars=['Recency', 'Frequency', 'Monetary'],
# ↪hue='Cluster', diag_kind='kde')
#     # plt.savefig(f'pairplot_{method.lower()}.png')
#     # plt.close()
def visualize_clusters(rfm, labels, method='KMeans'):
    """
    Visualize RFM customer segments in a Databricks notebook using 2D plots.

    Parameters:
    - rfm: DataFrame with columns ['CustomerID', 'Recency', 'Frequency',
    ↪'Monetary']
    - labels: Cluster labels from K-Means
    - method: Clustering method name (default: 'KMeans')
    """
    rfm = rfm.copy()
    rfm['Cluster'] = labels.astype(str) # Convert labels to string for better
    ↪coloring

    try:
        # 1. Pairplot (Seaborn)
```

```

plt.figure(figsize=(10, 8))
sns.pairplot(
    rfm,
    vars=['Recency', 'Frequency', 'Monetary'],
    hue='Cluster',
    diag_kind='kde',
    palette='Set1',
    plot_kws={'alpha': 0.6, 's': 30}
)
plt.suptitle(f'Pairplot of RFM by Cluster ({method})', y=1.02)
plt.show()
print("Displayed pairplot.")

# 2. Cluster Profile Bar Plot (Plotly)
cluster_means = rfm.groupby('Cluster')[['Recency', 'Frequency', 'Monetary']].mean().reset_index()
fig = go.Figure()
for metric in ['Recency', 'Frequency', 'Monetary']:
    fig.add_trace(go.Bar(
        x=cluster_means['Cluster'],
        y=cluster_means[metric],
        name=metric
    ))
fig.update_layout(
    title=f'Cluster Profiles: Mean RFM Values ({method})',
    xaxis_title='Cluster',
    yaxis_title='Value',
    barmode='group',
    template='plotly_dark'
)
fig.show()
print("Displayed cluster profile bar plot.")

# 3. Box Plots (Seaborn)
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
for idx, metric in enumerate(['Recency', 'Frequency', 'Monetary']):
    sns.boxplot(x='Cluster', y=metric, hue='Cluster', data=rfm,
ax=axes[idx], palette='Set2')
    axes[idx].set_title(f'{metric} Distribution by Cluster')
plt.tight_layout()
plt.show()
print("Displayed box plots.")

# 4. Heatmap of Standardized Centroids (Seaborn)
scaler = StandardScaler()
rfm_scaled = scaler.fit_transform(rfm[['Recency', 'Frequency', 'Monetary']])

```

```

rfm_scaled_df = pd.DataFrame(rfm_scaled, columns=['Recency',
↪ 'Frequency', 'Monetary'])
rfm_scaled_df['Cluster'] = rfm['Cluster']
centroids = rfm_scaled_df.groupby('Cluster')[['Recency', 'Frequency',
↪ 'Monetary']].mean()

plt.figure(figsize=(8, 6))
sns.heatmap(
    centroids,
    annot=True,
    cmap='coolwarm',
    center=0,
    fmt='.2f'
)
plt.title(f'Standardized Cluster Centroids ({method})')
plt.show()
print("Displayed centroid heatmap.")

# 5. 2D Scatter Plots (Plotly)
for x_metric, y_metric in [('Recency', 'Monetary'), ('Recency',
↪ 'Frequency'), ('Frequency', 'Monetary')]:
    fig = px.scatter(
        rfm,
        x=x_metric,
        y=y_metric,
        color='Cluster',
        title=f'{x_metric} vs. {y_metric} by Cluster ({method})',
        hover_data=['CustomerID'],
        color_discrete_sequence=px.colors.qualitative.Bold,
        opacity=0.6
    )
    fig.show()
    print(f"Displayed 2D scatter plot: {x_metric} vs. {y_metric}")

# 6. Violin Plots (Seaborn)
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
for idx, metric in enumerate(['Recency', 'Frequency', 'Monetary']):
    sns.violinplot(x='Cluster', y=metric, hue='Cluster', data=rfm,
↪ ax=axes[idx], palette='Set3')
    axes[idx].set_title(f'{metric} Distribution by Cluster')
plt.tight_layout()
plt.show()
print("Displayed violin plots.")

## 7. Parallel Coordinates Plot (Plotly)
# rfm_scaled_df['CustomerID'] = rfm['CustomerID']
# fig = px.parallel_coordinates(

```

```

        # rfm_scaled_df,
        # color='Cluster',
        # dimensions=['Recency', 'Frequency', 'Monetary'],
        # title=f'Parallel Coordinates of Standardized RFM by Cluster',
        ↪({method})',
        # color_continuous_scale=px.colors.diverging.Tealrose
        # )
        # fig.show()
        # print("Displayed parallel coordinates plot.")

    except Exception as e:
        print(f"Error during visualization: {e}")

```

```

[40]: # Summarizing RFM per cluster
def summarize_rfm_per_cluster(rfm, labels):
    rfm['Cluster'] = labels
    summary = rfm.groupby('Cluster').agg({
        'Recency': ['mean', 'std'],
        'Frequency': ['mean', 'std'],
        'Monetary': ['mean', 'std'],
        'CustomerID': 'count'
    }).round(2)
    summary.columns = ['Recency_Mean', 'Recency_Std', 'Frequency_Mean', ↪
        ↪'Frequency_Std',
        'Monetary_Mean', 'Monetary_Std', 'Count']

    return summary

```

```

[41]: # Applying K-Means
kmeans_labels, kmeans_model, kmeans_sil_score = apply_kmeans(rfm_scaled, ↪
    ↪optimal_k)

```

```

[42]: rfm = rfm.copy()
rfm['Cluster'] = kmeans_labels.astype(str) # Convert labels to string for ↪
    ↪better coloring
method = 'KMeans'

```

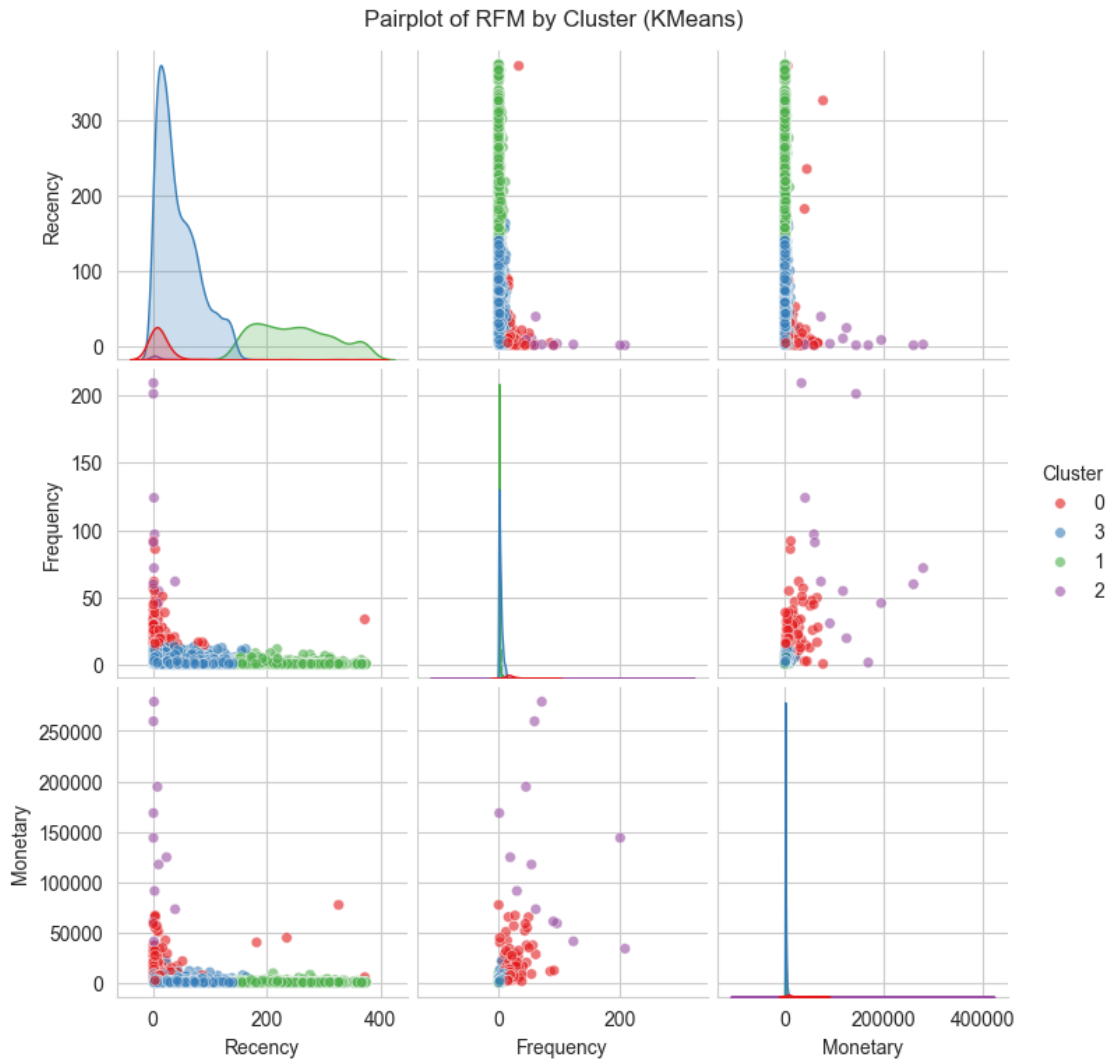
```

[43]: # 1. Pairplot (Seaborn)
plt.figure(figsize=(10, 8))
sns.pairplot(
    rfm,
    vars=['Recency', 'Frequency', 'Monetary'],
    hue='Cluster',
    diag_kind='kde',
    palette='Set1',
    plot_kws={'alpha': 0.6, 's': 30}
)
plt.suptitle(f'Pairplot of RFM by Cluster ({method})', y=1.02)

```

```
plt.show()
print("Displayed pairplot.")
```

<Figure size 1000x800 with 0 Axes>



Displayed pairplot.

```
[44]: # 2. Cluster Profile Bar Plot (Plotly)
cluster_means = rfm.groupby('Cluster')[['Recency', 'Frequency', 'Monetary']].
    .mean().reset_index()
fig = go.Figure()
for metric in ['Recency', 'Frequency', 'Monetary']:
    fig.add_trace(go.Bar(
        x=cluster_means['Cluster'],
        y=cluster_means[metric],
```

```

        name=metric
    ))
    fig.update_layout(
        title=f'Cluster Profiles: Mean RFM Values ({method})',
        xaxis_title='Cluster',
        yaxis_title='Value',
        barmode='group',
    )
    fig.show()
print("Displayed cluster profile bar plot.")

```

Displayed cluster profile bar plot.

Cluster Profile Bar Plot (Plotly) Description: Bar plot comparing mean Recency, Frequency, and Monetary across clusters, with bars grouped by cluster. Significance: Highlights average RFM values per cluster, making it easy to compare segment characteristics. It shows which metrics define each cluster (e.g., high Monetary for Cluster 0).

Clustering Choice: Distinct bar heights (e.g., Cluster 0: low Recency, high Monetary vs. Cluster 2: high Recency, low Monetary) indicate good clustering.

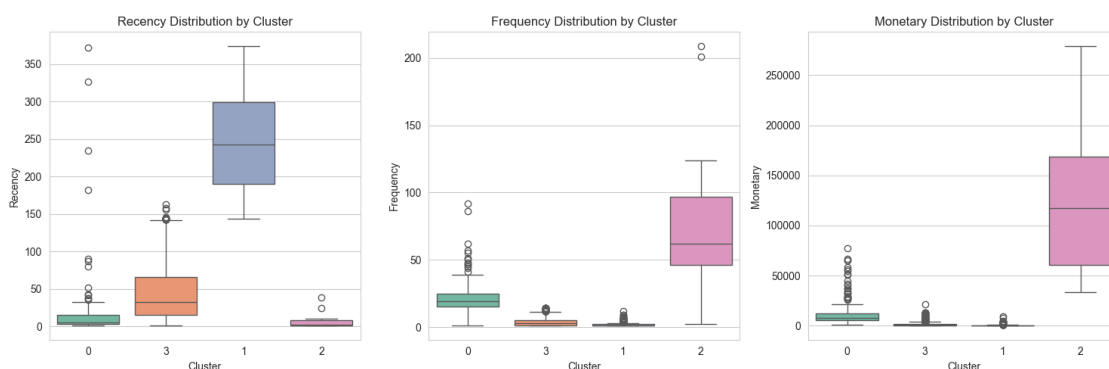
Business Insight: High-Value customers ie Cluster 0 and Cluster 2 need loyalty programs (e.g., VIP discounts in France), while At-Risk customers (Cluster 1) need re-engagement (e.g., €5 vouchers in Germany).

[45]: `# 3. Box Plots (Seaborn)`

```

fig, axes = plt.subplots(1, 3, figsize=(15, 5))
for idx, metric in enumerate(['Recency', 'Frequency', 'Monetary']):
    sns.boxplot(x='Cluster', y=metric, hue='Cluster', data=rfm, ax=axes[idx],
        palette='Set2')
    axes[idx].set_title(f'{metric} Distribution by Cluster')
plt.tight_layout()
plt.show()

```

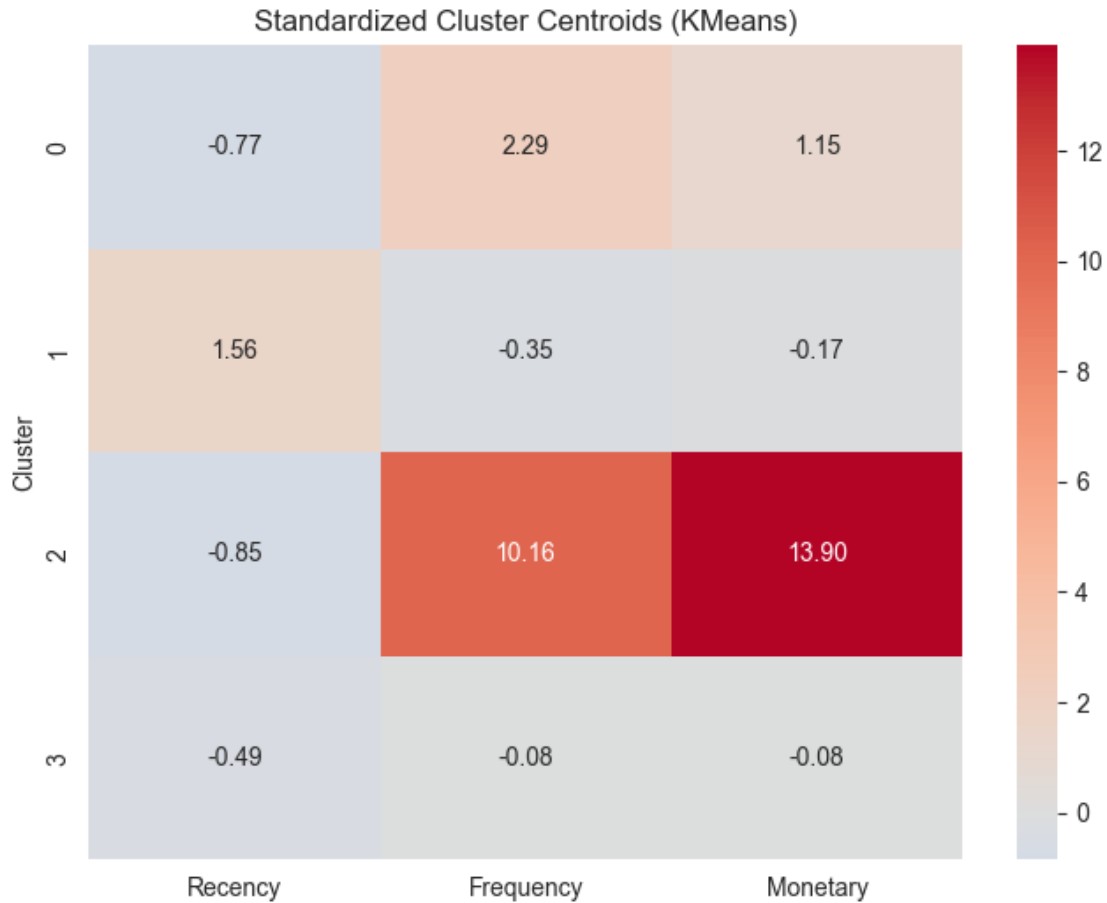


BOX Plots Description: Three box plots (one for each RFM metric) showing median, quartiles, and outliers per cluster. Significance: Displays the spread and variability of RFM metrics within each cluster. Identifies outliers (e.g., a customer in Cluster 2 with unusually high Monetary). Highlights median differences across clusters.

Business Insight: Clusters with wide Monetary spread (e.g., Cluster 2) may include diverse Loyal customers, suggesting targeted upsell campaigns (e.g., bundle offers in Germany).

```
[46]: # 4. Heatmap of Standardized Centroids (Seaborn)
scaler = StandardScaler()
rfm_scaled = scaler.fit_transform(rfm[['Recency', 'Frequency', 'Monetary']])
rfm_scaled_df = pd.DataFrame(rfm_scaled, columns=['Recency', 'Frequency', 'Monetary'])
rfm_scaled_df['Cluster'] = rfm['Cluster']
centroids = rfm_scaled_df.groupby('Cluster')[['Recency', 'Frequency', 'Monetary']].mean()

plt.figure(figsize=(8, 6))
sns.heatmap(
    centroids,
    annot=True,
    cmap='coolwarm',
    center=0,
    fmt='.2f'
)
plt.title(f'Standardized Cluster Centroids ({method})')
plt.show()
print("Displayed centroid heatmap.")
```



Displayed centroid heatmap.

Heatmap of Standardized Centroids (Seaborn) Description: Heatmap of standardized (z-scored) RFM means per cluster, with color intensity (red for positive, blue for negative) and annotations. Significance: Shows relative differences in RFM metrics across clusters on a standardized scale. Highlights defining traits (e.g., negative Recency, positive Monetary for High-Value). Easy to interpret for non-technical stakeholders.

Business Insight: Cluster 0's strong positive Monetary z-score suggests high-spending customers, ideal for premium offers (e.g., exclusive products in France).

```
[47]: # 5. 2D Scatter Plots (Plotly)
for x_metric, y_metric in [ ('Recency', 'Monetary'), ('Recency', 'Frequency'),
                             ('Frequency', 'Monetary') ]:
    fig = px.scatter(
        rfm,
        x=x_metric,
        y=y_metric,
        color='Cluster',
```

```

    title=f'{x_metric} vs. {y_metric} by Cluster ({method})',
    hover_data=['CustomerID'],
    color_discrete_sequence=px.colors.qualitative.Bold,
    opacity=0.6
)
fig.show()
print(f"Displayed 2D scatter plot: {x_metric} vs. {y_metric}")

```

Displayed 2D scatter plot: Recency vs. Monetary

Displayed 2D scatter plot: Recency vs. Frequency

Displayed 2D scatter plot: Frequency vs. Monetary

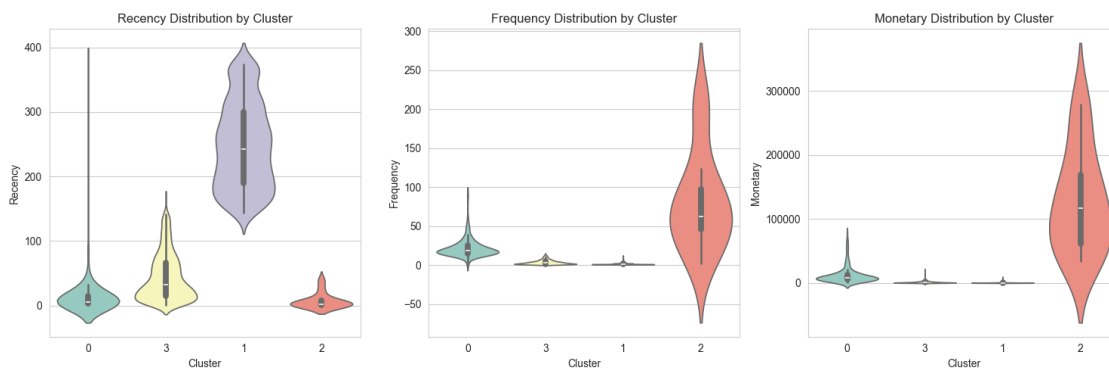
2D Scatter Plots (Plotly, for Recency vs. Monetary, Recency vs. Frequency, Frequency vs. Monetary) Description: Three interactive scatter plots showing pairs of RFM metrics, colored by cluster, with hover data (CustomerID).

Business Insight: Clear separation in Recency vs. Monetary (e.g., Cluster 1 at high Recency, low Monetary) supports re-engagement campaigns for At-Risk customers (e.g., Cluster 1).

```

[48]: # 6. Violin Plots (Seaborn)
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
for idx, metric in enumerate(['Recency', 'Frequency', 'Monetary']):
    sns.violinplot(x='Cluster', y=metric, hue='Cluster', data=rfm,
        ax=axes[idx], palette='Set3')
    axes[idx].set_title(f'{metric} Distribution by Cluster')
plt.tight_layout()
plt.show()
print("Displayed violin plots.")

```



Displayed violin plots.

Description: Three violin plots showing the density and distribution of Recency, Frequency, and Monetary per cluster.

Combines box plot (median, quartiles) and KDE to show distribution shape and density. Highlights multimodal distributions or skewness (e.g., Monetary in Cluster 3 may be skewed). Reveals variability and density differences across clusters. Relevance to Your Problem: Clustering Choice: Distinct violin shapes (e.g., Cluster 0's narrow Monetary violin at high values) suggest good clustering. Skewed distributions (e.g., Cluster 2's Monetary) may indicate diverse Emerging customers Business Insight: Wide violins in Cluster 2 (Loyal) suggest varied spending, ideal for personalized upsell offers (e.g., Germany).

1.9 KMeans Clustering Summary

```
[53]: kmeans_summary = summarize_rfm_per_cluster(rfm.copy(), kmeans_labels)
print("\nK-Means RFM Summary per Cluster:")
print(kmeans_summary)
```

K-Means RFM Summary per Cluster:

	Recency_Mean	Recency_Std	Frequency_Mean	Frequency_Std	\
Cluster					
0	15.57	40.11	21.89	11.56	
1	248.39	66.17	1.55	1.07	
2	7.38	11.49	82.31	63.39	
3	43.92	36.15	3.64	2.81	

	Monetary_Mean	Monetary_Std	Count
Cluster			
0	12345.37	12964.55	213
1	474.72	626.59	1064
2	126602.56	79979.48	13
3	1329.76	1504.99	3048

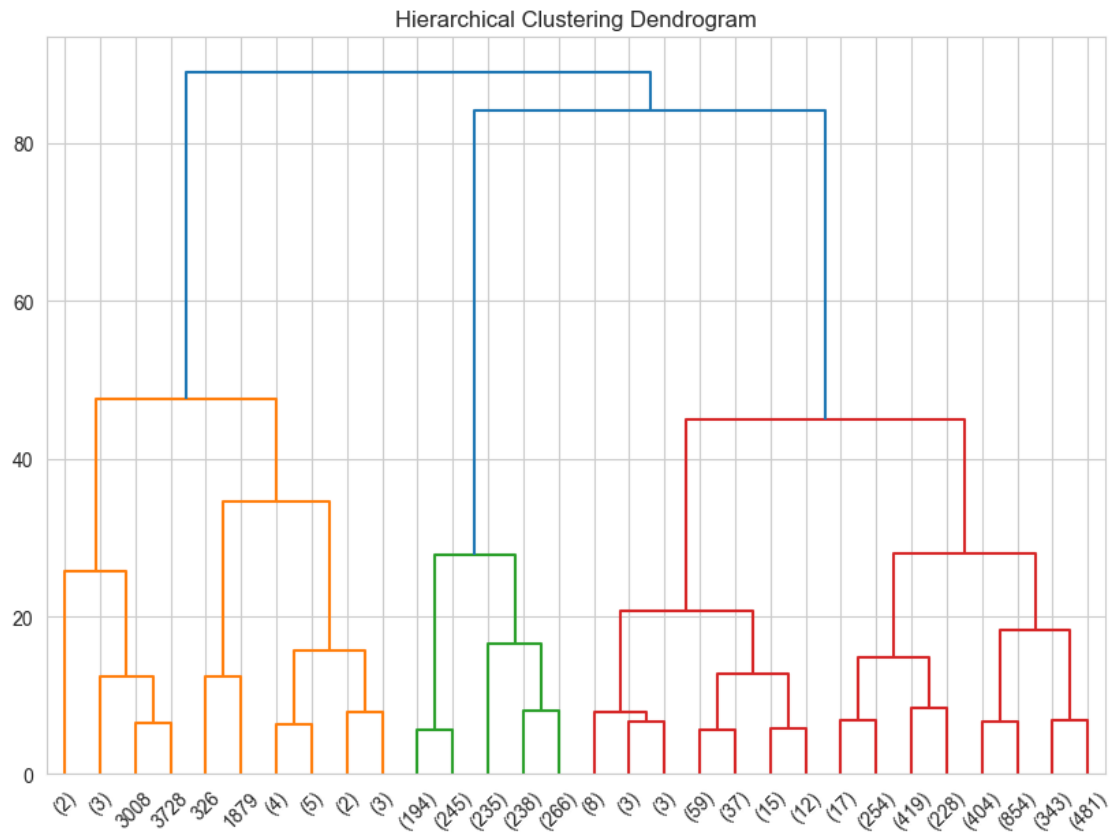
1.10 Hierarchical Clustering Visualization

```
[54]: # Applying hierarchical clustering
def apply_hierarchical(rfm_scaled, n_clusters):
    Z = linkage(rfm_scaled, method='ward')
    labels = fcluster(Z, n_clusters, criterion='maxclust')
    sil_score = silhouette_score(rfm_scaled, labels)

    # Plotting dendrogram
    plt.figure(figsize=(10, 7))
    dendrogram(Z, truncate_mode='lastp', p=30)
    plt.title('Hierarchical Clustering Dendrogram')
    plt.show()

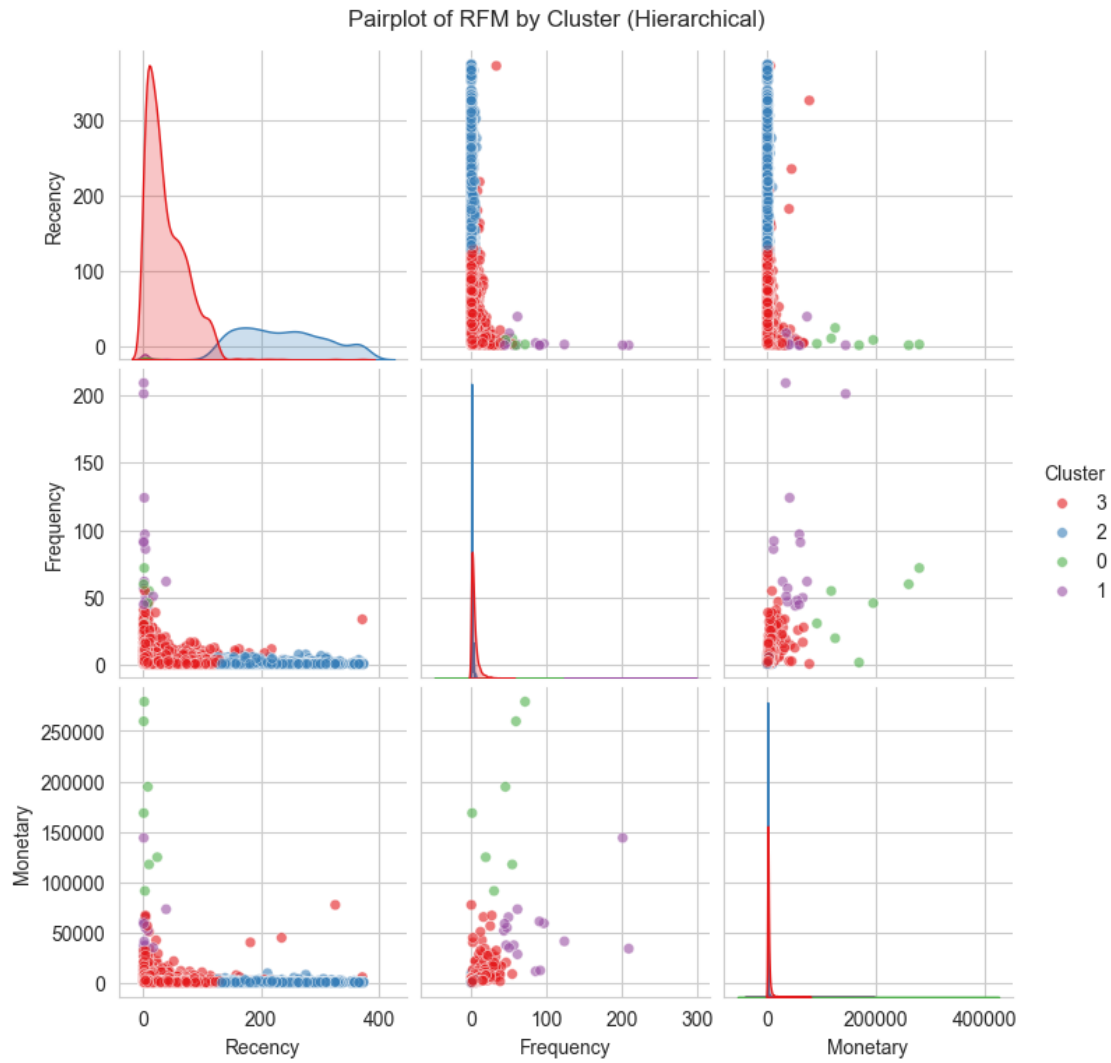
    return labels - 1, sil_score # Adjusting labels to start from 0
```

```
[55]: # Applying hierarchical clustering
hier_labels, hier_sil_score = apply_hierarchical(rfm_scaled, optimal_k)
```



```
[56]: visualize_clusters(rfm.copy(), hier_labels, 'Hierarchical')
```

<Figure size 1000x800 with 0 Axes>

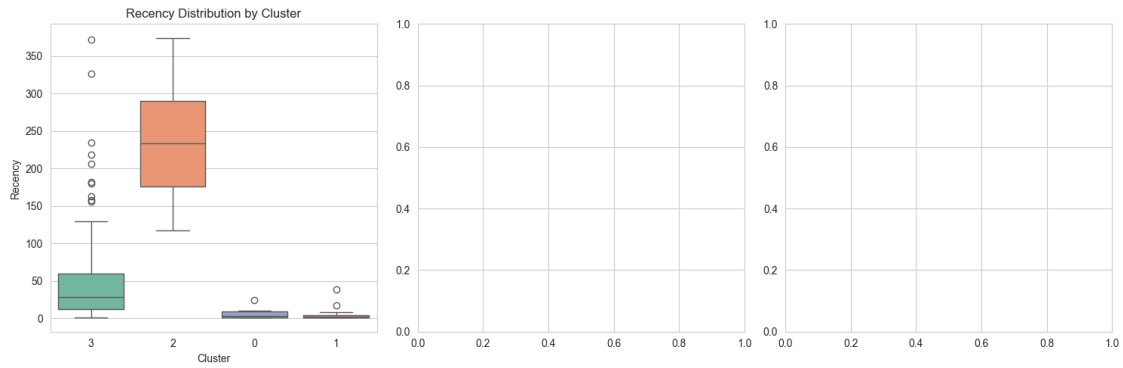


Displayed pairplot.

Displayed cluster profile bar plot.

Displayed cluster profile bar plot.

Displayed cluster profile bar plot.



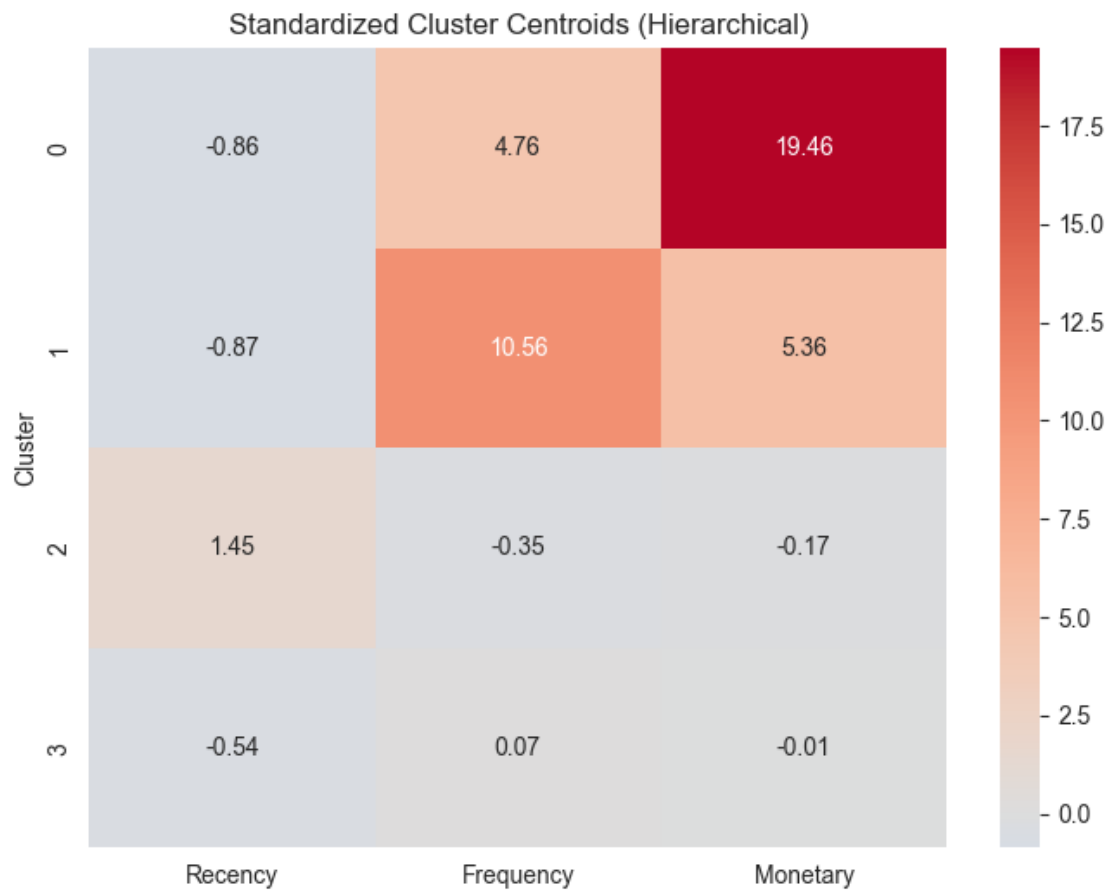
Displayed box plots.

<Figure size 1000x600 with 0 Axes>

Displayed box plots.

<Figure size 1000x600 with 0 Axes>

Displayed box plots.

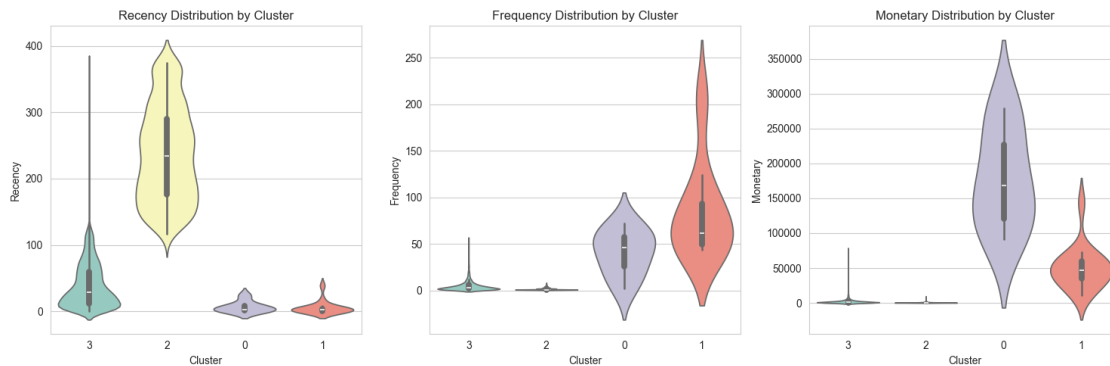


Displayed centroid heatmap.

Displayed 2D scatter plot: Recency vs. Monetary

Displayed 2D scatter plot: Recency vs. Frequency

Displayed 2D scatter plot: Frequency vs. Monetary



Displayed violin plots.

1.11 Hierarchical Clustering Summary

```
[57]: hier_summary = summarize_rfm_per_cluster(rfm.copy(), hier_labels)
print("\nHierarchical Clustering RFM Summary per Cluster:")
print(hier_summary)
```

Hierarchical Clustering RFM Summary per Cluster:

	Recency_Mean	Recency_Std	Frequency_Mean	Frequency_Std	\
Cluster					
0	7.00	8.29	40.86	24.51	
1	5.81	9.73	85.38	52.22	
2	237.20	71.51	1.59	1.03	
3	38.89	33.46	4.77	5.25	

	Monetary_Mean	Monetary_Std	Count
Cluster			
0	176403.59	72263.41	7
1	50058.26	30746.14	16
2	505.13	653.09	1178
3	1977.31	3979.83	3137

```
[58]: hier_labels
```

```
[58]: array([3, 3, 3, ..., 3, 3, 3], shape=(4338,), dtype=int32)
```

1.12 Compare SIL Score of both clustering methods

```
[59]: # Compare and decide
print("\nComparison and Decision:")
if kmeans_sil_score > hier_sil_score:
    print(f"K-Means is better (Silhouette Score: {kmeans_sil_score:.4f} > {hier_sil_score:.4f})")
    print("Recommendation: Use K-Means for RFM segmentation due to higher cluster quality.")
elif hier_sil_score > kmeans_sil_score:
    print(f"Hierarchical is better (Silhouette Score: {hier_sil_score:.4f} > {kmeans_sil_score:.4f})")
    print("Recommendation: Use Hierarchical clustering if nested structures are valuable.")
else:
    print(f"Equal Silhouette Scores ({kmeans_sil_score:.4f})")
    print("Recommendation: We would use K-Means for simplicity and scalability.")
```

Comparison and Decision:

K-Means is better (Silhouette Score: 0.6157 > 0.5806)

Recommendation: Use K-Means for RFM segmentation due to higher cluster quality.

1.13 Final Summary and clusters

```
[60]: # Display RFM summaries for the better method (K-Means by default if equal)
rfm['Cluster'] = kmeans_labels if kmeans_sil_score >= hier_sil_score else hier_labels
summary = rfm.groupby('Cluster').agg({
    'Recency': ['mean', 'std'],
    'Frequency': ['mean', 'std'],
    'Monetary': ['mean', 'std'],
    'CustomerID': 'count'
}).round(2)
summary.columns = ['Recency_Mean', 'Recency_Std', 'Frequency_Mean', 'Frequency_Std',
    'Monetary_Mean', 'Monetary_Std', 'Count']
print("\nRFM Summary for Chosen Method:")
display(summary)
```

RFM Summary for Chosen Method:

	Recency_Mean	Recency_Std	Frequency_Mean	Frequency_Std	\
Cluster					
0	15.57	40.11	21.89	11.56	
1	248.39	66.17	1.55	1.07	
2	7.38	11.49	82.31	63.39	

3	43.92	36.15	3.64	2.81
---	-------	-------	------	------

Cluster	Monetary_Mean	Monetary_Std	Count
0	12345.37	12964.55	213
1	474.72	626.59	1064
2	126602.56	79979.48	13
3	1329.76	1504.99	3048

```
[61]: # 5. 2D Scatter Plots (Plotly)
for x_metric, y_metric in [('Recency', 'Monetary'), ('Recency', 'Frequency'),
                           ('Frequency', 'Monetary')]:
    fig = px.scatter(
        rfm,
        x=x_metric,
        y=y_metric,
        color='Cluster',
        title=f'{x_metric} vs. {y_metric} by Cluster ({method})',
        hover_data=['CustomerID'],
        color_discrete_sequence=px.colors.qualitative.Bold,
        opacity=0.6
    )
    fig.show()
    print(f"Displayed 2D scatter plot: {x_metric} vs. {y_metric}")
```

Displayed 2D scatter plot: Recency vs. Monetary

Displayed 2D scatter plot: Recency vs. Frequency

Displayed 2D scatter plot: Frequency vs. Monetary

1.14 Business Insights

High-Value (e.g., Cluster 0): Bar plot/heatmap show low Recency, high Monetary → Offer VIP discounts (e.g., France).

At-Risk (e.g., Cluster 1): High Recency in scatters/box plots → Re-engagement emails (e.g., €5 vouchers in Norway).

Loyal (e.g., Cluster 2): Moderate RFM in bar plot, wide violin plots → Upsell bundles (e.g., Germany).

Emerging (e.g., Cluster 3): Moderate bars | Box plots in → Onboarding offers (e.g., free shipping).

Appendix 2: PowerPoint Presentation

Online Retails Sales: Customer Segmentation and Profiling

Probability and Statistics for Artificial Intelligence (AAI-500-IN2)



AA500:
Group 9



Meet the team

Nikhil Bembi

Team Lead

Led the work on the k-means and hierarchical clustering model.

Ms. Nehpreet Kaur

Team Member

Led the documentation and preparation of final submissions.

Tamayi Mlanda

Team Member

Led the work on the EDA and RFM model.

Dataset Selection

We chose to work with the Online Retail sales dataset which is found at:

UCI Repository Dataset:

<https://archive.ics.uci.edu/dataset/352/online+retail>

Project GitHub:

https://github.com/nbemb/AA_500_USD_Group9_CustomerSegmentation/tree/dev





Problem

Dataset selection

We chose the Online Retail dataset for a UK based online sales company. During our EDA, we formulated a number of research questions and possible areas for investigation.

Market gap

We have noticed that many customers are not returning to buy from the shop. We would like to identify the different groups of customers, understand their buying behavior and develop mitigation strategies to strengthen the relationships with these customers.

Customers

Customers are coming from different countries, especially the United Kingdom.

Financials

The average Customer spends an average of GBP 20.03 per order

Solution



Understand the Data

Perform EDA to understand the dataset and check for data quality and statistical metrics and properties of the data.



Customer Segments

Perform a RFM analysis to understand the different customer groups.



Data Modelling

Develop statistical models that help to understand the data and customer segments better.



Advise the Business

Provide recommendations and solutions to the business for strategies they can follow to improve sales and customer retention.

Recency, Frequency and Monetary Value (RFM) Analysis

Quartiles

Quartile-based scoring (1 - 4) will be used for simplicity and interpretability.

Distribution of RFM Scores

Understand the proportion of high-value vs. low-value customers, which is critical for marketing strategies

Recency vs Monetary Analysis

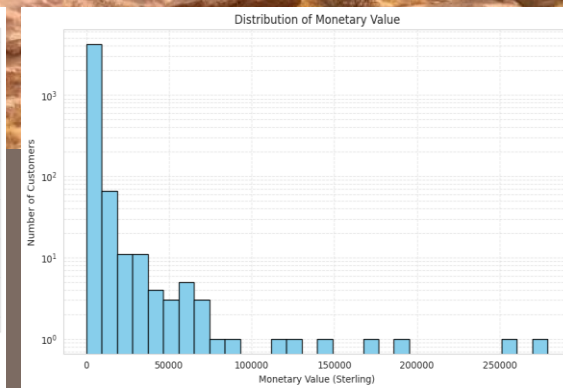
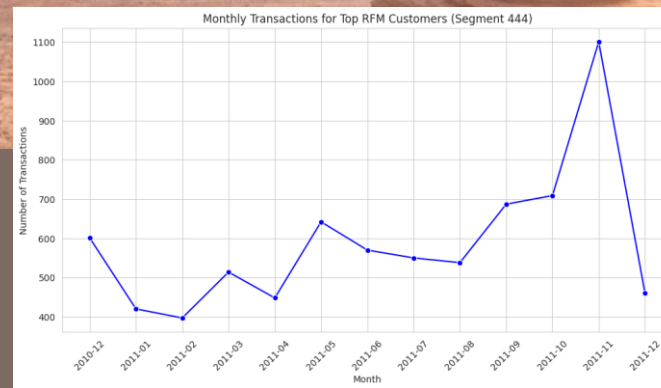
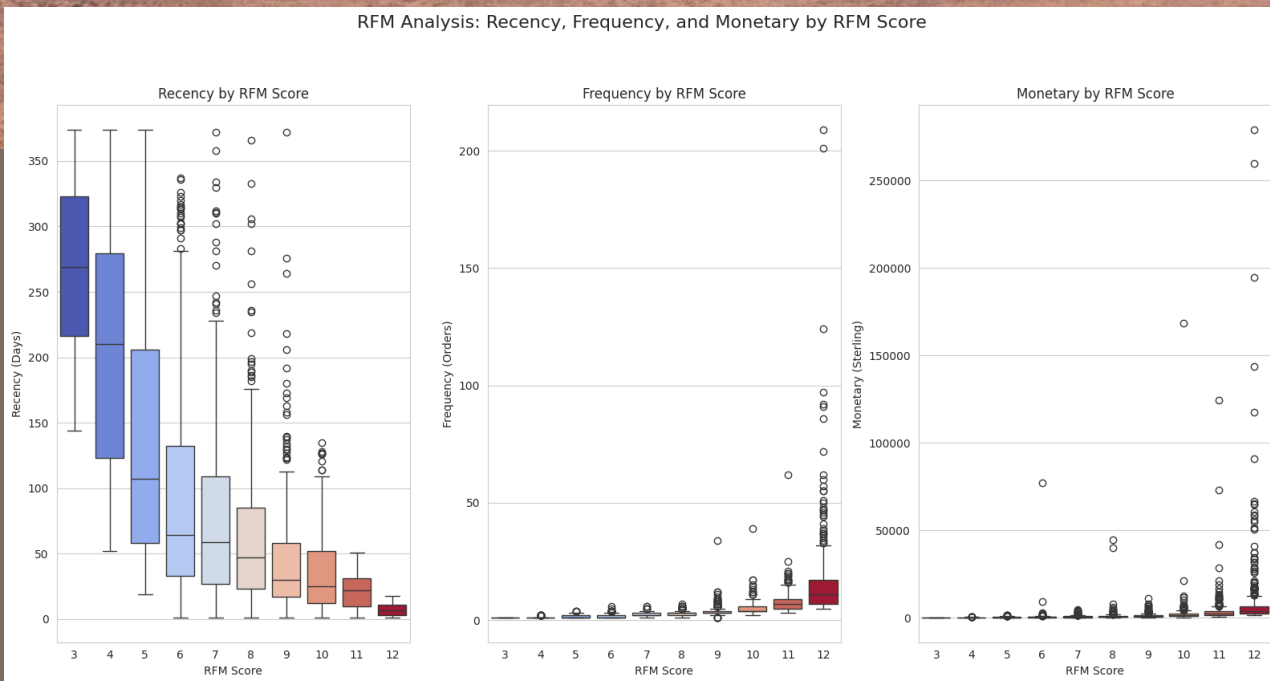
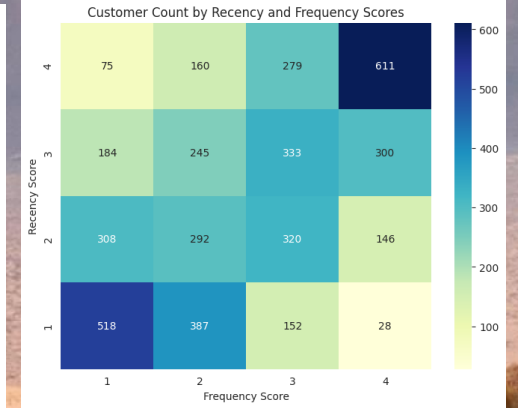
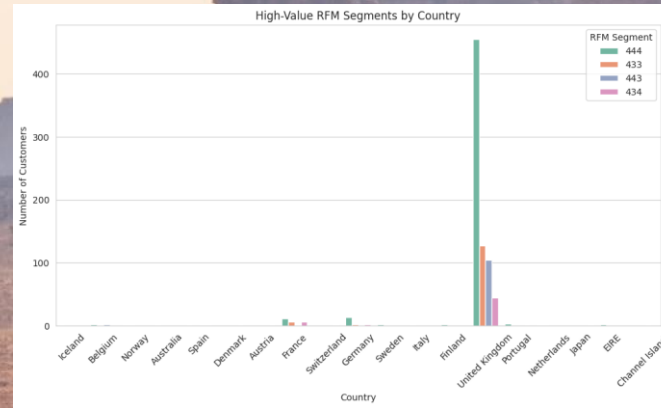
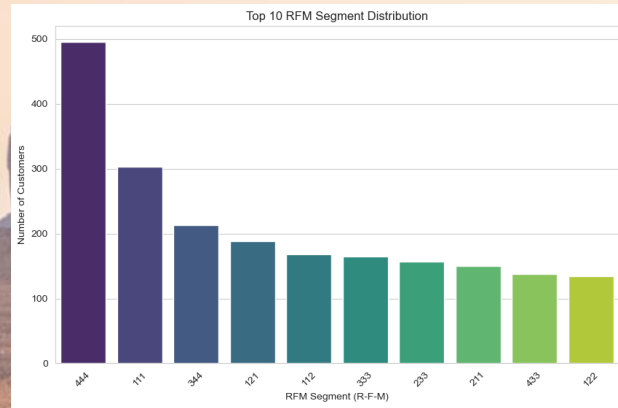
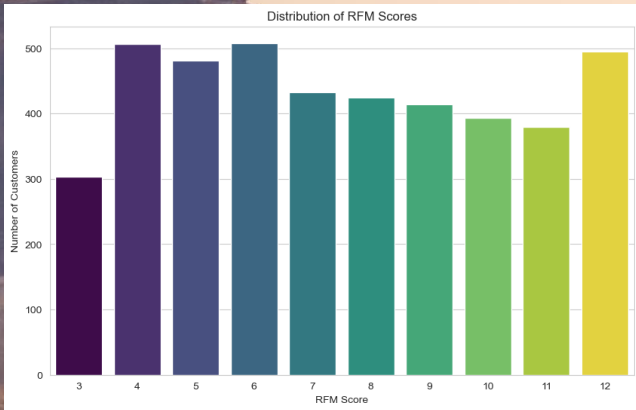
Insights into customer behaviour based on Recency (days since last purchase) and Monetary value (total spend).

Customer Spending Patterns

Understand the behaviour our customers spending patterns.



RFM Analysis Outputs



A comprehensive understanding of the customer behaviour from multiple perspectives. From this, we can start deriving some recommendations and build further statistical models.

Business Strategy & Recommendations

Growth

Opportunity to identify customers that should be prioritized for retention through loyalty programs or exclusive offers.

Re-engagement

Re-engagement campaigns targeting these customers could prevent churn. Upselling or increasing purchase frequency could move them toward higher scores.

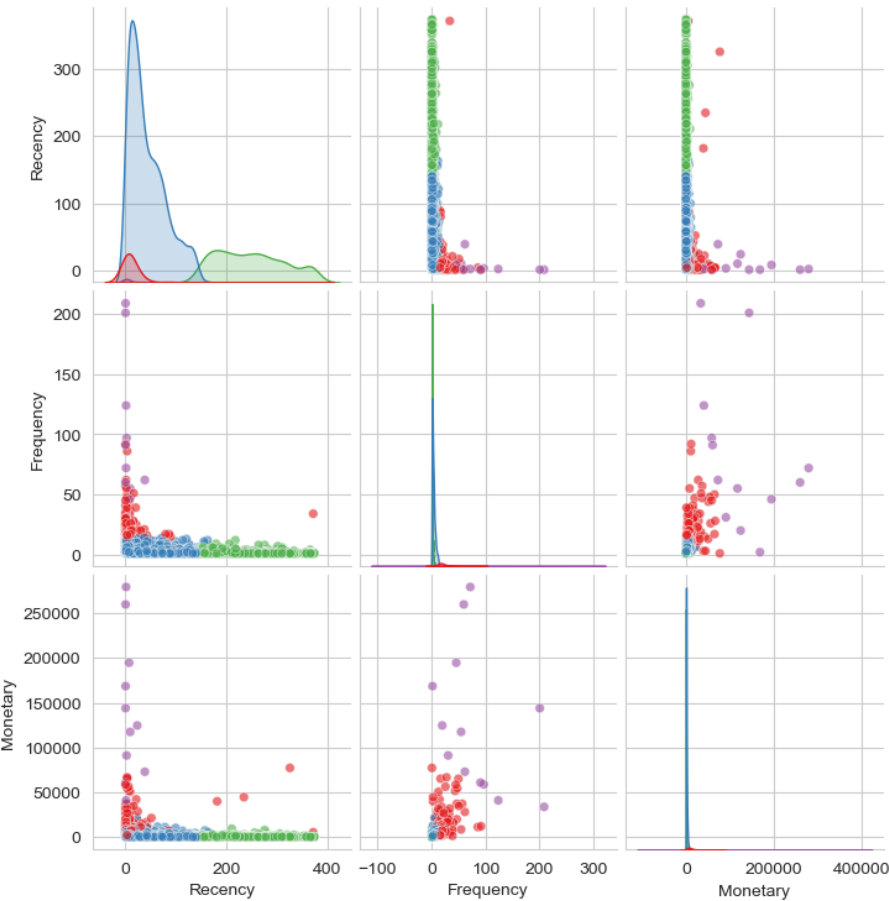
Deep-Dive

More detailed analysis required. This is the middle ground of customers with moderate recency, frequency, or monetary value.



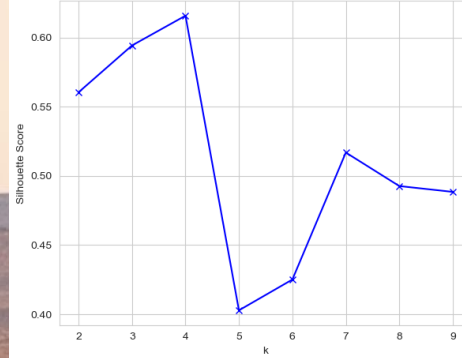
K-Means Clustering Outputs

Pairplot of RFM by Cluster (KMeans)

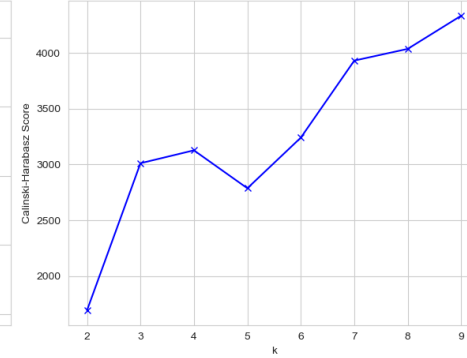


Cluster
● 0
● 3
● 1
● 2

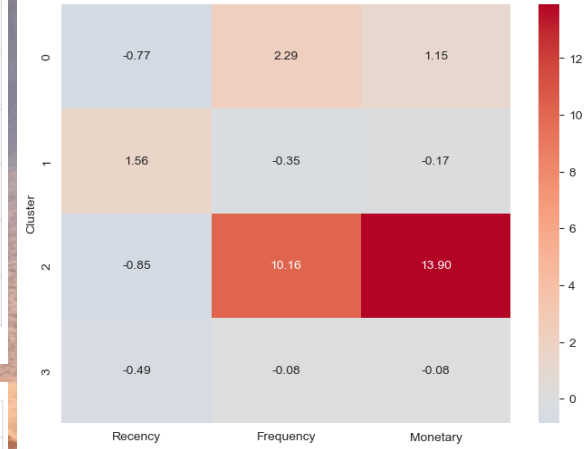
Silhouette Score vs. k



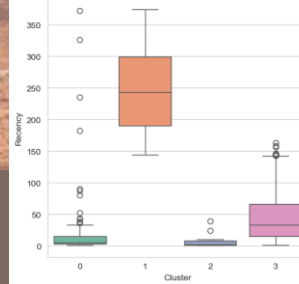
Calinski-Harabasz Score vs. k



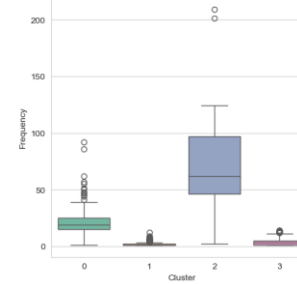
Standardized Cluster Centroids (KMeans)



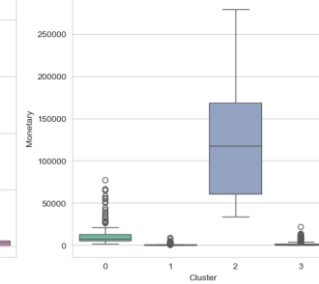
Recency Distribution by Cluster



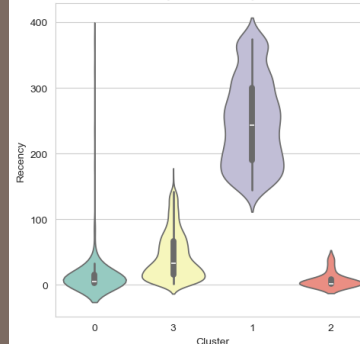
Frequency Distribution by Cluster



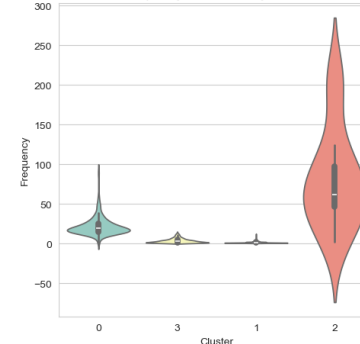
Monetary Distribution by Cluster



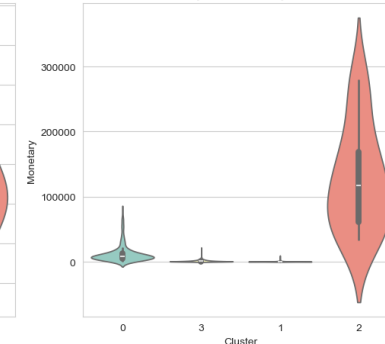
Recency Distribution by Cluster



Frequency Distribution by Cluster

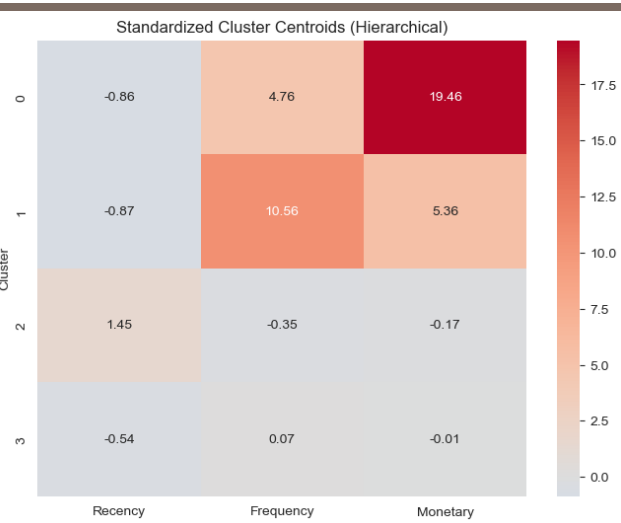
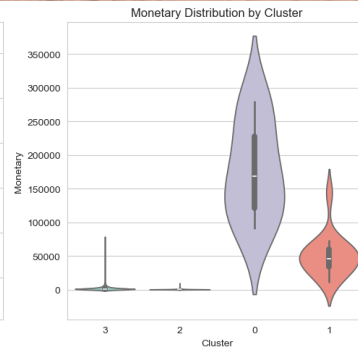
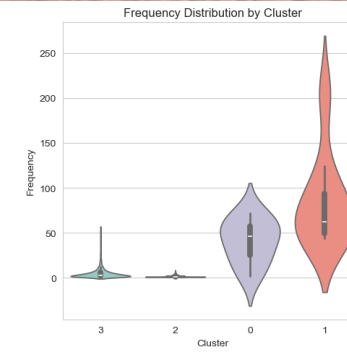
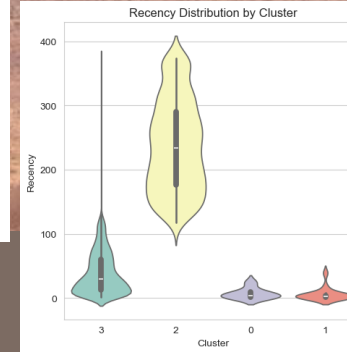
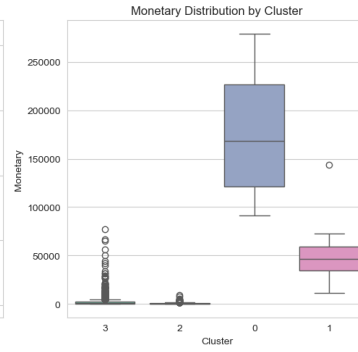
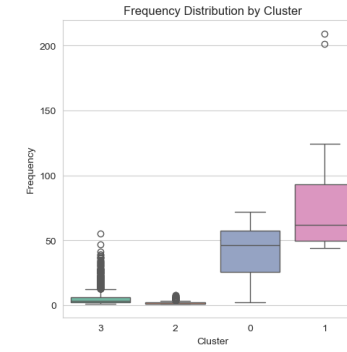
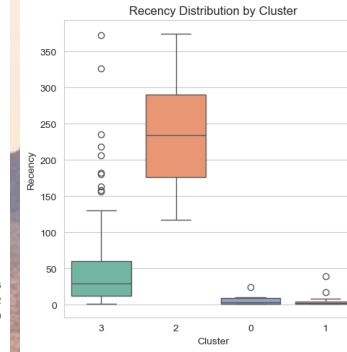
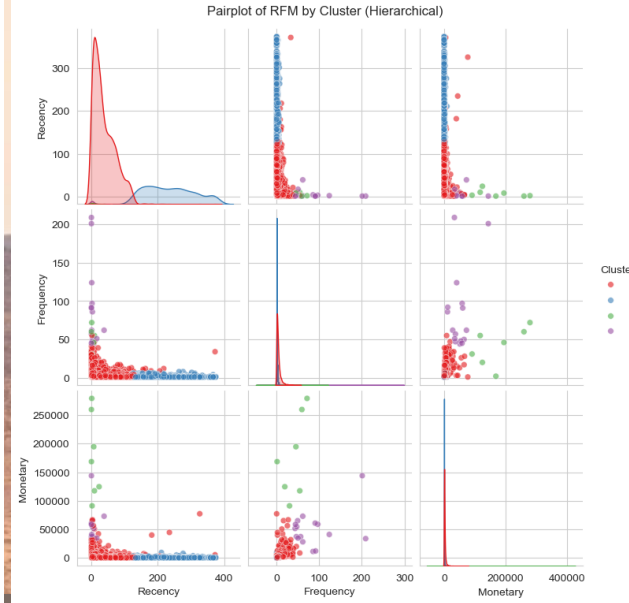
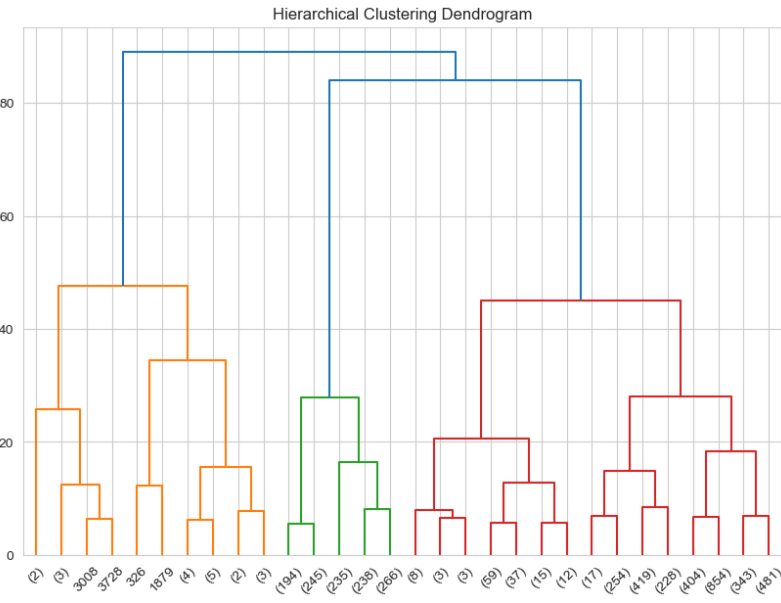


Monetary Distribution by Cluster



A comprehensive understanding of the customer behavior from multiple perspectives. From this, we can start deriving some recommendations and build further statistical models.

Hierarchical Clustering Outputs



A comprehensive understanding of the customer behavior from multiple perspectives.

From this, we can start deriving some recommendations and build further statistical models.

Findings

Through this analysis, we have identified some customer segments and the business strategies that we recommend for the business.

High-Value (e.g., Cluster 0): Bar plot/heatmap show low Recency, high Monetary → Offer VIP discounts (e.g., France).

At-Risk (e.g., Cluster 1): High Recency in scatters/box plots → Re-engagement emails (e.g., €5 vouchers in Norway).

Loyal (e.g., Cluster 2): Moderate RFM in bar plot, wide violin plots → Upsell bundles (e.g., Germany).

Emerging (e.g., Cluster 3): Moderate bars | Box plots in → Onboarding offers (e.g., free shipping).



Thank you

AA500:
Group 9

